

ONE DRAFTER, MANY LoRAs: SHARED DRAFTING THROUGH FEATURE ADAPTATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-LoRA services share a language-model backbone across specializations, but speculative decoding introduces a second adaptation problem: how should one shared drafter follow different target behaviors? We study specialization at the target-feature interface while freezing the pretrained draft transformer. Five identity-initialized linear maps train through the frozen drafter and fold into its fusion matrix for inference. A 42-cell architecture/objective comparison and 96-cell supervision-budget study characterize when this interface is useful and when body adaptation or decaying cross-entropy is stronger. On three Qwen3-4B LoRA workloads, the selected interface improves pooled standalone throughput by 30.6%, 87.5% and 5.3% over native DFlash. Shared-serving evidence distinguishes a rank-56 fusion bank, improving twelve-client throughput by 31.8–32.9%, from a newer five-map implementation with larger observed throughput gains. A pooled fusion adapter nearly matches separate experts, limiting claims that specialization must be target-specific. Mechanistic interventions distinguish identity preservation, directional correction and optimization geometry; they do not support simple cancellation of the target LoRA’s feature shift. The contribution is a measured adaptation and deployment strategy, not a new speculative verifier or acceptance-aware loss.

1 INTRODUCTION

LoRA makes it practical to offer several specialized assistants from one language-model backbone (Hu et al., 2021). The same server can host mathematical reasoning, circuit design and coding adapters without replicating the target’s full weights. Speculative decoding introduces a further deployment decision. A fast draft model proposes a block, and the selected target verifies it (Leviathan et al., 2022; Chen et al., 2023a). If the drafter follows one specialization poorly, the server spends computation proposing tokens that will be discarded. Maintaining a separate draft transformer for every specialization would erode the sharing that makes multi-LoRA serving attractive.

Draft adaptation is already a studied problem. Efficient Draft Adaptation separates shared and private draft components (Lin et al., 2026); OmniDraft adapts a drafter online across target vocabularies (Ramakrishnan et al., 2025). We ask a narrower architectural question: *how much specialization can be placed entirely in the interface that converts target features into draft context?* DFlash conditions a parallel block drafter on intermediate target states (Chen et al., 2026). Those states already reflect the active target LoRA. Changing how the frozen draft body reads them may improve its proposals without changing the body’s parameters.

Our primary parameterization inserts five full linear maps before DFlash’s frozen fusion blocks. Identity initialization preserves the native starting function. After training, matrix multiplication folds the maps into the existing projection: standalone inference adds no sequential mapping layer. A shared engine associates each request’s target identity with its corresponding fusion bank. This is localization of adaptation, not a claim of tiny parameter count: the 4B interface has 32.768 million trainable entries and training still differentiates through the frozen body.

We organize the evidence around three questions. First, does this interface compete with draft-body adaptation under matched data and optimization? Second, do extra training examples and proposal anchors justify their fitting cost? Third, can interface adaptation benefit a heterogeneous service once routing, batching and verification are included? We distinguish measured inference

contracts throughout. Serving demonstrates a 31.8–32.9% throughput improvement for a rank-56 fusion bank; newer five-map serving measurements broaden concurrency and model size under their own execution configuration.

Three findings constrain the story. The full interface is useful, but body LoRA wins the default NanoCoder comparison and decaying CE wins default KiCad. A pooled rank-56 adapter approaches separate experts on the tested mixture, so the evidence favors sharing draft computation more strongly than requiring a private adapter for every target. Finally, the five-map mechanism study supports useful directional conditioning and parameterization-dependent updates, not an explanation that the mapper simply reverses the target LoRA. These qualifications define the contribution: an experimentally characterized, foldable interface for reusing draft computation, with explicit evidence about its limits.

2 METHOD: SPECIALIZE THE CONDITIONING, PRESERVE THE DRAFT BODY

2.1 FIXED TARGET VERIFICATION AND FEATURE-CONDITIONED PROPOSALS

Let T_e denote the base target with LoRA e active, and D_ϕ a pretrained drafter. In greedy speculative decoding, a draft block is verified by T_e , accepting a matching prefix and committing a verifier token (Leviathan et al., 2022; Chen et al., 2023a). Changing proposals leaves the intended verification rule unchanged.

At prefix position t , the target supplies five states $h_{it} \in \mathbb{R}^d$. The native interface concatenates them and applies frozen fusion $F = [F_1, \dots, F_5] \in \mathbb{R}^{d \times 5d}$, then frozen normalization N :

$$z_{0t} = \sum_{i=1}^5 F_i h_{it}, \quad u_{0t} = N(z_{0t}). \quad (1)$$

We learn five $W_i \in \mathbb{R}^{d \times d}$ and replace the conditioning by

$$z_t = \sum_{i=1}^5 F_i W_i h_{it}, \quad u_t = N(z_t), \quad W_i^{(0)} = I. \quad (2)$$

The target, embeddings/head, normalization and draft transformer remain frozen. Gradients traverse the frozen draft computation to update W_i . For Qwen3-4B, $d = 2560$ and $5d^2 = 32,768,000$ parameters; for 8B, $d = 4096$ and $5d^2 = 83,886,080$. The two sizes have different widths: the 8B study adapts its native 8B drafter, not the 4B draft transferred across widths.

At export, precompute

$$G = [F_1 W_1, \dots, F_5 W_5]. \quad (3)$$

Inference applies the original-shaped Gh . There is no additional mapper matrix multiplication. Multi-target serving retains one folded G_e per target specialization, one target backbone and one draft body. Per-row bank selection must follow request ownership; a global mutable matrix swap is insufficient under heterogeneous batching.

2.2 ARCHITECTURE CONTROLS

The matched study compares seven architectures under each loss. Five-W uses Eq. 2. A *joint* W instead learns $W \in \mathbb{R}^{5d \times 5d}$ on the concatenation, giving FW_h and $25d^2 = 163.84\text{M}$ parameters at 4B. A *full fusion residual* learns $\Delta F \in \mathbb{R}^{d \times 5d}$, giving $(F + \Delta F)h$, with the same 32.768M parameters as five-W. Its residual starts at zero.

Three *five-BA replacement* variants use $W_i = B_i A_i$ at ranks $r \in \{28, 56, 128\}$, with $A_i \in \mathbb{R}^{r \times d}$ and $B_i \in \mathbb{R}^{d \times r}$. Their effective scale is one; the implementation directly uses $B_i A_i$. Initialization uses cached training-feature PCA bases Q_i , with $B_i = Q_i$ and $A_i = Q_i^T$. Thus these start as rank- r projectors, not identity plus a small residual. They have $10dr$ parameters. Their failures do not establish that an identity-preserving low-rank residual would fail.

The competing *body LoRA* uses rank 128, alpha 256, zero dropout on all draft attention/MLP projections: q/k/v/o and gate/up/down. Fusion is excluded. It trains 36.700M parameters at 4B, exceeding the mapper’s count; we do not weaken it to match a smaller parameter budget. All exportable static changes are merged for standalone evaluation.

2.3 MASKED-BLOCK TRAINING AND TWO OBJECTIVES

We generate continuations from each active target and persist the exact token IDs and target features. Fitting uses those caches; the target does not decode again at each update. Given a response anchor a , a DFlash block has one clean anchor token and fifteen masked prediction positions. It sees prefix target features, with the pinned upstream block attention/masking contract. This is block prediction, not ordinary causal next-word CE on an unmasked token stream.

Let m_{bj} be the valid response-label mask, excluding the anchor and padding, and $\ell_{bj} = -\log q_{bj}(y_{bj})$. Positions are zero-based, with the known anchor at $j = 0$. The pinned upstream DFlash decaying CE is

$$v_{bj} = m_{bj} \exp \left[-\frac{\max(j-1, 0)}{7} \right], \quad L_{\text{decay}} = \frac{\sum_{b,j} v_{bj} \ell_{bj}}{\sum_{b,j} v_{bj}}. \quad (4)$$

The first masked slot has weight one. Predictions after a mistake continue to contribute, with exponential position decay.

AUF uses the published accept-until-fail support rule (Yang & Li, 2026), implemented here on top of pinned SpecForge:

$$c_{bj} = \mathbf{1}[\arg \max_v q_{bj}(v) = y_{bj} \vee m_{bj} = 0], \quad (5)$$

$$s_{bj} = \text{stopgrad} \left(\prod_{k < j} c_{bk} \right), \quad (6)$$

$$L_{\text{AUF}} = \frac{\sum_{b,j} m_{bj} s_{bj} \ell_{bj}}{\sum_{b,j} m_{bj} s_{bj}}. \quad (7)$$

The first failure remains supervised; only later positions are removed. The support is recomputed from current predictions and has no gradient. We introduce neither AUF nor DFlash; our experiments test these objectives across adaptation parameterizations. Chunk numerators/denominators are aggregated within microbatches before the implemented accumulation/DDP average, not as one global dataset ratio.

Why might support matter? If L is the number of consecutive accepted proposals out of K , then

$$\mathbb{E}[L] = \sum_{j=1}^K \Pr(L \geq j). \quad (8)$$

No independence assumption is required. Fixing an unreachable late slot can lower token CE without affecting the accepted prefix. AUF addresses that mismatch, but Eq. 8 does not prove that AUF maximizes speed: verifier cost, draft cost and batching also matter.

3 EXPERIMENTAL DESIGN

Targets and workloads. The 4B targets are the existing GSM8K math adapter, KiCad netlist adapter and NanoCoder adapter. Each uses its task-specific frozen training/evaluation cohort; the same 128 held-out prompts per domain are reused across comparisons. The 8B study uses published math and CodeAlpaca target adapters, with 4,096 unique training prompts and 128 evaluation prompts each. The 8B math adapter is published as OpenMathSFT adaptation; it is not claimed to have been trained on GSM8K. Adapter paths, pins and prompts are recorded in Appendix A.

Matched fitting. The initial 42 cells use 4,096 examples, 32 sampled anchor blocks, one epoch and a 4,096-token response cap. AdamW uses learning rate 10^{-4} , zero weight decay, norm clipping 1, 5% warmup then cosine decay, seed 42. Two GPUs, two examples per GPU and two accumulation steps give global batch 8 and 512 updates. The final checkpoint is used, with no dev-set selection. Fitting times exclude previously generated rollouts/features. Frozen-body backpropagation remains substantial, so parameter savings are not asserted to imply proportional compute savings.

Scaling. For five-W and body-r128, under both objectives, we sweep anchors $\{8, 24, 32, 64, 128\}$ at 4,096 examples and examples $\{4096, 8192, 12288, 16384\}$ at 32 anchors. The intersecting cell is counted once: eight cells per architecture/loss/domain, 96 total. Each fit starts independently and runs one epoch, giving 512/1,024/1,536/2,048 updates. This is not a full factorial interaction study or a validated scaling law. GSM8K repeats only after exhausting the compatible 4,096-example cache; KiCad uses its 15,366 unique cached examples before repetition at the largest budget; NanoCoder has sufficient unique examples. Requested anchors are bounded by available positions.

Inference and metrics. Standalone measurements use pinned DFlash Transformers, BF16/default SDPA on L40S, greedy decoding, thinking disabled and one request at a time. Response caps are 2,048/8,192/2,048 for math/KiCad/NanoCoder and 2,048 for both 8B domains. For n requests with output lengths o_i and times t_i , report

$$\overline{\text{TPS}} = \frac{1}{n} \sum_i \frac{o_i}{t_i}, \quad S = \frac{\overline{\text{TPS}}_{\text{method}}}{\overline{\text{TPS}}_{\text{reference}}}, \quad g = 100(S - 1). \quad (9)$$

These give each request equal weight. Aggregate serving throughput is $\sum_i o_i / T_{\text{workload}}$, a separate metric. Acceptance includes committed bonus tokens according to each upstream counter convention. Live vLLM acceptance is computed from timed counter deltas; DFlash standalone uses its recorded acceptance-length lists. We retain raw counts rather than pretending backend counter implementations are identical.

The evaluation cohort has been repeatedly inspected for architecture choice and probe design; findings are exploratory, despite its training holdout. Paired prompt bootstrap intervals quantify within-cohort timing uncertainty, not seed or hardware variability. Excluded scaling repeats do not support any claim in this paper.

4 RESULTS

4.1 A FULL INTERFACE IS USEFUL; NO OBJECTIVE WINS UNIVERSALLY

Table 1: Matched32-anchor, one-epoch results: gain (percent) over native DFlash. All use 4,096 training examples. Body r128 is a genuine competing baseline.

Domain	W/AUF	W/Decay	Body/AUF	Body/Decay	$\Delta F/\text{AUF}$	W fit min
GSM8K	37.58	36.84	36.41	34.82	32.36	2.66
KiCad	89.52	93.53	75.46	84.95	44.46	5.32
NanoCoder	8.07	3.11	9.82	8.65	8.15	2.76

The selected five-W AUF gains 37.58%, 89.52%, 8.07% over native on GSM8K, KiCad, NanoCoder. Its fits take 2.66, 5.32, 2.76 minutes, respectively. Under the same AUF recipe, body-r128 gains 36.41%, 75.46%, 9.82%. The mapper’s benefit over that body baseline is domain-dependent; NanoCoder is a counterexample to universal superiority. The body comparator has 36.700M parameters versus 32.768M for five-W, a modest difference rather than an order-of-magnitude advantage.

Decaying CE is also competitive. Five-W CE reaches 36.84%, 93.53%, 3.11% native-relative gains. AUF improves the math and coding entries at this recipe but loses on KiCad. Consequently the deployment results establish the usefulness of the chosen AUF interface, not that this choice is the optimal objective for every domain.

The parameterization controls matter. Direct full fusion adaptation is relatively competitive on NanoCoder but much weaker than five-W on KiCad. Joint W has five times as many parameters yet is weaker than five-W in the matched cells. Low-rank replacement severely hurts math and coding, including negative native-relative gains. Because it deletes identity directions at initialization, this result should not be read as evidence against all low-rank adaptation. Appendix B gives all 42 cells, including negative outcomes and every fit cost.

Dense reconstruction preserves the coverage study’s conclusion. We fit five-W using normalized intermediate-feature and post-RMSNorm context reconstruction on 4,096 examples for one

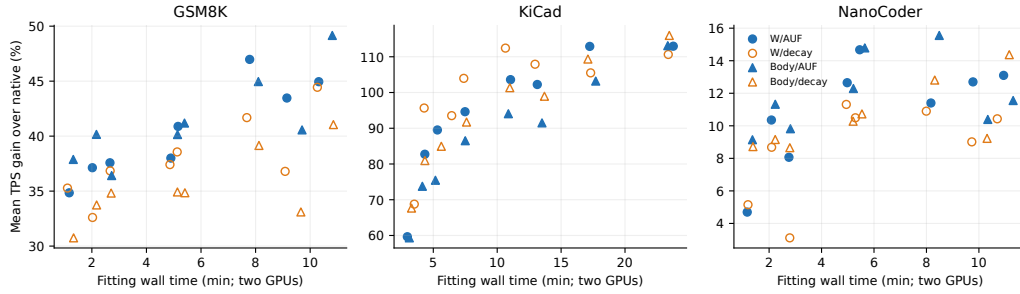


Figure 1: Observed throughput gain versus fitting time across all 96 cells. Both objectives and both architectures are retained, including dominated points. Curves describe this experiment; cache preparation is excluded and small ordering differences are not confirmed Pareto improvements.

Selection	Repeat	Native tok/s	Bank tok/s	Ratio
Explicit ID	0	1273.98	1678.61	1.3176
Explicit ID	1	1273.32	1691.96	1.3288
Learned	0	1280.52	1691.87	1.3212
Learned	1	1285.61	1702.59	1.3243

Table 2: rank-56 bank serving, one L40S per engine. Fits cost 3.83/10.51/4.54 two-GPU minutes for Math/KiCad/NanoCoder, excluding data preparation. These are repeated measurements, not confidence intervals.

epoch. With every prompt and response token supervised, mean throughput changes versus native DFlash are -1.83% , $+20.50\%$, and -1.37% for GSM8K, KiCad, and NanoCoder. Fitting takes 34.12, 381.61, and 37.45 seconds on two GPUs, excluding paired feature extraction and setup. Acceptance is nearly unchanged from 50% coverage: 4.4696 versus 4.4665, 7.2064 versus 7.2083, and 4.9742 versus 4.9573. Thus denser reconstruction does not close the gap to the tested AUF recipe; feature backend and supervision structure remain confounds. Appendix J retains all six coverage/domain results, the exact objective and timing limitations.

4.2 MORE SUPERVISION PRODUCES DOMAIN-DEPENDENT TRADEOFFS

The complete 96-cell sweep is in Appendix C. At 4,096 examples, increasing five-W AUF anchors from 8 to 128 changes native-relative gains from 34.85% to 43.47% on GSM8K, 59.65% to 102.24% on KiCad, and exposes much smaller coding improvements. At 32 anchors, the highest observed five-W AUF data-sweep gains are 46.98% on math (12,288 presentations), 112.96% on KiCad (16,384), and 14.67% on NanoCoder (8,192). They cost 7.78, 23.78, 5.46 fitting minutes, respectively.

These maxima are descriptive choices after observing the sweep. They are not new independently confirmed checkpoints or the banks used in serving. We keep the fixed 4,096-by 32 AUF banks for deployment, making serving comparisons interpretable. Body adaptation catches up or wins at several higher budgets, and decaying CE can define competitive points. We do not fit an asymptotic law to two intersecting one-dimensional sweeps.

4.3 SHARED SERVING: BENEFIT DEPENDS ON THE EXECUTION CONTRACT

An earlier deployment uses rank-56 residual fusion adaptation, $F_e = F + B_e A_e$, rather than five full maps. It retains the same shared-body principle but uses eight anchors, 2,000 AUF updates and 4,096 unique cached examples per domain. On the fixed three-domain mixture, twelve streaming clients yield the following aggregate rates; both repeats reverse native/mapped order.

A single pooled rank-56 correction reaches 98.94–99.60% of separate-bank aggregate throughput for explicit selection and 99.42–99.58% for learned selection. Thus these workloads do not establish a substantial need for per-target draft experts. The pooled control still requires the correct target LoRA;

it removes draft-adapter selection, not target selection. Appendix H supplies recipes, acceptance and pooled measurements.

The newer five-map deployment uses 384 requests at 4B and 256 at 8B, with explicit-ID clients 1/4/8/16/32 (capacity 32), plus a twelve-client learned-router test. The target’s request identity selects both target LoRA and draft fusion bank. No fitting is repeated across client counts. These optimized BF16 runs disable batch invariance; their output differences prevent treating their larger observed rates as lossless gains.

Table 3: Optimized five-map explicit-ID serving, one L40S. Mean-request gain and aggregate gain differ; outputs are not equal across methods. Same fits at every concurrency (4B:2.66/5.32/2.76min;8B:4.06/4.02min).

Target	Clients	Mean gain (%)	Native agg. TPS	W agg. TPS	Agg. gain (%)
4B	1	44.60	291.61	516.24	77.03
4B	4	46.93	889.11	1572.09	76.82
4B	8	43.75	1534.73	2689.53	75.24
4B	16	41.12	2097.75	3693.09	76.05
4B	32	43.77	2439.74	4323.96	77.23
8B	1	17.02	159.17	178.59	12.20
8B	4	15.66	525.87	593.43	12.85
8B	8	15.76	931.62	1043.65	12.03
8B	16	14.22	1482.33	1431.21	-3.45
8B	32	13.03	1569.56	1521.07	-3.09

At one client, AR, native DFlash and five-W achieve 64.39, 270.21 and 390.71 mean tokens/s: native is $4.196\times$ AR and the mapper is $6.067\times$ AR. The 4B mapper retains 41.12–46.93% higher mean request TPS than native across the concurrency sweep. Its aggregate throughput rises from 516 to 4,324 tokens/s while mean request TPS falls from 391 to 108 as clients share the GPU. These are different summaries, not contradictory measurements. The two-domain 8B engine gains 13.03–17.02% in mean request TPS, but its aggregate gain becomes negative at 16/32 clients. The learned router yields 43.82% improvement at 4B and 15.07% at 8B. Router accuracy is 383/384 and 255/256, respectively; this measures domain selection, not answer correctness.

Router fitting takes 0.889 CPU seconds for 12,288 labeled prompts at 4B and 3.233 seconds for 8,192 at 8B. All methods within a router comparison receive the same selected target assignments, so a routing error is not credited as draft speedup. The router is a frozen TF-IDF/logistic classifier, not a token-level mixture of expert weights. Main and operational tables, including TTFT, TPOT, latency, memory and fitting costs, appear in Appendix D.

Performance interpretation. Under an unchanged workload, a simplified Amdahl expression is $S = [(1 - p) + p/s]^{-1}$ when fraction p of baseline latency is accelerated by factors. We do not infer a measured p from acceptance alone. At the block level, cost per emitted token is approximately $(C_{\text{draft}} + C_{\text{verify}})/\tau$; improving τ need not translate linearly into end-to-end speed when costs, batching or scheduling change. Our folded mapper removes a separate mapping operation, but adds bank storage and dispatch. The serving experiment establishes a benefit on its measured workload; unchecked or mis curves remain descriptive. Neither extrapolates to thousands of adapters or arbitrary traffic.

4.4 TRANSFER TO 8B AND COMPARISON WITH EAGLE-3

At 8B, five-W AUF gains 35.1% over native DFlash on math and 6.2% on CodeAlpaca; it fits in 4.06 and 4.02 minutes, with 83.886M parameters per domain. This extends the deployment observation to another model width and two published target adapters, not a controlled 4B-to 8B causal scaling comparison: adapter identities and coding datasets differ.

We also evaluate published EAGLE-3 drafts (Li et al., 2025). Standalone uses its canonical custom-KV implementation and tree 60/depth 7/top-k 10, with a documented Qwen3 head-dimension compatibility patch. Its separate Transformers environment receives an AR companion; serving uses vLLM EAGLE with three speculative tokens versus DFlash’s fifteen. These are disclosed implementation/proposal differences, so the measurements do not establish a budget-matched algorithm ranking. Appendix E reports all standalone entries; the serving tables include EAGLE for every completed workload.

5 MECHANISM: DISTINGUISH REPRESENTATION, CAPACITY AND OPTIMIZATION

5.1 LOW-RANK MAPS AND LOW-RANK CORRECTIONS ARE DIFFERENT

Write $D_i = W_i - I$. Whole-W SVD truncation uses $[W_i]_r$, while identity-preserving truncation uses $I + [D_i]_r$. If $W_i = I + D_i$, a low-rank replacement is constrained to erase most input directions even when D_i itself is small. In contrast, retaining identity leaves the pretrained channel intact. This algebra explains why whole-map rank 1/2 failures cannot identify the intrinsic rank of the learned correction.

We measure live speed after folding every static intervention into the same dense fusion kernel. Define retained latency reduction

$$R_r = \frac{T_0 - T_r}{T_0 - T_*}, \quad T = \sum_i t_i, \quad (10)$$

where 0 is native and * is full mapper from the matching execution group: static, dynamic-hook, or repeat round. Native and full therefore define retention zero and one within each group. It is not a singular-value energy percentage. Whole-W rank 1/2 greatly reduce acceptance on math and NanoCoder. Identity-preserving ranks improve the outcome, but do not establish universal rank-two sufficiency. Math rank 128 reaches about 95.1% and 99.6% retention in two matched mechanistic timing rounds. NanoCoder rank 1024/2048 nearly preserve acceptance but not the initial full-run latency reduction. These discrepancies limit claims about a stable 98% compression threshold. No faster low-rank inference kernel is evaluated.

5.2 DIRECTION HELPS; A SIMPLE TARGET-LoRA REVERSAL DOES NOT FOLLOW

For native pre-norm z_0 and correction $\delta = z - z_0$, decompose

$$\delta_{\parallel} = \frac{\delta^T z_0}{\max(\|z_0\|^2, \epsilon_p)} z_0, \quad \delta_{\perp} = \delta - \delta_{\parallel}. \quad (11)$$

RMSNorm is $N(z) = g \odot z/s$, with $s = \sqrt{\|z\|^2/d + \epsilon}$ and Jacobian

$$J_N(z) = \text{diag}(g) (I/s - z z^T / (d s^3)). \quad (12)$$

The projection uses a denominator floor $\epsilon_p = 10^{-20}$, distinct from RMSNorm’s $\epsilon = 10^{-6}$. Positive rescaling is invariant when RMSNorm epsilon is zero, irrespective of the frozen gain; finite epsilon and arithmetic make it approximate. General finite corrections can also change direction or reverse sign. Dynamic hooks benchmark their own native/full endpoints. Radial-only acceptance stays near native (math 4.466 versus 4.472; NanoCoder 5.114 versus 5.111), whereas perpendicular-only approaches full (5.924 versus 5.988; 5.527 versus 5.552).

Using dynamic endpoints in Eq. 10, perpendicular-only retains 93.8% of the measured full time saving on math and 196.6% on NanoCoder. The latter is not stronger functional retention: NanoCoder’s full-hook saving is only 9.218 seconds, versus 18.118 seconds for perpendicular-only, despite slightly lower acceptance. Hook-full and static-full also have different times at matching reported acceptance. Thus the directionality conclusion rests on the acceptance interventions, not a stable latency-retention threshold. Appendix F reports matching controls and normalization explicitly.

To test literal target-LoRA reversal, teacher-force identical cached tokens through base and adapted targets offline. Set $b = \sum_i F_i h_i^{\text{base}}$, $z_0 = \sum_i F_i h_i^{\text{LoRA}}$, and $t = z_0 - b$. The mapper supplies $z = z_0 + \delta = b + t + \delta$; exact restoration before normalization would require $\delta = -t$. Across 1,024 training response positions/domain, mean positionwise $\cos(\delta, t)$ is -0.0429 on math and 0.1248 on NanoCoder; shuffled-position controls give -0.0413 and 0.1052 . These means do not support consistent token-specific cancellation, but neither a single shuffle nor a mean cosine rules out heterogeneous effects. Cosine ignores correction magnitude, and cancellation before RMSNorm is stronger than functional recovery after it.

Offline radial energy fractions are 25.22% and 34.82%, despite little radial-only acceptance benefit. The pooled relative norm error of the first-order RMSNorm prediction is 17.55% and 14.88%; a

local geometric approximation is informative but not exact at full correction strength. These are neither token-error rates nor speedup-retention percentages. Fixed training-derived shift-subspace projections are tested live: their complements retain higher acceptance than their leading spaces in both domains. Useful correction is therefore not confined to the dominant sampled target-shift space, subject to unequal subspace dimensions and nonlinear downstream processing.

5.3 EQUIVALENT EFFECTIVE MAPS CAN HAVE DIFFERENT UPDATE GEOMETRY

If F_i is invertible, unrestricted W_i and unrestricted additive ΔF_i span the same effective block, $G_i = F_i W_i = F_i + \Delta F_i$. This is a conditional algebraic proposition, not a numerical assumption: measured fusion blocks can be poorly conditioned. Exact reparameterization of an already fitted checkpoint tests algebra/export behavior without retraining; BF16 rounding can still alter greedy trajectories.

For $H_i = \partial L / \partial G_i$, plain SGD on W_i gives

$$\frac{\partial L}{\partial W_i} = F_i^T H_i, \quad \Delta G_i = -\eta F_i F_i^T H_i. \quad (13)$$

Thus the parameterization induces a fusion-dependent preconditioner in effective coordinates. AdamW does not inherit this identity unchanged because its elementwise moments and clipping depend on coordinates. We capture an actual AUF gradient and compare one zero-moment AdamW step at identical fitted effective weights. The resulting directions differ; the diagnostic is not a resumed optimizer trajectory or proof of why a final checkpoint wins. Conditioning, subspace projections, leave-one-stream-out, correction-strength and cross-domain bank tests are fully reported for math/NanoCoder in Appendix F. Remaining KiCad probes were stopped and contribute no paper claim.

6 RELATED WORK AND LIMITATIONS

Draft adaptation and reusable computation. EDA is the closest problem-level comparison: it adapts a lightweight private draft component, regenerates data using the fine-tuned target, and selects training examples (Lin et al., 2026). Our contribution is not the first shared/private draft design. We study a frozen DFlash transformer with foldable feature maps, matched full-fusion/body controls, and request-indexed serving. We did not reproduce EDA’s complete method; our body-LoRA baseline is not labeled EDA. OmniDraft combines online distillation, vocabulary translation and adaptive drafting (Ramakrishnan et al., 2025); our LoRA experiments instead fit offline and share a tokenizer. Its published speedups do not form a matched numerical baseline here.

Reparameterization and conditioning. RepSpec trains redundant linear structures and merges them at inference (Huo et al., 2026). Consequently neither folding nor the observation that training parameterization matters is new by itself. Our contribution tests where adaptation can be localized while freezing the pretrained draft body. DFlare learns richer layer-wise target conditioning and scales draft capacity and data (Zhang et al., 2026); our fixed-body study asks what can be gained without that retraining. Representation finetuning also changes model behavior through intermediate activations (Wu et al., 2024); here only speculative proposals change and target verification remains responsible for commitment.

Objectives and serving. DFlash and EAGLE-3 provide the underlying drafting algorithms (Chen et al., 2026; Li et al., 2025). Spec-AUF introduced the detached first-failure support objective (Yang & Li, 2026); we evaluate it rather than claim a new loss, and preserve cases where DFlash’s decaying CE wins. Punica and S-LoRA provide heterogeneous target-adaptor batching and memory management (Chen et al., 2023b; Sheng et al., 2023), complementary to adapting draft conditioning. Our request-indexed integration uses vLLM’s serving substrate (Kwon et al., 2023); it does not reproduce those systems’ full schedulers. Appendix I separates conceptual comparisons, our implemented controls and unperformed method reproductions.

The strongest limitations are empirical. The LoRA study uses one target family and a small number of adapters. The 128-prompt cohorts are held out from training but repeatedly inspected; a selected maximum is not independent validation. Fits largely use one seed. Cached fitting costs exclude

generation/extraction and do not establish universally cheaper total adaptation. Selected output audits reveal substantial AR and serving differences; native equality holds for the audited standalone LoRA rows. EAGLE and DFlash differ in implementation and proposal budgets. Router accuracy is domain-label accuracy. A100/DGX results are deferred, so hardware generalization is unmeasured. The main loss comparison does not favor AUF everywhere, and body adaptation remains a credible alternative. These bounds are part of the claim rather than omitted counterexamples.

7 CONCLUSION

A pretrained feature-conditioned draft body can be specialized at its target-feature interface. Full linear maps provide a competitive adaptation location; a separately trained rank-56 interface establishes an heterogeneous-serving benefit. Architecture, objective and supervision budget interact with the domain: neither more parameters nor AUF guarantees the best cell. Mechanistic tests separate identity preservation, directional conditioning and coordinate-dependent optimization. The evidence supports interface adaptation as a practical deployment option and motivates further independent validation, without claiming universal superiority or a new speculative-decoding objective.

AI USE STATEMENT

Generative AI coding assistants were used to propose and refine experimental designs, implement and debug training and evaluation code, prepare datasets and experiment reports, analyze results, develop mathematical explanations, search related literature, and draft and edit this manuscript and its figures. Target language models generated the cached training continuations described in the methods. Validation included source-code comparisons, bounded implementation checks and saved-output audits; the recorded measurement contracts remain explicit. AI-assisted interpretation is constrained by the recorded measurements, and this disclosure does not imply independent human verification of every artifact.

REPRODUCIBILITY STATEMENT

The appendices specify target/draft releases, adapter identities, dataset splits, prompt templates, loss formulas, optimization and measurement boundaries. Tables and figures are generated from retained result records with source hashes. The mathematical identities state their assumptions and do not replace inference validation. No unreported running experiment contributes a result.

ETHICS STATEMENT

The experiments use existing model adapters and public task datasets; circuit and code generation are evaluated as inference workloads, not validated engineering outputs. Faster generation does not establish safety or correctness of those outputs. The work does not report human-subject experiments.

REFERENCES

- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*, 2023a. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block Diffusion for Flash Speculative Decoding. *arXiv preprint arXiv:2602.06036*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-Tenant LoRA Serving. *arXiv preprint arXiv:2310.18547*, 2023b. URL <https://arxiv.org/abs/2310.18547>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. Repspec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554clf-Abstract-Conference.html.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *arXiv preprint arXiv:2211.17192*, 2022. URL <https://arxiv.org/abs/2211.17192>.

-
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. *arXiv preprint arXiv:2503.01840*, 2025. URL <https://arxiv.org/abs/2503.01840>.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently Aligning Draft Models via Parameter- and Data-Efficient Adaptation. *arXiv preprint arXiv:2603.09527*, 2026. URL <https://arxiv.org/abs/2603.09527>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. Omnidraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://papers.nips.cc/paper_files/paper/2025/file/3c2fe1417eed1c6ff9acfl69617981ea-Paper-Conference.pdf.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *arXiv preprint arXiv:2311.03285*, 2023. URL <https://arxiv.org/abs/2311.03285>.
- Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. ReFT: Representation Finetuning for Language Models. *arXiv preprint arXiv:2404.03592*, 2024. URL <https://arxiv.org/abs/2404.03592v3>.
- Tianjian Yang and Meng Li. Spec-AUF: Accept-Until-Fail Training under Train-Inference Misalignment for Masked Block Drafters. *arXiv preprint arXiv:2607.01893*, 2026. URL <https://arxiv.org/abs/2607.01893v1>.
- Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J. Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, Jianchen Zhu, and Sujian Li. DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding. *arXiv preprint arXiv:2606.02091*, 2026. URL <https://arxiv.org/abs/2606.02091>.

A REPRODUCTION CONTRACT AND EVIDENCE INVENTORY

This appendix is an expanded evidence record, not a proposed submission page allocation. All tables are generated from machine-readable measurements rather than transcribed from rounded prose. The accompanying `data/sweep/evidence.json` retains the underlying fields and `source_hashes.json` pins the input files. The full-precision input result tables are also retained in `data/sweep/source_results/`, with stopped-study records excluded. It contains 42 architecture/loss cells, 96 scaling cells, 52 completed serving measurements, ten 8B standalone measurements and 76 math/NanoCoder live mechanism measurements. The six original 4B AR/native controls and five EAGLE/companion-AR pairs are also printed. Shared baseline cells are reused, not additional fits. Individual raw outputs remain in the repository at the source paths in that inventory. No stopped scaling repeat or KiCad mechanism result enters these tables.

A.1 TARGETS, ADAPTERS AND DATA

The 4B base is Qwen/Qwen3-4B, revision `1cfa9a7208912126459214e8b04321603b3df60c`. The three deployed specializations are `witcheer/qwen3-4b-gsm8k-grpo`, `AbijahKaj/qwen3-4b-kicad-netlist`, and `usernamebetter/nanocoder-v1`. The math adapter is pinned at `f52e1ed1b3accd404d37a9621db0536dd99bdfc5`; NanoCoder at `f078bfc2722bdb7bf644906cc7d81662ab3de9d2`. Exact exported checkpoint and tokenizer file hashes are in `experiments/dflash_transformers_20260912/assets/manifest.json` and the acquisition manifests. These are published target adaptations, not newly trained targets. An adapter’s declared base repository does not certify its author’s unpublished base commit.

GSM8K uses `openai/gsm8k`, revision `740312add88f781978c0658806c59bc2815b9866`. Seed 42 selects 4,096 unique training questions and 128 test questions, deduplicated by stripped-question hash. The stored user message is:

```
Solve the math problem. Show brief reasoning, then end with
'#### <answer>'.
Problem: {q}
Solution:
```

It is rendered through the pinned base chat template with thinking disabled. Author-code prompt provenance is revision `816390766c9251cb0e8a1699b145ec56dd83bf67`. Generation uses the declared BF16 target and benchmark settings; this is not a replication of the author’s quantized target-quality score.

KiCad uses historical `AbijahKaj/kicad-netlist-sft-dataset` revision `52ce8608c3c0fd2f1b6b68e93b8784b88bc92373`. The publisher’s tool-then-direct ordering, chat rendering, full-text encoding and 8,192-token filter retain 16,738 rows. A 2% split with seed 42 gives 16,403 training and 335 validation rows. Direct non-tool training prompts are deduplicated and shuffled with seed 42; 4,096 enter the default fit. Direct validation prompts with no exact training-prompt overlap are shuffled with seed 20260908 to select 128. This reconstructs a publisher-style validation cohort, not an independently certified target-quality benchmark; repository-level source overlap is possible.

NanoCoder uses `HuggingFaceH4/CodeAlpaca_20K`, revision `798c567f69c8f4b12fc191015e59ee34e9afe00d`, with released prompt strings and train/test splits. Within-split shuffle seed 20260909, stripped-prompt deduplication, exclusion of test prompts present anywhere in source training, and a 3,072-token prompt limit define the cohort. The default training selection includes the shortest and longest eligible prompts first, followed by the shuffled pool. The system message is “You are NanoCoder, an expert Senior Full-Stack Engineer and debugging agent.” The user message is the released prompt verbatim, rendered with the adapter-specific nonthinking template. Mapper continuations come from the active NanoCoder target, not dataset reference solutions used as if they were target outputs. The larger cache has 18,016 unique training examples; the fixed ablation uses its selected 4,096-example cohort. Train/evaluation prompt identities and exact messages are fixed across architecture/loss variants. These experiments test draft adaptation on this coding workload, not replication of NanoCoder’s original multi-source target training distribution.

At 8B the target revision is b968826d9c46dd6066d109eabc6255188de91218 and z-lab/Qwen3-8B-DFlash-b16 is pinned at 9b41424b7109f9c5413454f481b09a82b85333f4. Math uses sumitdotml/lora-and-friends, revision 6c91e19043bb9c784b7f982f488a134f043dd944, subfolder checkpoints/best-checkpoints/attention_only/seed-0/step-3169: saved q/k/v/o rank-8, alpha-32 weights. The target was published as OpenMathSFT-adapted; “GSM8K” denotes our workload, not its original training dataset. Coding uses lovepon/Qwen3-8B-code_alpaca-lora, revision c0b5f6ed334666be14a09faf1d1363029ae13a76, rank 8, alpha 16, q/v. Each 8B mapper has 4,096 unique cached training prompts and 128 held-out prompts. The coding system message is “You are a helpful coding assistant.” No 8B target adapter is trained in this study.

A.2 CACHE, BLOCK SAMPLING AND OPTIMIZATION

Teacher-forced prefill on the persisted target-generated token IDs supplies the selected hidden states. Persisted features and tokens are reused across fits; dense feature storage does not imply dense supervision. The 4B context streams use layer indices 1, 9, 17, 25 and 33 in the pinned feature-extraction convention. Each requested anchor starts a length-16 block; the known first token is not a label. Sample at most $\min(A, L - 1)$ anchors from valid response positions, where L is the response length; near the end, padding is masked. Distinct anchor blocks may overlap. Thus neither $A \times 16$ nor the generation cap is the number of distinct supervised response tokens. AUF further changes effective label support with current predictions.

The default fit has $N = 4096$, $A = 32$, one epoch, two GPUs, two examples per GPU and two gradient-accumulation steps: global batch 8 and 512 optimizer updates. AdamW uses learning rate 10^{-4} , zero weight decay, norm clipping 1, 5% linear warmup and cosine decay, seed 42. The final checkpoint is evaluated. The target and frozen drafter parameters do not receive updates; gradients still propagate through draft computation. Each objective uses its own valid-weight denominator inside the implemented microbatch/chunk reduction, followed by accumulation and distributed averaging. It is not a single ratio accumulated over the entire dataset.

Decaying CE follows SpecForge revision 953d43a0c1c0f5e32989dc43f91ce5fc2d9ddfef, with $\gamma = 7$ and Eq. 4. Our AUF implementation changes the loss support as Eq. 7; the surrounding DFlash masked-block contract is retained. It is not ordinary next-token training or an upstream release of our modified code. The known anchor and padding cannot terminate AUF support. The first incorrect supervised prediction contributes CE, and all later slots in that block contribute zero. The support mask is detached.

In the scaling study, N is an example-presentation budget and one epoch means one pass over that constructed list. The grid uses $A \in \{8, 24, 32, 64, 128\}$ at $N = 4096$, and $N \in \{4096, 8192, 12288, 16384\}$ at $A = 32$. Their shared cell is counted once. Fits at distinct cells are independent, not optimizer continuations with an altered learning-rate schedule. Compatible unique cached examples are consumed before repetition; the GSM8K scaling cache supplies 4,096 such examples, KiCad 15,366, and NanoCoder 18,016. A presentation count exceeding available unique rows is not claimed as new data. Updates are $N/8$, respectively 512, 1,024, 1,536 and 2,048. Fitting time is measured separately for each domain and architecture. Reported GPU-hours multiply fit wall time by two and exclude historical rollout/cache construction.

A.3 STANDALONE TIMING

DFlash evaluation uses its upstream dflash_generate (package 0.1.0), Torch 2.13.0+cu129, Transformers 5.16.1 and PEFT 0.20.0. Active target LoRAs are merged for these standalone evaluations. BF16, default SDPA, greedy decoding, temperature zero, top-p one, top-k zero, and thinking disabled are fixed. AR uses the upstream block-size-1 path; DFlash uses block size 16. A single L40S handles one request at a time; GPU parallelism is across jobs, not tensor parallelism. There are 128 prompts per domain. Response caps are 2,048 for GSM8K/NanoCoder/8B coding and 8,192 for KiCad. Upstream stopping-token rules apply.

Each method receives two 64-token warmup requests from training prompts. CUDA synchronization brackets generation. Tokenization, model loading, warmup and result serialization are excluded; target prefill and decoding are included. The drafter resets to native weights before each exported

architecture is overlaid. Primary TPS gives each rollout equal weight. Summed latency and aggregate TPS are separate metrics and need not rank methods identically. Acceptance counts produced tokens per verification step, including bonus tokens and excluding the initial prefill token.

B COMPLETE ARCHITECTURE AND OBJECTIVE ABLATION

Table 4: Standalone Transformers controls on 128 prompts per domain. No adaptation fitting.

Domain	AR TPS	DFlash TPS	DFlash τ	DFlash $\times$ AR
GSM8K	38.81	127.13	4.472	3.276
KiCad	38.42	163.34	5.991	4.251
NanoCoder	39.01	150.21	5.111	3.851

Table 5: GSM8K: all matched architecture/loss cells. Gain is percent over native DFlash; parameters are millions of fitted parameters; fitting excludes cache construction.

Loss	Architecture	TPS	Gain (%)	$\times$ AR	τ	M params	Fit min	GPU h
AUF	Five W	174.90	37.58	4.506	5.992	32.768	2.66	0.0888
AUF	Body r128	173.42	36.41	4.468	5.937	36.700	2.72	0.0908
AUF	Fusion ΔF	168.27	32.36	4.335	5.809	32.768	2.60	0.0868
AUF	Joint W	158.06	24.33	4.072	5.064	163.840	3.41	0.1138
AUF	Five BA-128	131.24	3.24	3.382	4.339	3.277	2.47	0.0822
AUF	Five BA-56	99.26	-21.93	2.557	3.213	1.434	2.54	0.0848
AUF	Five BA-28	72.39	-43.06	1.865	2.382	0.717	2.52	0.0839
Decay	Five W	173.97	36.84	4.482	5.780	32.768	2.67	0.0890
Decay	Body r128	171.40	34.82	4.416	5.754	36.700	2.71	0.0902
Decay	Fusion ΔF	168.44	32.49	4.340	5.621	32.768	2.67	0.0891
Decay	Joint W	152.08	19.63	3.919	4.987	163.840	3.39	0.1128
Decay	Five BA-128	127.70	0.45	3.290	4.223	3.277	2.45	0.0818
Decay	Five BA-56	97.19	-23.55	2.504	3.104	1.434	2.45	0.0816
Decay	Five BA-28	70.44	-44.59	1.815	2.278	0.717	2.44	0.0814

Table 6: KiCad: all matched architecture/loss cells. Gain is percent over native DFlash; parameters are millions of fitted parameters; fitting excludes cache construction.

Loss	Architecture	TPS	Gain (%)	$\times$ AR	τ	M params	Fit min	GPU h
AUF	Five W	309.56	89.52	8.057	11.259	32.768	5.32	0.1774
AUF	Joint W	307.78	88.43	8.011	10.869	163.840	5.67	0.1892
AUF	Body r128	286.59	75.46	7.459	10.495	36.700	5.17	0.1722
AUF	Five BA-128	258.69	58.37	6.733	9.456	3.277	4.82	0.1607
AUF	Fusion ΔF	235.96	44.46	6.142	8.777	32.768	5.08	0.1693
AUF	Five BA-56	220.72	35.13	5.745	8.119	1.434	4.99	0.1662
AUF	Five BA-28	184.28	12.82	4.796	6.618	0.717	5.18	0.1725
Decay	Five W	316.11	93.53	8.227	11.375	32.768	6.44	0.2146
Decay	Joint W	308.23	88.70	8.022	10.933	163.840	5.72	0.1906
Decay	Body r128	302.10	84.95	7.863	10.875	36.700	5.63	0.1876
Decay	Five BA-128	266.09	62.91	6.926	9.354	3.277	4.64	0.1548
Decay	Fusion ΔF	244.52	49.70	6.364	8.901	32.768	6.00	0.1999
Decay	Five BA-56	227.60	39.34	5.924	8.069	1.434	5.47	0.1822
Decay	Five BA-28	172.99	5.91	4.503	6.412	0.717	5.65	0.1884

Table 7: NanoCoder: all matched architecture/loss cells. Gain is percent over native DFlash; parameters are millions of fitted parameters; fitting excludes cache construction.

Loss	Architecture	TPS	Gain (%)	$\times$ AR	τ	M params	Fit min	GPU h
AUF	Body r128	164.96	9.82	4.229	5.593	36.700	2.81	0.0938
AUF	Fusion ΔF	162.45	8.15	4.165	5.542	32.768	2.76	0.0918
AUF	Five W	162.32	8.07	4.161	5.549	32.768	2.76	0.0919
AUF	Joint W	139.62	-7.05	3.579	4.486	163.840	3.46	0.1153
AUF	Five BA-128	106.96	-28.79	2.742	3.387	3.277	2.59	0.0862
AUF	Five BA-56	76.12	-49.32	1.951	2.400	1.434	2.54	0.0848
AUF	Five BA-28	56.40	-62.45	1.446	1.833	0.717	2.61	0.0871
Decay	Body r128	163.20	8.65	4.184	5.457	36.700	2.79	0.0930
Decay	Fusion ΔF	159.43	6.14	4.087	5.384	32.768	2.70	0.0900
Decay	Five W	154.88	3.11	3.971	5.362	32.768	2.79	0.0929
Decay	Joint W	137.17	-8.68	3.516	4.465	163.840	3.47	0.1156

Loss	Architecture	TPS	Gain (%)	$\times$ AR	τ	M params	Fit min	GPU h
Decay	Five BA-128	108.00	-28.10	2.769	3.302	3.277	2.53	0.0842
Decay	Five BA-56	73.03	-51.38	1.872	2.346	1.434	2.52	0.0840
Decay	Five BA-28	56.77	-62.20	1.455	1.818	0.717	2.52	0.0839

C COMPLETE SCALING MEASUREMENTS

The tables include every original completed cell, including unfavorable results. A mean-TPS gain of g percent corresponds to multiplier $1 + g/100$, not to a g percent reduction in elapsed time. Paired bootstrap intervals resample prompt identities 2,000 times with seed 42, computing the stated statistic on each draw. They do not quantify training-seed uncertainty or future deployment variability. Evaluation GPU-hours sum timed request seconds and exclude setup. Marginal generation/extraction cost for reused caches is zero; their historical creation cost is excluded and must not be described as zero.

Table 8: GSM8K: complete anchor/data scaling. One epoch;4,096-by32 cells are reused. Parameters:32.768M (W) or36.700M (body).

Loss	Arch.	Examples	Anchors	TPS	Gain (%)	$\times$ AR	τ	Fit min	GPU h
AUF	Body r128	4096	8	175.29	37.88	4.516	5.877	1.32	0.0440
AUF	Body r128	4096	24	178.19	40.16	4.591	5.922	2.17	0.0723
AUF	Body r128	4096	32	173.42	36.41	4.468	5.937	2.72	0.0908
AUF	Body r128	4096	64	178.17	40.14	4.590	5.990	5.14	0.1713
AUF	Body r128	4096	128	178.70	40.57	4.604	6.014	9.70	0.3234
AUF	Body r128	8192	32	179.50	41.19	4.625	6.051	5.39	0.1798
AUF	Body r128	12288	32	184.29	44.96	4.748	6.159	8.10	0.2701
AUF	Body r128	16384	32	189.62	49.16	4.886	6.175	10.81	0.3604
AUF	Five W	4096	8	171.43	34.85	4.417	5.797	1.17	0.0390
AUF	Five W	4096	24	174.33	37.13	4.492	5.924	2.02	0.0673
AUF	Five W	4096	32	174.90	37.58	4.506	5.992	2.66	0.0888
AUF	Five W	4096	64	175.44	38.00	4.520	6.044	4.89	0.1631
AUF	Five W	4096	128	182.39	43.47	4.699	6.112	9.15	0.3049
AUF	Five W	8192	32	179.11	40.88	4.615	6.065	5.16	0.1720
AUF	Five W	12288	32	186.85	46.98	4.814	6.134	7.78	0.2594
AUF	Five W	16384	32	184.28	44.95	4.748	6.198	10.31	0.3438
Decaying CE	Body r128	4096	8	166.22	30.75	4.283	5.685	1.33	0.0443
Decaying CE	Body r128	4096	24	170.02	33.74	4.381	5.733	2.17	0.0725
Decaying CE	Body r128	4096	32	171.40	34.82	4.416	5.754	2.71	0.0902
Decaying CE	Body r128	4096	64	171.53	34.92	4.419	5.759	5.14	0.1712
Decaying CE	Body r128	4096	128	169.22	33.10	4.360	5.759	9.66	0.3220
Decaying CE	Body r128	8192	32	171.43	34.85	4.417	5.853	5.41	0.1803
Decaying CE	Body r128	12288	32	176.90	39.15	4.558	5.902	8.12	0.2708
Decaying CE	Body r128	16384	32	179.31	41.05	4.620	5.988	10.85	0.3617
Decaying CE	Five W	4096	8	171.98	35.28	4.431	5.632	1.11	0.0369
Decaying CE	Five W	4096	24	168.58	32.60	4.343	5.733	2.02	0.0675
Decaying CE	Five W	4096	32	173.97	36.84	4.482	5.780	2.67	0.0890
Decaying CE	Five W	4096	64	174.69	37.41	4.501	5.823	4.86	0.1621
Decaying CE	Five W	4096	128	173.91	36.80	4.481	5.865	9.08	0.3027
Decaying CE	Five W	8192	32	176.16	38.56	4.539	5.928	5.13	0.1709
Decaying CE	Five W	12288	32	180.12	41.68	4.641	5.960	7.68	0.2559
Decaying CE	Five W	16384	32	183.63	44.44	4.731	5.981	10.26	0.3421

Table 9: KiCad: complete anchor/data scaling. One epoch;4,096-by32 cells are reused. Parameters:32.768M (W) or36.700M (body).

Loss	Arch.	Examples	Anchors	TPS	Gain (%)	$\times$ AR	τ	Fit min	GPU h
AUF	Body r128	4096	8	260.26	59.33	6.774	9.528	3.12	0.1041
AUF	Body r128	4096	24	283.83	73.77	7.387	10.302	4.14	0.1382
AUF	Body r128	4096	32	286.59	75.46	7.459	10.495	5.17	0.1722
AUF	Body r128	4096	64	304.65	86.51	7.929	10.956	7.50	0.2500
AUF	Body r128	4096	128	312.80	91.50	8.141	11.284	13.50	0.4502
AUF	Body r128	8192	32	316.99	94.07	8.250	11.562	10.87	0.3625
AUF	Body r128	12288	32	331.91	103.20	8.639	12.009	17.71	0.5903
AUF	Body r128	16384	32	348.09	113.11	9.060	12.293	23.35	0.7782
AUF	Five W	4096	8	260.77	59.65	6.787	10.006	2.97	0.0990
AUF	Five W	4096	24	298.47	82.73	7.768	11.137	4.34	0.1446
AUF	Five W	4096	32	309.56	89.52	8.057	11.259	5.32	0.1774
AUF	Five W	4096	64	317.91	94.63	8.274	11.717	7.48	0.2494
AUF	Five W	4096	128	330.35	102.24	8.598	11.930	13.14	0.4379
AUF	Five W	8192	32	332.56	103.60	8.656	12.096	11.04	0.3681
AUF	Five W	12288	32	347.75	112.90	9.051	12.419	17.24	0.5747

Loss	Arch.	Examples	Anchors	TPS	Gain (%)	$\times$ AR	τ	Fit min	GPU h
AUF	Five W	16384	32	347.85	112.96	9.053	12.594	23.78	0.7925
Decaying CE	Body r128	4096	8	273.86	67.66	7.128	9.896	3.30	0.1099
Decaying CE	Body r128	4096	24	295.57	80.95	7.693	10.684	4.34	0.1447
Decaying CE	Body r128	4096	32	302.10	84.95	7.863	10.875	5.63	0.1876
Decaying CE	Body r128	4096	64	313.17	91.73	8.151	11.271	7.60	0.2533
Decaying CE	Body r128	4096	128	324.97	98.95	8.458	11.496	13.70	0.4565
Decaying CE	Body r128	8192	32	328.87	101.34	8.559	11.716	10.99	0.3662
Decaying CE	Body r128	12288	32	341.97	109.36	8.901	12.096	17.10	0.5699
Decaying CE	Body r128	16384	32	352.76	115.97	9.181	12.366	23.46	0.7822
Decaying CE	Five W	4096	8	275.67	68.77	7.175	10.169	3.51	0.1171
Decaying CE	Five W	4096	24	319.59	95.66	8.318	11.189	4.28	0.1428
Decaying CE	Five W	4096	32	316.11	93.53	8.227	11.375	6.44	0.2146
Decaying CE	Five W	4096	64	333.15	103.96	8.671	11.737	7.38	0.2461
Decaying CE	Five W	4096	128	339.61	107.91	8.839	12.033	12.96	0.4321
Decaying CE	Five W	8192	32	346.97	112.42	9.031	12.118	10.64	0.3548
Decaying CE	Five W	12288	32	335.66	105.50	8.736	12.366	17.31	0.5770
Decaying CE	Five W	16384	32	344.07	110.64	8.955	12.594	23.38	0.7795

Table 10: NanoCoder: complete anchor/data scaling. One epoch; 4,096-by-32 cells are reused. Parameters: 32.768M (W) or 36.700M (body).

Loss	Arch.	Examples	Anchors	TPS	Gain (%)	$\times$ AR	τ	Fit min	GPU h
AUF	Body r128	4096	8	163.94	9.15	4.203	5.493	1.37	0.0455
AUF	Body r128	4096	24	167.22	11.33	4.287	5.567	2.24	0.0745
AUF	Body r128	4096	32	164.96	9.82	4.229	5.593	2.81	0.0938
AUF	Body r128	4096	64	168.67	12.29	4.324	5.602	5.21	0.1738
AUF	Body r128	4096	128	165.81	10.39	4.251	5.628	10.34	0.3446
AUF	Body r128	8192	32	172.42	14.79	4.420	5.700	5.65	0.1883
AUF	Body r128	12288	32	173.57	15.55	4.450	5.699	8.49	0.2829
AUF	Body r128	16384	32	167.56	11.55	4.296	5.742	11.31	0.3769
AUF	Five W	4096	8	157.26	4.70	4.032	5.331	1.17	0.0389
AUF	Five W	4096	24	165.76	10.36	4.249	5.517	2.09	0.0696
AUF	Five W	4096	32	162.32	8.07	4.161	5.549	2.76	0.0919
AUF	Five W	4096	64	169.21	12.65	4.338	5.587	4.98	0.1659
AUF	Five W	4096	128	169.28	12.70	4.340	5.625	9.77	0.3258
AUF	Five W	8192	32	172.25	14.67	4.416	5.605	5.46	0.1818
AUF	Five W	12288	32	167.33	11.40	4.290	5.607	8.17	0.2723
AUF	Five W	16384	32	169.88	13.10	4.355	5.672	10.94	0.3647
Decaying CE	Body r128	4096	8	163.30	8.72	4.186	5.395	1.38	0.0461
Decaying CE	Body r128	4096	24	163.96	9.16	4.203	5.456	2.23	0.0744
Decaying CE	Body r128	4096	32	163.20	8.65	4.184	5.457	2.79	0.0930
Decaying CE	Body r128	4096	64	165.64	10.28	4.246	5.465	5.20	0.1735
Decaying CE	Body r128	4096	128	164.06	9.23	4.206	5.477	10.31	0.3437
Decaying CE	Body r128	8192	32	166.32	10.73	4.264	5.528	5.54	0.1847
Decaying CE	Body r128	12288	32	169.45	12.81	4.344	5.548	8.32	0.2772
Decaying CE	Body r128	16384	32	171.79	14.37	4.404	5.595	11.16	0.3719
Decaying CE	Five W	4096	8	157.94	5.15	4.049	5.220	1.19	0.0398
Decaying CE	Five W	4096	24	163.24	8.68	4.185	5.382	2.10	0.0699
Decaying CE	Five W	4096	32	154.88	3.11	3.971	5.362	2.79	0.0929
Decaying CE	Five W	4096	64	167.20	11.31	4.286	5.435	4.94	0.1648
Decaying CE	Five W	4096	128	163.74	9.01	4.198	5.474	9.73	0.3243
Decaying CE	Five W	8192	32	165.97	10.49	4.255	5.485	5.29	0.1763
Decaying CE	Five W	12288	32	166.58	10.90	4.270	5.512	7.99	0.2664
Decaying CE	Five W	16384	32	165.87	10.43	4.252	5.527	10.70	0.3567

Table 11: math: paired-prompt 95% bootstrap intervals; evaluation GPU-hours. These quantify prompt variability, not training-seed or hardware-repeat variability.

Loss	Arch.	N	A	TPS gain CI	Latency reduction	Latency CI	Eval GPU h
AUF	Five W	4096	32	34.02 / 41.24	23.41	21.33 / 25.73	0.0331
AUF	Body r128	4096	32	32.80 / 40.05	23.01	21.09 / 25.13	0.0332
AUF	Body r128	4096	8	34.51 / 41.41	23.74	21.84 / 25.76	0.0329
AUF	Body r128	4096	24	36.68 / 43.74	25.34	23.46 / 27.31	0.0322
AUF	Body r128	4096	64	36.28 / 44.15	24.97	23.01 / 27.04	0.0324
AUF	Body r128	4096	128	36.71 / 44.50	25.03	22.82 / 27.26	0.0324
AUF	Body r128	8192	32	37.23 / 45.11	25.00	22.88 / 27.22	0.0324
AUF	Body r128	12288	32	41.12 / 49.03	27.09	24.89 / 29.38	0.0315
AUF	Body r128	16384	32	44.97 / 53.67	28.70	26.31 / 31.15	0.0308
AUF	Five W	4096	8	31.20 / 38.54	22.21	20.14 / 24.43	0.0336
AUF	Five W	4096	24	33.58 / 40.64	22.99	20.76 / 25.34	0.0332
AUF	Five W	4096	64	34.33 / 41.81	23.39	21.24 / 25.73	0.0331
AUF	Five W	4096	128	39.56 / 47.65	25.85	23.58 / 28.37	0.0320

Loss	Arch.	N	A	TPS gain CI	Latency reduction	Latency CI	Eval GPU h
AUF	Five W	8192	32	36.78 / 45.29	24.25	21.79 / 26.82	0.0327
AUF	Five W	12288	32	42.45 / 51.61	27.28	24.75 / 29.97	0.0314
AUF	Five W	16384	32	40.74 / 49.51	26.41	23.72 / 29.08	0.0318
Decaying CE	Body r128	4096	32	31.58 / 38.05	22.64	20.72 / 24.59	0.0334
Decaying CE	Five W	4096	32	33.31 / 40.59	23.44	21.41 / 25.63	0.0330
Decaying CE	Body r128	4096	8	27.88 / 33.84	20.28	18.42 / 22.24	0.0344
Decaying CE	Body r128	4096	24	30.43 / 37.10	21.67	19.64 / 23.64	0.0338
Decaying CE	Body r128	4096	64	31.55 / 38.38	22.36	20.44 / 24.48	0.0335
Decaying CE	Body r128	4096	128	29.51 / 36.83	21.30	19.23 / 23.49	0.0340
Decaying CE	Body r128	8192	32	31.44 / 38.56	22.48	20.48 / 24.65	0.0335
Decaying CE	Body r128	12288	32	35.54 / 43.16	24.50	22.21 / 26.80	0.0326
Decaying CE	Body r128	16384	32	36.96 / 45.25	24.96	22.73 / 27.43	0.0324
Decaying CE	Five W	4096	8	31.96 / 38.71	22.49	20.37 / 24.72	0.0335
Decaying CE	Five W	4096	24	29.18 / 36.01	21.17	18.98 / 23.48	0.0340
Decaying CE	Five W	4096	64	33.89 / 41.07	23.65	21.49 / 25.81	0.0330
Decaying CE	Five W	4096	128	33.40 / 40.38	23.32	21.19 / 25.56	0.0331
Decaying CE	Five W	8192	32	34.94 / 42.17	23.92	21.76 / 26.11	0.0328
Decaying CE	Five W	12288	32	37.83 / 45.57	25.28	23.09 / 27.47	0.0322
Decaying CE	Five W	16384	32	40.51 / 48.66	26.39	23.97 / 28.82	0.0318

Table 12: kicad: paired-prompt 95% bootstrap intervals; evaluation GPU-hours. These quantify prompt variability, not training-seed or hardware-repeat variability.

Loss	Arch.	N	A	TPS gain CI	Latency reduction	Latency CI	Eval GPU h
AUF	Five W	4096	32	83.51 / 95.04	46.66	45.03 / 47.94	0.5816
AUF	Body r128	4096	32	71.02 / 79.52	42.79	41.80 / 43.68	0.6238
AUF	Body r128	4096	8	55.75 / 62.46	37.58	36.58 / 38.42	0.6805
AUF	Body r128	4096	24	69.27 / 77.74	42.34	41.35 / 43.19	0.6287
AUF	Body r128	4096	64	81.39 / 91.39	45.91	44.83 / 46.87	0.5898
AUF	Body r128	4096	128	85.80 / 96.74	47.17	46.06 / 48.15	0.5760
AUF	Body r128	8192	32	88.30 / 99.64	47.99	46.78 / 49.09	0.5670
AUF	Body r128	12288	32	96.62 / 109.34	50.11	48.86 / 51.21	0.5439
AUF	Body r128	16384	32	106.27 / 119.85	52.37	51.20 / 53.44	0.5193
AUF	Five W	4096	8	55.66 / 63.19	37.47	36.30 / 38.45	0.6818
AUF	Five W	4096	24	77.30 / 87.71	44.95	43.67 / 46.06	0.6002
AUF	Five W	4096	64	88.62 / 100.63	48.15	46.83 / 49.38	0.5654
AUF	Five W	4096	128	95.57 / 108.41	49.97	48.54 / 51.12	0.5455
AUF	Five W	8192	32	97.10 / 109.73	50.30	49.01 / 51.42	0.5419
AUF	Five W	12288	32	105.82 / 119.66	52.30	51.05 / 53.37	0.5201
AUF	Five W	16384	32	105.73 / 119.92	52.21	50.96 / 53.28	0.5211
Decaying CE	Body r128	4096	32	79.58 / 89.91	45.30	44.16 / 46.31	0.5964
Decaying CE	Five W	4096	32	87.58 / 98.96	47.68	46.35 / 48.82	0.5705
Decaying CE	Body r128	4096	8	63.38 / 71.39	40.01	39.02 / 40.84	0.6541
Decaying CE	Body r128	4096	24	76.17 / 85.49	44.21	43.17 / 45.14	0.6083
Decaying CE	Body r128	4096	64	86.09 / 97.11	47.23	46.07 / 48.29	0.5753
Decaying CE	Body r128	4096	128	92.82 / 104.99	48.99	47.80 / 50.08	0.5561
Decaying CE	Body r128	8192	32	95.07 / 107.45	49.60	48.38 / 50.71	0.5495
Decaying CE	Body r128	12288	32	102.42 / 115.98	51.39	50.12 / 52.53	0.5300
Decaying CE	Body r128	16384	32	108.66 / 123.17	52.82	51.57 / 53.96	0.5144
Decaying CE	Five W	4096	8	64.56 / 72.57	40.53	39.43 / 41.46	0.6484
Decaying CE	Five W	4096	24	89.95 / 101.03	48.09	46.97 / 49.12	0.5660
Decaying CE	Five W	4096	64	97.01 / 110.48	49.94	48.53 / 51.14	0.5458
Decaying CE	Five W	4096	128	101.34 / 114.17	51.06	49.78 / 52.19	0.5336
Decaying CE	Five W	8192	32	105.57 / 119.04	52.05	50.85 / 53.12	0.5228
Decaying CE	Five W	12288	32	98.95 / 111.77	50.51	49.30 / 51.64	0.5395
Decaying CE	Five W	16384	32	103.41 / 117.53	51.76	50.44 / 52.91	0.5259

Table 13: nanocoder: paired-prompt 95% bootstrap intervals; evaluation GPU-hours. These quantify prompt variability, not training-seed or hardware-repeat variability.

Loss	Arch.	N	A	TPS gain CI	Latency reduction	Latency CI	Eval GPU h
AUF	Five W	4096	32	5.68 / 10.44	5.01	3.33 / 6.81	0.0505
AUF	Body r128	4096	32	7.74 / 11.87	6.62	5.11 / 8.17	0.0496
AUF	Body r128	4096	8	7.12 / 11.12	5.94	4.55 / 7.43	0.0500
AUF	Body r128	4096	24	9.15 / 13.59	7.86	6.38 / 9.45	0.0490
AUF	Body r128	4096	64	10.17 / 14.44	8.61	7.21 / 10.08	0.0486
AUF	Body r128	4096	128	8.17 / 12.68	7.10	5.58 / 8.77	0.0494
AUF	Body r128	8192	32	12.56 / 16.98	10.63	9.14 / 12.21	0.0475
AUF	Body r128	12288	32	13.06 / 18.17	10.32	8.62 / 12.14	0.0477
AUF	Body r128	16384	32	9.16 / 14.19	7.61	5.76 / 9.55	0.0491
AUF	Five W	4096	8	2.51 / 6.98	1.91	0.07 / 3.73	0.0521
AUF	Five W	4096	24	8.17 / 12.59	7.07	5.32 / 8.88	0.0494

Loss	Arch.	N	A	TPS gain CI	Latency reduction	Latency CI	Eval GPU h
AUF	Five W	4096	64	10.34 / 15.03	8.44	6.95 / 10.18	0.0487
AUF	Five W	4096	128	10.25 / 15.30	8.42	6.55 / 10.36	0.0487
AUF	Five W	8192	32	12.19 / 17.24	9.71	7.88 / 11.56	0.0480
AUF	Five W	12288	32	8.76 / 14.23	7.60	5.90 / 9.52	0.0491
AUF	Five W	16384	32	10.21 / 15.97	8.50	6.71 / 10.42	0.0486
Decaying CE	Body r128	4096	32	6.46 / 11.05	5.01	3.48 / 6.73	0.0505
Decaying CE	Five W	4096	32	0.83 / 5.53	0.31	-1.66 / 2.39	0.0530
Decaying CE	Body r128	4096	8	6.82 / 10.82	5.66	4.24 / 7.18	0.0502
Decaying CE	Body r128	4096	24	6.99 / 11.51	5.71	4.32 / 7.23	0.0501
Decaying CE	Body r128	4096	64	8.08 / 12.51	6.60	5.15 / 8.15	0.0497
Decaying CE	Body r128	4096	128	7.09 / 11.53	5.69	4.09 / 7.42	0.0501
Decaying CE	Body r128	8192	32	8.64 / 13.00	7.26	5.72 / 8.90	0.0493
Decaying CE	Body r128	12288	32	10.28 / 15.56	8.19	6.52 / 9.96	0.0488
Decaying CE	Body r128	16384	32	11.99 / 17.02	9.69	8.14 / 11.34	0.0480
Decaying CE	Five W	4096	8	2.93 / 7.47	2.59	0.92 / 4.46	0.0518
Decaying CE	Five W	4096	24	6.56 / 10.83	5.78	4.21 / 7.43	0.0501
Decaying CE	Five W	4096	64	8.84 / 13.94	7.58	6.07 / 9.23	0.0491
Decaying CE	Five W	4096	128	6.65 / 11.39	5.81	4.12 / 7.56	0.0501
Decaying CE	Five W	8192	32	8.15 / 12.97	7.03	5.28 / 8.81	0.0494
Decaying CE	Five W	12288	32	8.30 / 13.73	6.67	4.83 / 8.65	0.0496
Decaying CE	Five W	16384	32	8.01 / 13.14	6.62	4.82 / 8.56	0.0496

D SHARED SERVING AND TARGET ROUTING

The vLLM benchmark is a separate optimized serving profile, not a numerical continuation of the Transformers timing denominator. It uses vLLM 0.28, compilation mode 3, CUDA graphs, supported asynchronous scheduling, BF16, and batch-invariant execution disabled. Warmup covers the evaluated LoRA/prefill cases; prefix caches are cleared before timed requests. Only complete measurements with zero timed JIT events enter the tables. DFlash proposes 15 speculative tokens. EAGLE’s vLLM setting uses three; this is not a proposal-budget-matched comparison.

The explicit-ID workload contains 384 requests at 4B (128 per domain) and 256 at 8B (128 per domain), with a fixed order shared by methods. Concurrency is 1, 4, 8, 16 or 32, with engine capacity 32. The additional 4B 12-client reference and both learned-router runs use capacity 12. A request’s target identity selects both its LoRA and its folded fusion bank. Selection is propagated per request/token ownership through heterogeneous batches; it is not a process-wide mutable weight swap. The same native draft body is shared. Only the fusion bank differs for mapped drafting. The target identity, workload and requested concurrency are held fixed across AR/native/mapped within each condition.

The router is prompt-only TF-IDF plus logistic regression, not a token-wise mixture of expert weights. At 4B it uses word 1–2 grams, up to 32,768 features, $C = 1$, L-BFGS, and 12,288 labeled training prompts. It fits 98,307 parameters in 0.889 CPU seconds, with held-out confusion matrix

$$\begin{pmatrix} 128 & 0 & 0 \\ 0 & 128 & 0 \\ 1 & 0 & 127 \end{pmatrix},$$

ordered GSM8K/KiCad/NanoCoder: 383/384 correct. At 8B it fits 27,615 parameters on 8,192 prompts in 3.233 CPU seconds (37 iterations), with confusion matrix $\begin{pmatrix} 128 & 0 \\ 1 & 127 \end{pmatrix}$, ordered math/code: 255/256 correct. Exported router parity is checked to floating-point tolerance. These accuracies measure intended domain selection, not task correctness. The frozen router is reused for all methods in its workload, including AR. Mapper fitting is separate; no MoE drafter body is trained here.

One BF16 folded fusion matrix has $5d^2$ entries. Three 4B banks occupy 196,608,000 payload bytes; two 8B banks occupy 335,544,320 bytes, excluding container metadata and runtime allocations. This is a localized bank, not a claim of fewer parameters than every body adapter. The operational memory table reports allocator-reserved memory at the recorded snapshot, not a measured peak or bank-only allocation.

Client timing includes loopback HTTP and routing after acquiring the concurrency semaphore; semaphore wait is excluded from request latency. Aggregate serving throughput includes workload drain. TTFT is first-token latency and TPOT is the recorded inter-token metric. Small-output requests, different completion lengths and drain effects can separate mean request TPS from aggregate throughput. In particular, 8B mapped aggregate throughput at higher concurrency need not

improve even when mean request TPS does; both are reported. The final 4B one-client AR run covers all 384 requests with zero timed JIT events; its mean request TPS is 64.394 and workload duration is 10,821.843 seconds.

Table 14: Qwen3-4b serving; primary per-request throughput. Fit times are per-domain minutes, not one pooled fitting cost. Explicit capacity32 except the4B12-client reference (capacity12); router capacity12.

Routing	Clients	Method	Mean TPS	Gain (%)	$\times$ AR	τ	Fit min/domain
learned	12	AR	43.80	-68.81	1.000	0	
learned	12	EAGLE-3	78.71	-43.96	1.797	2.561	0
learned	12	Five W	201.98	43.82	4.611	10.459	2.66/5.32/2.76
learned	12	DFlash	140.44	0.00	3.206	5.827	0
oracle	1	AR	64.39	-76.17	1.000	0	
oracle	1	EAGLE-3	135.14	-49.99	2.099	2.542	0
oracle	1	Five W	390.71	44.60	6.067	10.463	2.66/5.32/2.76
oracle	1	DFlash	270.21	0.00	4.196	5.784	0
oracle	4	AR	55.44	-71.24	1.000	0	
oracle	4	EAGLE-3	111.32	-42.25	2.008	2.539	0
oracle	4	Five W	283.22	46.93	5.109	10.507	2.66/5.32/2.76
oracle	4	DFlash	192.75	0.00	3.477	5.793	0
oracle	8	AR	49.44	-71.23	1.000	0	
oracle	8	EAGLE-3	92.52	-46.16	1.871	2.545	0
oracle	8	Five W	246.99	43.75	4.996	10.464	2.66/5.32/2.76
oracle	8	DFlash	171.82	0.00	3.476	5.906	0
oracle	12	AR	43.81	-68.48	1.000	0	
oracle	12	EAGLE-3	78.53	-43.50	1.793	2.563	0
oracle	12	Five W	202.42	45.64	4.621	10.497	2.66/5.32/2.76
oracle	12	DFlash	138.99	0.00	3.173	5.868	0
oracle	16	AR	40.91	-67.18	1.000	0	
oracle	16	EAGLE-3	73.87	-40.75	1.806	2.534	0
oracle	16	Five W	175.93	41.12	4.300	10.496	2.66/5.32/2.76
oracle	16	DFlash	124.67	0.00	3.047	5.832	0
oracle	32	AR	31.01	-58.78	1.000	0	
oracle	32	EAGLE-3	58.31	-22.48	1.880	2.547	0
oracle	32	Five W	108.14	43.77	3.488	10.576	2.66/5.32/2.76
oracle	32	DFlash	75.22	0.00	2.426	5.790	0

Table 15: Serving operational metrics. TTFT/TPOT in milliseconds; memory is allocator reserved GiB, not peak model size. Fitting costs are in the preceding table.

Routing	Clients	Method	Aggregate TPS	Latency s	TTFT ms	TPOT ms	GiB	Eval min
learned	12	AR	503.17	40.891	105.59	22.60	34.82	22.73
learned	12	EAGLE-3	960.25	21.283	106.04	12.47	34.67	12.02
learned	12	Five W	3195.90	6.304	113.98	5.22	34.56	3.52
learned	12	DFlash	1824.21	11.035	113.70	7.18	34.90	6.22
oracle	1	AR	63.93	28.182	86.01	15.20	34.75	180.36
oracle	1	EAGLE-3	136.27	13.087	43.37	7.26	34.57	83.76
oracle	1	Five W	516.24	3.444	43.11	2.69	33.62	22.04
oracle	1	DFlash	291.61	6.033	45.64	3.69	33.63	38.61
oracle	4	AR	226.28	30.938	91.67	17.73	34.75	50.12
oracle	4	EAGLE-3	484.34	14.628	70.35	8.81	34.57	23.53
oracle	4	Five W	1572.09	4.485	79.12	3.75	33.68	7.25
oracle	4	DFlash	889.11	7.993	81.14	5.17	33.75	12.83
oracle	8	AR	390.70	35.621	99.12	19.94	34.78	29.24
oracle	8	EAGLE-3	775.09	17.985	82.96	10.61	34.60	14.80
oracle	8	Five W	2689.53	5.207	92.57	4.29	33.82	4.26
oracle	8	DFlash	1534.73	9.169	93.94	5.83	33.91	7.56
oracle	12	AR	491.38	40.834	106.94	22.58	34.82	23.28
oracle	12	EAGLE-3	967.81	21.240	105.65	12.52	34.67	11.87
oracle	12	Five W	3227.26	6.352	112.39	5.27	34.56	3.54
oracle	12	DFlash	1842.77	11.472	115.31	7.26	34.82	6.37
oracle	16	AR	595.46	44.556	112.48	24.30	34.82	19.12
oracle	16	EAGLE-3	1150.17	22.824	103.53	13.41	34.72	9.79
oracle	16	Five W	3693.09	7.430	129.22	6.03	34.26	3.11
oracle	16	DFlash	2097.75	12.880	128.68	8.05	34.10	5.54
oracle	32	AR	827.73	60.910	140.80	32.77	34.90	13.74
oracle	32	EAGLE-3	1639.70	30.780	151.20	17.38	34.89	7.00
oracle	32	Five W	4323.96	12.221	212.98	9.94	34.76	2.69
oracle	32	DFlash	2439.74	20.736	210.31	13.47	34.58	4.67

Table 16: Qwen3-8b serving: primary per-request throughput. Fit times are per-domain minutes, not one pooled fitting cost. Explicit capacity32 except the4B12-client reference (capacity12); router capacity12.

Routing	Clients	Method	Mean TPS	Gain (%)	$\times$ AR	τ	Fit min/domain
learned	12	AR	34.20	-68.33	1.000	0	
learned	12	EAGLE-3	63.43	-41.27	1.855	2.352	0
learned	12	Five W	124.28	15.07	3.634	6.005	4.06/4.02
learned	12	DFlash	108.00	0.00	3.158	5.184	0
oracle	1	AR	39.22	-75.01	1.000	0	
oracle	1	EAGLE-3	77.87	-50.39	1.985	2.376	0
oracle	1	Five W	183.69	17.02	4.683	5.930	4.06/4.02
oracle	1	DFlash	156.97	0.00	4.002	5.186	0
oracle	4	AR	35.38	-72.85	1.000	0	
oracle	4	EAGLE-3	72.74	-44.18	2.056	2.353	0
oracle	4	Five W	150.72	15.66	4.260	5.896	4.06/4.02
oracle	4	DFlash	130.31	0.00	3.683	5.060	0
oracle	8	AR	35.20	-70.70	1.000	0	
oracle	8	EAGLE-3	68.61	-42.90	1.949	2.364	0
oracle	8	Five W	139.09	15.76	3.951	5.912	4.06/4.02
oracle	8	DFlash	120.15	0.00	3.413	5.108	0
oracle	16	AR	33.51	-65.49	1.000	0	
oracle	16	EAGLE-3	62.56	-35.58	1.867	2.374	0
oracle	16	Five W	110.93	14.22	3.310	5.899	4.06/4.02
oracle	16	DFlash	97.12	0.00	2.898	5.174	0
oracle	32	AR	30.59	-50.64	1.000	0	
oracle	32	EAGLE-3	55.80	-9.96	1.824	2.371	0
oracle	32	Five W	70.05	13.03	2.290	5.985	4.06/4.02
oracle	32	DFlash	61.98	0.00	2.026	5.199	0

Table 17: Serving operational metrics. TTFT/TPOT in milliseconds; memory is allocator reserved GiB, not peak model size. Fitting costs are in the preceding table.

Routing	Clients	Method	Aggregate TPS	Latency s	TTFT ms	TPOT ms	GiB	Eval min
learned	12	AR	394.54	4.595	116.44	28.05	34.05	1.73
learned	12	EAGLE-3	737.91	2.444	117.64	14.88	33.78	0.93
learned	12	Five W	1384.27	1.306	136.23	7.27	34.10	0.50
learned	12	DFlash	1248.05	1.468	135.25	8.71	34.12	0.56
oracle	1	AR	39.69	4.070	88.49	24.77	34.55	17.36
oracle	1	EAGLE-3	80.67	2.015	66.67	12.42	34.42	8.60
oracle	1	Five W	178.59	0.914	67.13	5.19	33.21	3.90
oracle	1	DFlash	159.17	1.030	69.88	6.23	31.81	4.40
oracle	4	AR	143.12	4.539	106.52	27.27	34.55	4.87
oracle	4	EAGLE-3	299.96	2.138	98.43	12.98	34.42	2.30
oracle	4	Five W	593.43	1.072	109.30	6.04	33.21	1.15
oracle	4	DFlash	525.87	1.215	109.82	7.21	33.33	1.30
oracle	8	AR	272.58	4.537	107.53	27.40	34.55	2.54
oracle	8	EAGLE-3	555.52	2.275	115.21	13.74	32.77	1.25
oracle	8	Five W	1043.65	1.190	119.35	6.58	31.84	0.67
oracle	8	DFlash	931.62	1.343	119.86	7.85	33.47	0.75
oracle	16	AR	495.40	4.633	133.01	28.64	33.98	1.36
oracle	16	EAGLE-3	945.01	2.488	141.20	14.97	32.77	0.74
oracle	16	Five W	1431.21	1.472	154.25	8.16	32.11	0.49
oracle	16	DFlash	1482.33	1.646	151.63	9.69	32.08	0.47
oracle	32	AR	512.06	5.397	135.85	31.35	34.06	1.42
oracle	32	EAGLE-3	1270.93	2.760	202.59	16.49	32.77	0.54
oracle	32	Five W	1521.07	2.221	247.90	13.02	32.52	0.45
oracle	32	DFlash	1569.56	2.499	245.68	15.12	32.50	0.44

E 8B STANDALONE AND EAGLE-3

The 8B five-W fit uses the same 4,096/32/one-epoch recipe, native 8B drafter, and width 4,096. Math takes 243.683 seconds (0.13538 GPU-hours), code 241.380 seconds (0.13410 GPU-hours). Each fits 83,886,080 parameters and 512 updates. The target’s untied output head is loaded separately from its input embedding. Tokenizer compatibility and model-width compatibility are distinct; these are same-size target specializations, not heterogeneous-vocabulary transfer.

Table 18: 8B standalone. EAGLE uses its own AR companion. Five-W has 83.886M fitted parameters.

Domain	Method	TPS	Gain (%)	$\times$ AR	τ	Fit min	M params
GSM8K	AR	37.89	-70.56	1.000	1.000	0.00	0.000
GSM8K	DFlash	128.69	0.00	3.397	4.764	0.00	0.000

Domain	Method	TPS	Gain (%)	$\times$ AR	τ	Fit min	M params
GSM8K	Five W	173.88	35.12	4.589	6.350	4.06	83.886
GSM8K	EAGLE AR	31.19	-75.77	1.000		0.00	0.000
GSM8K	EAGLE-3	76.25	-40.75	2.445	3.728	0.00	0.000
CodeAlpaca	AR	37.56	-75.40	1.000	1.000	0.00	0.000
CodeAlpaca	DFlash	152.70	0.00	4.065	5.618	0.00	0.000
CodeAlpaca	Five W	162.13	6.18	4.316	5.631	4.02	83.886
CodeAlpaca	EAGLE AR	30.32	-80.15	1.000		0.00	0.000
CodeAlpaca	EAGLE-3	76.21	-50.09	2.514	4.051	0.00	0.000

EAGLE-3 uses the pinned upstream implementation, revision prefix `cb7e084`, Transformers 4.53.3 and PEFT 0.15.2, with the shared Torch installation. Standalone tree size 60, depth 7, top-k 10 and greedy decoding follow the declared EAGLE configuration. Qwen’s explicit head dimension is respected (128 for the 4B checkpoint); assuming hidden-size divided by head count would be incorrect for that configuration. Its AR companion uses that same target/environment. We report its own AR multiplier, rather than attribute backend differences solely to drafting algorithms. Published EAGLE weights incur no adaptation fit in this comparison.

Table 19: EAGLE-3 standalone and its AR companion. Published draft weights; no adaptation fit. Tree60/depth7/top-k10 differs from vLLM’s3 speculative-token configuration.

Size	Domain	AR TPS	EAGLE TPS	$\times$ AR	τ	Fit min
4B	GSM8K	31.49	71.19	2.260	3.347	0
4B	KiCad	30.37	73.82	2.431	3.794	0
4B	NanoCoder	31.80	81.77	2.572	3.852	0
8B	GSM8K	31.19	76.25	2.445	3.728	0
8B	CodeAlpaca	30.32	76.21	2.514	4.051	0

F MECHANISTIC PROTOCOLS AND COMPLETE INTERVENTIONS

Only GSM8K and NanoCoder enter this section. All probes operate on the canonical five-W AUF fit, not the old residual-fusion architecture. Static interventions are exported to the same dense fusion kernel. They do not demonstrate low-rank-kernel acceleration. Evaluation has the same 128-prompt cohorts and domain caps. Mapper-bank swaps retain the destination target LoRA and prompts; source task, adapter and training distribution still covary.

Identity versus correction rank. For $D_i = W_i - I$ and truncated SVD $[M]_r$, whole-map compression uses $[W_i]_r$, whereas correction compression uses $I + [D_i]_r$. SVD is computed in FP32 before BF16 folded export. An algebraic control exports $F_i + F_i D_i$. Its exact-real equivalence to $F_i W_i$ does not imply bitwise equality after reordered computation and rounding. Rank choices use training-feature calibration; every reported live rank evaluation is retained. For summed request times T_0, T_*, T_r , retention is $(T_0 - T_r)/(T_0 - T_*)$. Static rows use static native/full controls; dynamic-hook rows use hook-native/hook-full; repeat rows use their own round. Both gain and retention follow this grouping. Native retention is zero and full retention one by definition. Paired-prompt intervals do not measure between-run timing variability; the explicit rank/control repeats show why one crossing of 0.98 is insufficient for a stable retention claim.

Norm and directional interventions. Let $z_0 = Fh$, $\delta = Gh - z_0$, $s = \sqrt{\|z_0\|^2/d + \epsilon}$ and frozen gain g . RMSNorm is $N(z) = g \odot z / \sqrt{\|z\|^2/d + \epsilon}$, with $\epsilon = 10^{-6}$. Its Jacobian is

$$J_N(z_0) = \text{diag}(g) \left(s^{-1} I - \frac{z_0 z_0^T}{ds^3} \right).$$

With denominator floor $\epsilon_p = 10^{-20}$, $\delta_{\parallel} = z_0(z_0^T \delta) / \max(\|z_0\|^2, \epsilon_p)$ and $\delta_{\perp} = \delta - \delta_{\parallel}$. Hooks evaluate $N(z_0 + \delta_{\parallel})$ or $N(z_0 + \delta_{\perp})$, together with hook-native and hook-full controls. Output rescaling evaluates $N(2Gh)$; correction strength instead evaluates $N(z_0 + \alpha \delta)$, $\alpha \in \{0.5, 1.5, 2\}$. These are different perturbations. Positive scale invariance is exact at zero epsilon in real arithmetic, including with a frozen nonuniform gain; finite epsilon and finite precision make it approximate. Radial sign reversal is not a positive rescaling.

Calibration and subspaces. Paired base/active-LoRA teacher-forced prefill uses identical token IDs from 64 training examples and 16 evenly spaced response positions each: 1,024 positions. No new tokens are generated. Offline preparation takes 29.19 seconds for math and 30.11 for NanoCoder. Radial energy fractions are 0.25223 and 0.34823; first-order Jacobian relative errors are 0.17553 and 0.14875. The actual target shift is $t = Fh_{\text{LoRA}} - Fh_{\text{base}}$. Mean correction/shift cosine is -0.04289 for math and 0.12477 for NanoCoder; shuffled-position controls are -0.04129 and 0.10525 . These do not establish direct cancellation or alignment with the target’s parameter-space LoRA factors.

For orthonormal bases U, V of the recorded rank r , overlap is $\|U^T V\|_F^2/r$, comparing leading right-singular vectors of F_i with leading left-singular vectors of D_i , with principal cosines from singular values of $U^T V$. The fusion intervention uses the leading *right* singular vectors Q_i of each frozen F_i , with $P_i = Q_i Q_i^T$. It projects the output of each learned stream correction *before fusion*:

$$\delta_{\text{fusion, top}} = \sum_i F_i P_i D_i h_i, \quad \delta_{\text{fusion, comp}} = \sum_i F_i (I - P_i) D_i h_i.$$

The common rank 404 covers at least 50% of every block’s squared singular-value energy. This is not a projection of the final fused correction. By contrast, the shift projector $P_t = Q_t Q_t^T$ acts *after fusion*: $\delta_{\text{shift, top}} = P_t \delta$, $\delta_{\text{shift, comp}} = (I - P_t) \delta$. Ranks 413 for math and 244 for NanoCoder capture 90% of the sampled off-centered target-shift energy. These fixed training-derived spaces require no second live target and do not project onto each token’s instantaneous shift. Top and complement have unequal dimensions; their nonlinear effects are not additive. Leave-one-stream-out sets one W_i to identity; cross-bank interventions use another domain’s complete maps.

What the geometry statistics measure. At sampled position p , $t_p = z_{0,p} - b_p$ is the target-induced fused shift and $\delta_p = z_p - z_{0,p}$ is the mapper correction. Exact pre-normalization restoration requires $\delta_p = -t_p$, including magnitude. We report $P^{-1} \sum_p \cos(\delta_p, t_p)$, not the cosine of mean vectors; shuffling reassigns t_p among positions. A near-zero mean can hide positive and negative positionwise effects, and the one-shuffle comparison has no significance estimate. The radial axis is $z_{0,p}$, not t_p . Define $a_p = N(z_{0,p} + \delta_p) - N(z_{0,p})$ and $\hat{a}_p = J_N(z_{0,p}) \delta_p$. The reported statistics are

$$E_{\text{rad}} = \frac{\sum_p \|\delta_{\parallel, p}\|^2}{\sum_p \|\delta_p\|^2}, \quad E_J = \sqrt{\frac{\sum_p \|\hat{a}_p - a_p\|^2}{\sum_p \|a_p\|^2}}.$$

The former is an energy ratio; the latter is a relative norm error, not a squared ratio or a per-position median. Neither predicts latency by definition. The data below are offline training-position diagnostics, separate from live inference tables.

Domain	Own cosine	Shuffled cosine	E_{rad}	E_J
GSM8K	-0.0429	-0.0413	25.22%	17.55%
NanoCoder	0.1248	0.1052	34.82%	14.88%

Literal cancellation is not supported by these means. This does not rule out useful information recovery, effects hidden by averaging, or equivalence after normalization. Nor does the radial intervention prove that training explicitly learned to avoid radial changes.

Update geometry. At effective $G_i = F_i W_i$, writing $H_i = \partial L / \partial G_i$ gives $\partial L / \partial W_i = F_i^T H_i$. Unclipped SGD therefore induces $\Delta G_i = -\eta F_i F_i^T H_i$, whereas direct fusion SGD gives $-\eta H_i$. The equality is verified numerically on actual AUF gradients. For Adam, a zero-moment first step is computed with learning rate 10^{-4} , epsilon 10^{-8} , zero weight decay and norm clipping 1 separately in each coordinate system. We compare effective-update cosine, norm ratio, descent cosine and energy in leading left-singular fusion spaces at ranks 16/64/256. The gradient comes from the first canonical minibatch, seed 42000, 32 anchors, at the fitted checkpoint. No historical Adam moments are restored. It is a local diagnostic, not a causal attribution of the entire learning trajectory.

Table 20: math: per-stream geometry on 1,024 training positions. Condition number belongs to frozen fusion; overlap compares the recorded leading subspaces.

Stream	$\sigma_{max}(W)$	$\sigma_{min}(W)$	$\kappa(F_i)$	Rank	Overlap	Random
0	1.657	0.666	1198460	2	0.4845	0.0023
1	1.366	0.731	70074	2	0.0393	0.0003
2	1.367	0.720	95404	2	0.0039	0.0010
3	1.323	0.786	36386	2	0.2517	0.0006
4	1.394	0.727	80481	2	0.0914	0.0017

Table 21: math: actual AUF-gradient coordinate diagnostic. Energy columns show W/direct update fractions in fusion output singular subspaces. One zero-moment step, not optimizer continuation.

Stream	SGD error	Adam cos.	Norm ratio	W descent	Direct descent	E16	E64	E256
0	2.5e-07	0.238	79.11	0.167	0.466	0.935/0.033	0.976/0.089	0.995/0.236
1	3.5e-07	0.466	18.79	0.291	0.405	0.435/0.021	0.695/0.076	0.906/0.231
2	3.2e-07	0.457	27.35	0.291	0.414	0.347/0.021	0.714/0.078	0.955/0.255
3	3.0e-07	0.481	19.68	0.394	0.522	0.394/0.023	0.677/0.081	0.926/0.251
4	4.9e-07	0.515	5.85	0.371	0.458	0.175/0.019	0.323/0.051	0.604/0.144

Table 22: nanocoder: per-stream geometry on 1,024 training positions. Condition number belongs to frozen fusion; overlap compares the recorded leading subspaces.

Stream	$\sigma_{max}(W)$	$\sigma_{min}(W)$	$\kappa(F_i)$	Rank	Overlap	Random
0	1.793	0.676	1198460	2	0.5122	0.0023
1	1.354	0.770	70074	2	0.3390	0.0003
2	1.294	0.780	95404	2	0.1500	0.0010
3	1.322	0.828	36386	2	0.7222	0.0006
4	1.313	0.790	80481	2	0.1047	0.0017

Table 23: nanocoder: actual AUF-gradient coordinate diagnostic. Energy columns show W/direct update fractions in fusion output singular subspaces. One zero-moment step, not optimizer continuation.

Stream	SGD error	Adam cos.	Norm ratio	W descent	Direct descent	E16	E64	E256
0	2.3e-07	0.269	93.55	0.190	0.480	0.965/0.059	0.986/0.117	0.997/0.266
1	2.9e-07	0.473	22.75	0.250	0.350	0.680/0.071	0.844/0.144	0.948/0.292
2	2.9e-07	0.455	30.30	0.251	0.358	0.519/0.032	0.781/0.095	0.967/0.291
3	2.8e-07	0.480	21.43	0.376	0.496	0.515/0.035	0.755/0.100	0.944/0.289
4	5.3e-07	0.495	5.72	0.326	0.419	0.145/0.014	0.286/0.043	0.568/0.130

The following task identifiers are stable source keys: 0_full is unmodified five-W; 1_native is native; 2_reparameterized is algebraic export; whole_rank/delta_rank identify the two SVD interventions; leave_out sets one stream to identity; cal denotes training-calibrated rank/subspace exports; repeat denotes mechanism-specific control repeats. Numerical task suffixes are implementation identifiers, not a ranking of results. Full spectra, principal cosines, gradient diagnostics and calibration provenance accompany the scalar tables in data/sweep/*_geometry.json and *_gradient.json.

Table 24: GSM8K: every completed live intervention. Hook rows use dynamic native/full references for gain and retention; static rows use static references, and repeat rows their own round. Retention is a fraction (1 means 100%). Intervals resample prompts, not timing runs. Static variants use dense fusion; fitted interventions inherit 32.768M parameters.

Intervention	TPS	Gain (%)	τ	Retention	95% CI	Fit min
full	179.83	39.51	5.988	1.000	1.000–1.000	2.66
leave_out layer=3	176.18	36.68	5.908	0.949	0.914–0.984	2.66
leave_out layer=4	171.68	33.18	5.833	0.864	0.829–0.898	2.66
full hook=native	131.47	0.00	4.472	0.000	0.000–0.000	2.66
full hook=full	180.17	37.05	5.988	1.000	1.000–1.000	2.66
full hook=radial	124.61	-5.21	4.466	-0.236	-0.273–0.203	2.66
full hook=perpendicular	177.64	35.12	5.924	0.938	0.910–0.967	2.66
strength alpha=0.5	177.02	37.33	5.757	0.989	0.954–1.027	2.66
strength alpha=1.5	175.61	36.24	5.802	0.898	0.862–0.935	2.66
strength alpha=2.0	165.93	28.72	5.400	0.728	0.672–0.781	2.66
full source_bank=kicad	127.63	-0.99	4.430	-0.025	-0.072–0.020	5.32
native	128.90	0.00	4.472	0.000	0.000–0.000	0.00

Intervention	TPS	Gain (%)	τ	Retention	95% CI	Fit min
full source_bank=nanocoder	129.72	0.63	4.565	0.042	-0.003–0.086	2.76
reparameterized	181.94	41.14	5.995	1.032	1.018–1.047	2.66
whole_rank rank=1	34.16	-73.50	1.145	-11.425	-12.692–10.181	2.66
whole_rank rank=2	34.50	-73.24	1.165	-11.250	-12.511–10.029	2.66
delta_rank rank=1	143.26	11.14	4.913	0.369	0.328–0.410	2.66
delta_rank rank=2	148.99	15.58	5.072	0.496	0.457–0.535	2.66
leave_out layer=0	174.69	35.52	5.982	0.886	0.856–0.917	2.66
leave_out layer=1	178.72	38.65	5.979	0.979	0.949–1.009	2.66
leave_out layer=2	184.68	43.27	6.006	1.096	1.067–1.124	2.66
delta_rank rank=128	179.10	38.94	5.928	0.992	0.971–1.012	2.66
delta_rank rank=256	172.59	33.89	5.964	0.870	0.847–0.891	2.66
delta_rank rank=64	172.81	34.06	5.931	0.876	0.847–0.904	2.66
fusion_complement rank=404	148.42	15.14	5.273	0.461	0.422–0.499	2.66
fusion_top rank=404	174.08	35.05	5.846	0.905	0.877–0.934	2.66
output_scale alpha=2	181.07	40.47	5.988	1.021	1.014–1.028	2.66
shift_complement rank=413	163.41	26.77	5.713	0.736	0.699–0.772	2.66
shift_top rank=413	156.94	21.75	5.417	0.622	0.577–0.664	2.66
full (repeat 0)	176.03	40.92	5.988	1.000	1.000–1.000	2.66
native (repeat 0)	124.91	0.00	4.472	0.000	0.000–0.000	0.00
delta_rank rank=128 (repeat 0)	172.81	38.35	5.928	0.951	0.932–0.968	2.66
delta_rank rank=256 (repeat 0)	180.10	44.18	5.964	1.067	1.043–1.090	2.66
full (repeat 1)	184.00	44.50	5.988	1.000	1.000–1.000	2.66
native (repeat 1)	127.34	0.00	4.472	0.000	0.000–0.000	0.00
delta_rank rank=128 (repeat 1)	183.07	43.76	5.928	0.996	0.975–1.016	2.66
delta_rank rank=256 (repeat 1)	176.10	38.29	5.964	0.886	0.867–0.903	2.66

Table 25: NanoCoder: every completed live intervention. Hook rows use dynamic native/full references for gain and retention; static rows use static references, and repeat rows their own round. Retention is a fraction (1 means 100%). Intervals resample prompts, not timing runs. Static variants use dense fusion; fitted interventions inherit 32.768M parameters.

Intervention	TPS	Gain (%)	τ	Retention	95% CI	Fit min
full	171.73	13.89	5.552	1.000	1.000–1.000	2.76
leave_out layer=3	170.51	13.08	5.539	0.928	0.855–1.000	2.76
leave_out layer=4	168.31	11.62	5.447	0.849	0.777–0.933	2.76
full hook=naive	150.58	0.00	5.111	0.000	0.000–0.000	2.76
full hook=full	161.63	7.33	5.552	1.000	1.000–1.000	2.76
full hook=radial	150.78	0.13	5.114	0.016	-0.113–0.135	2.76
full hook=perpendicular	170.39	13.15	5.527	1.966	1.687–2.419	2.76
strength alpha=0.5	169.23	12.23	5.532	0.908	0.835–0.992	2.76
strength alpha=1.5	163.65	8.53	5.321	0.540	0.427–0.637	2.76
strength alpha=2.0	152.46	1.11	5.032	-0.044	-0.257–0.121	2.76
full source_bank=math	148.97	-1.20	5.016	-0.020	-0.158–0.105	2.66
native	150.79	0.00	5.111	0.000	0.000–0.000	0.00
full source_bank=kicad	148.12	-1.77	4.990	-0.111	-0.268–0.026	5.32
reparameterized	165.03	9.44	5.539	0.657	0.590–0.715	2.76
whole_rank rank=1	31.06	-79.40	1.057	-34.844	-40.415–30.472	2.76
whole_rank rank=2	33.06	-78.08	1.076	-32.481	-37.691–28.355	2.76
delta_rank rank=1	157.11	4.19	5.239	0.388	0.309–0.462	2.76
delta_rank rank=2	151.77	0.65	5.259	0.021	-0.077–0.114	2.76
leave_out layer=0	167.18	10.87	5.577	0.749	0.680–0.812	2.76
leave_out layer=1	170.44	13.03	5.569	0.906	0.850–0.959	2.76
leave_out layer=2	170.78	13.26	5.587	0.929	0.872–0.991	2.76
delta_rank rank=128	163.60	8.50	5.492	0.601	0.521–0.674	2.76
delta_rank rank=256	168.04	11.44	5.521	0.831	0.788–0.874	2.76
delta_rank rank=512	167.20	10.89	5.528	0.762	0.708–0.808	2.76
fusion_complement rank=404	159.58	5.83	5.337	0.447	0.368–0.524	2.76
fusion_top rank=404	157.99	4.78	5.484	0.284	0.174–0.389	2.76
output_scale alpha=2	166.34	10.32	5.552	0.724	0.680–0.763	2.76
shift_complement rank=244	162.92	8.05	5.479	0.612	0.526–0.691	2.76
shift_top rank=244	152.37	1.05	5.299	0.077	-0.039–0.175	2.76
delta_rank rank=1024	168.84	11.97	5.546	0.845	0.779–0.899	2.76
delta_rank rank=2048	166.49	10.41	5.553	0.723	0.646–0.788	2.76
full (repeat 0)	171.03	12.20	5.552	1.000	1.000–1.000	2.76
native (repeat 0)	152.44	0.00	5.111	0.000	0.000–0.000	0.00
delta_rank rank=256 (repeat 0)	163.49	7.25	5.521	0.578	0.506–0.647	2.76
delta_rank rank=512 (repeat 0)	166.52	9.24	5.528	0.724	0.656–0.784	2.76
full (repeat 1)	172.93	10.12	5.552	1.000	1.000–1.000	2.76
native (repeat 1)	157.04	0.00	5.111	0.000	0.000–0.000	0.00
delta_rank rank=256 (repeat 1)	171.45	9.18	5.521	0.893	0.818–0.972	2.76
delta_rank rank=512 (repeat 1)	166.80	6.22	5.528	0.521	0.404–0.622	2.76

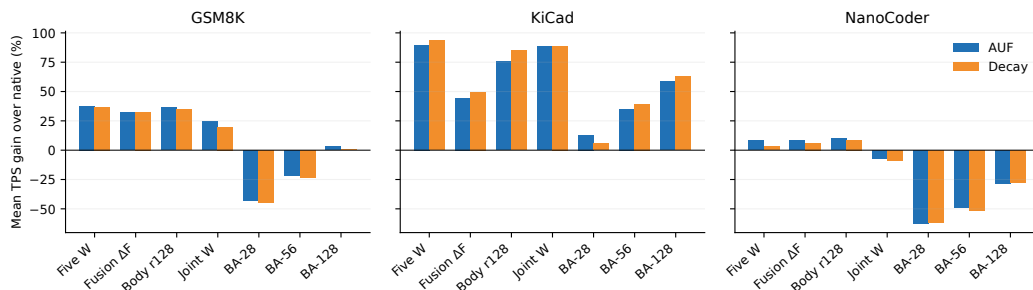


Figure 2: Matched architecture/loss comparison. Every bar uses 4,096 examples, 32 anchors and one epoch; negative values are slower than native DFlash. Parameterization and domain matter at least as much as objective choice.

G DEFERRED MEASUREMENTS AND REPRODUCTION ENTRY POINTS

Measurement	Status
A100 standalone, baselines and deployment	Deferred
DGX Spark standalone, baselines and deployment	Deferred
KiCad mechanistic study	Stopped; separate <code>incomplete.md</code>
Optional scaling timing repeats	Stopped; separate <code>incomplete.md</code>

The standalone runner is `experiments/dflash_transformers_20260912/benchmark.py`. Matched decay and joint-W fits are in `experiments/dflash_decay32_20260912/` and `experiments/dflash_jointw32_20260912/`. Scaling aggregation and paired uncertainty are in `experiments/dflash_scaling_decay_20260913/`. Serving uses `experiments/dflash_peak_deployment_20260912/` and `experiments/dflash_portable_deployment_20260913/`; 8B fits use `experiments/dflash_qwen8b_20260913/`. Mechanism exports/evaluations are in `experiments/dflash_ready_20260913/`. The paper’s `build.sh` regenerates tables and plots from frozen local measurement files; it launches no GPU work. Exact commands, assets and fitted checkpoint provenance remain in each family’s setup manifests and detailed experiment log.

H LORA EVIDENCE AND COMPLEMENTARY CONTROLS

This section prevents the newest optimized runs from replacing stronger evidence. It distinguishes five full maps from rank-56 fusion residuals; their results must not be combined as one checkpoint family. The five-map mechanistic interpretation is the one developed in the mechanism section and Appendix F; earlier rank-56 observations do not establish the rank or geometry of five-map updates.

H.1 FIVE-MAP vLLM SIZE AND ANCHOR STUDY

The same five-map AUF training family also has a 24-cell vLLM standalone study. Each uses one epoch, seed 42, global batch eight and the cached target-generated 4,096-response-cap corpus, with other settings as in the main methods. The inherited native summed-request timings provide speed ratios. These are one-request-per-worker measurements on L40S, with domain caps 2,048/8,192/2,048. They are distinct from the later 96-cell Transformers comparison. Training presentations are not necessarily unique prompts; both counts are shown. No confidence interval for training-seed or independent timing variability is claimed.

Table 26: All 24 five-map vLLM cells. Speed is inherited native summed time divided by mapped summed time; fit seconds exclude cache preparation. This is not the 96-cell Transformers campaign.

Domain	Presented	Unique	Anchors	Speed/native	τ	Fit s
GSM8K	4096	4096	8	1.2637	5.7336	70.20
GSM8K	4096	4096	24	1.2977	5.8984	121.09

Domain	Presented	Unique	Anchors	Speed/native	τ	Fit s
GSM8K	4096	4096	32	1.3172	5.9840	159.76
GSM8K	4096	4096	64	1.3208	6.0202	293.66
GSM8K	4096	4096	128	1.3282	6.0526	548.78
GSM8K	8192	4096	32	1.3368	6.0844	309.52
GSM8K	12288	4096	32	1.3411	6.1023	466.97
GSM8K	16384	4096	32	1.3505	6.1997	618.88
KiCad	4096	4096	8	1.6671	9.9357	178.18
KiCad	4096	4096	24	1.8505	11.0734	260.22
KiCad	4096	4096	32	1.8815	11.2774	319.25
KiCad	4096	4096	64	1.9501	11.6779	448.98
KiCad	4096	4096	128	1.9850	11.9066	788.16
KiCad	8192	8192	32	2.0121	12.0614	662.57
KiCad	12288	12288	32	2.0705	12.4166	1034.44
KiCad	16384	15366	32	2.0993	12.6120	1426.56
NanoCoder	4096	4096	8	1.0407	5.2446	69.94
NanoCoder	4096	4096	24	1.0783	5.4408	125.28
NanoCoder	4096	4096	32	1.0872	5.4929	165.37
NanoCoder	4096	4096	64	1.0894	5.4964	298.66
NanoCoder	4096	4096	128	1.0969	5.5348	586.43
NanoCoder	8192	8192	32	1.1013	5.5495	327.32
NanoCoder	12288	12288	32	1.0974	5.5311	490.16
NanoCoder	16384	16384	32	1.1128	5.6300	656.52

H.2 RANK-56 AUF BANK AND A POOLED ALTERNATIVE

The bank in Table 2 trains only $\Delta F_e = B_e A_e$, rank 56 and scale one, with random A_e /zero B_e initialization. The native fusion, target, embeddings, output head, normalization and draft body stay frozen. Each domain uses 4,096 unique cached examples, a 4,096 response cap, eight sampled blocks per example, 16,000 presentations and 2,000 updates. Two GPUs, microbatch two and accumulation two give batch eight; AdamW uses 10^{-4} , zero weight decay, clipping one, 100-update warmup/cosine and seed 42. Frozen-body backpropagation is still required.

Domain	Native tok/s	Bank tok/s	Ratio	τ : native/bank	Fit min
GSM8K	121.85	153.44	1.2593	4.477/5.691	3.83
KiCad	159.37	213.37	1.3389	5.953/7.912	10.51
NanoCoder	139.34	147.55	1.0590	5.042/5.356	4.54

These standalone rates pool tokens over summed request seconds. Fit time excludes rollout generation, feature extraction and evaluation. The serving engine reuses these exact fits through request-indexed low-rank residual computation, padded from rank 56 to 64. This differs from a standalone folded export and from the newer five-full-map bank.

The pooled control trains one rank-56 update over 12,288 unique examples (4,096/domain), retaining 16,000 presentations/domain, 48,000 total and 6,000 updates. Domain-homogeneous round-robin batches use the corresponding target features, with shared optimizer moments; the schedule advances once per triplet of domain updates. Only its final checkpoint is evaluated. Fitting takes 27.12 minutes on two GPUs. One pooled correction removes draft-bank selection, but target LoRA selection remains.

Selection	Repeat	Native tok/s	Separate tok/s	Pooled tok/s	Pooled/native
Learned	0	1280.52	1691.87	1682.03	1.3136
Learned	1	1285.61	1702.59	1695.52	1.3188

For explicit ID, the two pooled/native ratios are 1.3123 and 1.3147. Across the two repeats, pooled/separate aggregate throughput is 98.94–99.60% with explicit selection and 99.42–99.58% with learned selection. The pooled learned acceptance is 7.6188 versus 7.6798 for separate experts and 5.8560 native, using $1 + A/V$. Fresh learned-policy AR rates are 195.45/195.59 tok/s; inherited native and bank timings remain cross-trial comparisons, not new paired hardware repeats. These data do not establish a substantial benefit from separate draft experts on this three-domain mixture.

H.3 TRAINING-CONTRACT AND OBJECTIVE CONTROLS

In the earlier matched rank-56, three-epoch comparison (4,096 examples, eight anchors), uniform CE versus AUF gives Math speed/native 1.2074 versus 1.2490 and acceptance 5.4482 versus 5.6604; KiCad gives 1.2665 versus 1.3227 and 7.5045 versus 7.8209. AUF fits take 3.41/8.93 minutes versus 3.22/10.00 for uniform CE, excluding data preparation. This evidence is specific to rank-56 fusion; the later five-map comparison against positional decay does not preserve a universal AUF ordering.

The custom AUF-plus-post-failure-decay family has only partial saved output at the evidence cutoff and is not a completed comparative result. No number is inferred for missing domains. Hardware extensions and cancelled experiments likewise do not count as zero effects.

I SCOPE OF COMPARISONS TO RELATED METHODS

Published cross-paper speedups are not interchangeable with our measured controls: workloads, target adapters, hardware, draft sizes, precision and proposal budgets differ. The following table records the methodological distinction and whether a numerical reproduction exists in this study. It is not a performance leaderboard.

Work	Relevant contribution	Comparison and evidence boundary
DFlash (Chen et al., 2026)	Parallel masked-block drafting conditioned on target features.	Native DFlash is the primary same-target comparator; our interface modifies its conditioning, not its verifier.
EAGLE-3 (Li et al., 2025)	Feature-conditioned draft model and training-time testing.	Published drafts evaluated without fitting. Separate runtime/proposal contracts and cap overshoots prevent an algorithm-only ranking.
EDA (Lin et al., 2026)	Shared/private draft adaptation, target-regenerated data and sample selection.	Closest adaptation problem; no complete EDA reproduction. Our body LoRA is an explicit control, not EDA.
RepSpec (Huo et al., 2026)	Redundant linear training structures merged into draft weights for inference.	Folding and optimization-coordinate effects are prior ideas. We test localization to a frozen body’s feature interface; no RepSpec reproduction.
Spec-AUF (Yang & Li, 2026)	Detached CE support through the first draft failure.	The objective is credited to this work; we compare its interaction with adaptation architecture against DFlash decay.
DFlare (Zhang et al., 2026)	Richer per-draft-layer target fusion, deeper drafts and expanded training data.	Contrasts with adapting a fixed pre-trained body. No matched DFlare checkpoint or training comparison is claimed.
OmniDraft (Ramakrishnan et al., 2025)	Online draft distillation, cross-vocabulary translation and adaptive drafting.	Our main LoRA deployment is offline-fitted and same-tokenizer. External cross-family transfer is separately labeled; no numerical OmniDraft reproduction.
Punica / S-LoRA (Chen et al., 2023b; Sheng et al., 2023)	Shared-target heterogeneous LoRA execution and memory management.	Complementary serving infrastructure. We align request identity with draft fusion; no claim to replace their schedulers or scale to thousands of adapters.

The absence of matched EDA, RepSpec and DFlare reproductions limits claims of superiority to those methods. Our supported comparisons are native DFlash, the implemented architecture/objective controls, published EAGLE configurations with disclosed differences, and the separately attributed transfer campaigns.

J PAIRED FEATURE RECONSTRUCTION CONTROL

The target remains Qwen3-4B with the active domain LoRA. On its saved token sequences, let h_i be adapted-target features and y_i base-target features from layers 1/9/17/25/33. The base model provides offline labels only. With frozen fusion blocks F_i and the original RMSNorm N (including

its frozen learned gain), optimize five identity-initialized full maps using

$$D(u, v) = \frac{\|u - v\|_2^2}{\|v\|_2^2 + 10^{-6}}, \quad (14)$$

$$\ell_t = \frac{1}{5} \sum_{i=1}^5 D(W_i h_{i,t}, y_{i,t}) + D\left(N\left(\sum_i F_i W_i h_{i,t}\right), N\left(\sum_i F_i y_{i,t}\right)\right). \quad (15)$$

Average positions within each example, then examples equally. Each domain uses 4,096 saved trajectories capped at 4,096 response tokens. We compare 50% of prompt and response positions sampled separately, and 100% of both segments including the final token. Padding is excluded. Sampling is fixed across epochs. Both feature sides use the same token IDs and Transformers BF16 SDPA. All fits use one epoch, global batch eight, 512 updates, two GPUs, AdamW learning rate 10^{-4} , zero weight decay, 5% warmup/cosine, clipping one and the final checkpoint. Maps and reconstruction arithmetic use FP32. There is no token loss or draft-body backpropagation. Export folds $F_i W_i$ into the BF16 fusion projection. Every row has 32,768,000 trainable parameters.

Domain	Coverage	Mean TPS	Gain/native	τ	Fit s	GPU h	AR ratio
GSM8K	50%	128.54	+1.11%	4.4665	24.00	0.0133	3.312
GSM8K	100%	124.80	-1.83%	4.4696	34.12	0.0190	3.215
KiCad	50%	194.45	+19.05%	7.2083	181.75	0.1010	5.061
KiCad	100%	196.83	+20.50%	7.2064	381.61	0.2120	5.123
NanoCoder	50%	146.78	-2.28%	4.9573	29.13	0.0162	3.763
NanoCoder	100%	148.15	-1.37%	4.9742	37.45	0.0208	3.798

Table 28: LoRA-target five-W reconstruction with half and complete token coverage. Each row evaluates the same 128 prompts/domain. TPS is the mean of request rates; τ includes the correction/bonus token. Fit costs exclude paired prefill, setup, export and evaluation.

The evaluation caps are 2,048/8,192/2,048 for GSM8K/KiCad/NanoCoder. Complete versus half coverage gives essentially unchanged acceptance. Separate timing passes do not establish the significance of small throughput changes. At complete coverage the throughput changes relative to the existing AUF checkpoints are -28.65%, -36.42%, -8.73%, respectively. AUF used vLLM features and block-token supervision, whereas reconstruction uses Transformers paired features. Therefore the comparison remains a practical recipe comparison rather than an isolated loss-only experiment.

Output agreement. In the verified selected Transformers comparisons, native and adapted drafters matched all 128 output sequences per domain. In vLLM, token sequences diverged, consistent with documented numerical and batching variability. Semantic similarity does not imply identical token counts. See the [vLLM batch-invariance documentation](#).

K METRIC CONTRACT

The LoRA audit retains source hashes and selected raw request comparisons. The auditable quantity is the recorded result under its actual contract, not an assumption of universal implementation equivalence.

Define pooled standalone throughput as $Q = \sum_i n_i / \sum_i t_i$, with returned target-token count n_i and end-to-end request time t_i , excluding model loading and warmup. Mean-request throughput $N^{-1} \sum_i n_i / t_i$ gives equal weight to each request and remains the historical ablation statistic. Serving throughput $\sum_i n_i / T_{\text{workload}}$ includes concurrent workload drain and must not be substituted for either standalone definition. Summed latency ratios equal pooled TPS ratios only when output-token counts agree. Committed progress excludes the initial prefill token in our recorded step lists; vLLM’s $1 + A/V$ is a separately labeled accepted-plus-bonus proxy.

The original LoRA five-W AUF rows reproduce pooled native/mapped TPS of 125.8430/164.3052 on GSM8K, 164.3630/308.1137 on KiCad and 145.5864/153.2697 on NanoCoder. These correspond to 30.56%, 87.46%, 5.28%, rather than the historical mean-request gains 37.58%, 89.52%, 8.07%. Those matched-native comparisons are stronger than the AR comparisons.

Every audited multi-LoRA serving request obeys its domain cap and matched methods use identical selected adapter assignments. The 8B aggregate mapper gain varies from positive at low concur-

rency to negative at16/32 clients even though its historical mean-request statistic improves. This distinction prevents a universal serving-scaling claim.

Final-block cap overshoots occur for21 KiCad, one NanoCoder and one8B CodeAlpaca request. We retain actual token counts and measured times; post-hoc token clipping cannot create a correctly timed strict-cap result.

L HISTORICAL LORA STUDY CONTRACTS

L.1 EVIDENCE AND TERMINOLOGY

The following contracts describe the earlier LoRA studies tabulated below. They are distinct from the main matched sweep. Repeated control rows are shared references, not additional independent trials. Original screening and subsequent heldout GSM8K cohorts are distinct 128-question sets, both disjoint from adapter-training questions. Repeated inspection makes later comparisons exploratory.

Native means the original drafter paired with the selected target. Base explicitly means the target without domain adaptation. Full mapper means five dense maps before frozen fusion, not training the entire drafter. Fusion full means training every fusion entry. A suffix such as `svd2_from_r56` denotes compression after fitting, with inherited fitting cost. Rank bounds the additive update, not the complete pretrained matrix. CE labels are actual target continuations, including their errors.

L.2 MODELS AND PINNED IMPLEMENTATIONS

Common Qwen3-4B revision is `1cfa9a7208912126459214e8b04321603b3df60c`; common `z-lab/Qwen3-4B-DFlash-b16` revision is `b74e3a329c4d963783143b1e970d95b002be72bd`. The draft has five blocks, width 2,560, intermediate width 9,728, 32 attention heads, eight KV heads and head dimension 128. Its 537,427,200 checkpoint parameters exclude attached embedding/head weights. Fusion is 2560×12800 . Selected target layers [1, 9, 17, 25, 33] are zero-based; vLLM boundaries [2, 10, 18, 26, 34] denote the same post-layer states. Training uses SpecForge `953d43a0c1c0f5e32989dc43f91ce5fc2d9ddfef`, concrete OnlineDFlashModel. Recorded runtime is vLLM 0.28.0+cu129, PyTorch 2.13.0+cu129, Transformers 5.16.1 and PEFT 0.20.0 where specified. Source-family manifests retain installed runtime hashes. Data selection, capture, parameter selection, export, probes and serving integrations are custom, not unchanged official benchmarks.

Target releases and immutable revisions:

- **Math:** `witcheer/qwen3-4b-gsm8k-grpo`, `f52e1ed1b3accd404d37a9621db0536dd99bdfc5`; rank 32, alpha 32.
- **KiCad:** `AbijahKaj/qwen3-4b-kicad-netlist`, `ba1878e0777bf02c22bceflc3e9b79095181d455`; rank 64, alpha 32, 132,120,576 target-adapter parameters.
- **Subtitles:** `Hookem22/qwen3-4b-subtitle-es-v4adv-lora`, `3b0b976a76472013b9bc99bbae52d11e1c221776`; rank 16, alpha 16.
- **NanoCoder:** `usernamebetter/nanocoder-v1`, `f078bfc2722bdb7bf644906cc7d81662ab3de9d2`; rank 32, alpha 32, attention/MLP LoRA.

These target adapters do not modify embedding/head. Their BF16 deployment differs from publisher quantized setups where applicable. Target LoRA wrappers are kept separate from the draft's shared frozen weight parameters.

L.3 DATASETS AND EXACT PROMPT CONTRACTS

GSM8K. Use `openai/gsm8k` revision `740312add88f781978c0658806c59bc2815b9866`. Python `Random(42)` selects 4,096 unique train and 128 original test questions after stripped-question SHA256 deduplication. A second test subset uses `Random(20260907)`, excluding screening and training hashes. The user prompt is:

```

1566         Solve the math problem. Show brief reasoning, then
1567         end with '#### <answer>'.
1568         Problem: {question}
1569         Solution:
1570

```

Use the released Qwen chat template and thinking disabled. There is no correctness filtering. Adapted-target collection contains 544,585 response tokens, two capped at 4,096; dense prompt/response capture contains 930,941 positions and 23.84 GB features. Base-target controls generate their own answers to the same questions: 558,647 response tokens, one cap. Both labels and features differ in that comparison.

KiCad. Use AbijahKaj/kicad-netlist-sft-dataset revision 52ce8608c3c0fd2f1b6b68e93b8784b88bc92373, the historical s-expression release. Reproduce publisher ordering (tool then direct rows), original messages/template, full-text encoding, ≤ 8192 -token filtering, and `train_test_split(test_size=.02, seed=42)`. The 16,738 retained rows split into 16,403 train and 335 validation. Select 4,096 deduplicated direct-training prompts with `Random(42)`, and 128 direct non-tool validation prompts without exact training overlap using `Random(20260908)`. Preserve system/user messages verbatim; remove assistant references. The system requests complete KiCad s-expression design/components/nets, pins, library and footprint metadata. This reconstructs publisher validation, not independent target-quality evaluation; source repositories can overlap. Collection contains 13,708,854 response tokens, 2,200 caps and 382.246 GB dense features. Native DFlash accelerates generation with the same verifier and bounded exact AR checks; capture replays saved tokens.

Subtitles. Python `Random(42)` selects 1,536 of 1,577 coherent scenes from `Hookem22/subtitles-es-v4adv`, revision 56c15228d70ccc8c5d4def53a0c9fcee8aeb30c4. Retain system/user messages and replace author completions with target-generated labels. The instruction is: “Translate these subtitles from English to Spanish. Output exactly one Spanish line per source line, and keep every line to 42 characters or fewer.” Evaluation uses the English–Spanish test split of `Helsinki-NLP/opus-100`, revision 805090dc28bf78897da9641cdf08b61287580df9. `Random(20260908)` selects unique nonempty English single lines of at most 80 characters, excluding every English line in the author’s training set; group the first 1,024 into 128 batches of eight. The coherent-scene versus unrelated-line mismatch is material and does not reproduce the publisher’s unavailable 36-scene evaluation. Collection contains 319,221 response tokens, no caps. Training uses 576 updates, final checkpoint. Math adapters are reused for cross-workload controls. The declared formatting contract checks line count and 42-character limits: base and target-LoRA pass 6/128 and 56/128. This is formatting compliance, not translation accuracy.

NanoCoder. Use `HuggingFaceH4/CodeAlpaca_20K`, revision 798c567f69c8f4b12fc191015e59ee34e9afe00d, one publisher-listed source. Apply instruction-strip + newline + input-strip and the released 90/10 split (seed 42). Select 4,096/128 train/test with seed 20260909, exact deduplication, no selected test prompt anywhere in source train, and a 3,072-token prompt limit. System: “You are NanoCoder, an expert Senior Full-Stack Engineer and debugging agent.” Use the adapter chat template including its closed empty thinking block, with thinking disabled. Collection contains 996,580 response tokens, 21 caps and 31.68 GB features. All texts were checked for thinking tags. The publisher’s private training mixture is unknown. The original study uses this checkpoint; the separately reported continuations and dense/objective comparisons use the same target and evaluation contract.

1612 L.4 OPTIMIZATION AND ARCHITECTURAL EXCEPTIONS

Historical uniform-CE token training: three epochs, seed 42, 1,536 steps for 4,096 examples, global batch eight (two per GPU, two GPUs, two accumulation microbatches), final checkpoint, no dev selection. Fused AdamW uses betas (.9, .999), epsilon 10^{-8} , zero weight decay, clipping one, 5% warmup and cosine decay from 10^{-4} . Trainable parameters are FP32 with BF16 forward autocast. Vocabulary CE uses two-block chunks. For the original uniform-CE fits, SpecForge settings are `loss_type=dflash`, `loss_decay_gamma=None`; no KL, reward, position decay or balancing loss. Mask ID is 151669. Deterministic shuffling/length bucketing and epoch/rank/microbatch

anchor seeds define the stream. Eight sampled blocks use dense cached context, not dense supervision.

Concrete token-training loop. The historical uniform-CE loop is the following; the later AUF support and 2,000-update recipe are specified in Appendix N. the subtitle and mixture step-count exceptions are specified above and below.

1. Before fitting, generate and persist the selected target’s token sequences once. Replay the same tokens through that target and save all selected layer states in BF16. These target states and labels are fixed during adapter training; no target gradient is computed.
2. For each of three epochs, deterministically shuffle the cached stream, bucket examples by length, and partition microbatches across two GPU ranks. Each rank loads two examples per microbatch. Sample eight eligible distinct anchors per example, construct the clean-anchor-plus-15-mask blocks, and apply the strictly prior context and isolated block attention masks from the block construction described above.
3. Clear trainable gradients once per optimizer update. On each GPU, process two successive microbatches: compute adapted fusion (or the selected body update), forward through the draft, and compute the uniform valid-token CE objective over valid labels. Full-vocabulary CE is accumulated in two-block chunks; chunking does not replace its microbatch-wide valid-token denominator. Backpropagate half the local microbatch loss. Averaging gradients across the two ranks implements

$$\mathcal{L}_{\text{step}} = \frac{1}{4} \sum_{g=1}^2 \sum_{b=1}^2 \mathcal{L}_{\text{micro},g,b}. \quad (16)$$

The draft body still participates in differentiation when frozen: only its weight gradients and optimizer updates are absent. Four local normalizations are deliberately retained, rather than substituting one global token normalization.

4. Clip the aggregated trainable-gradient norm at one; take one fused AdamW update with the stated warmup/cosine schedule. Repeat for 512 updates per epoch on 4,096 examples, totaling 1,536. Save the final trained parameters and export the effective BF16 weights or request-selected factors. Compression is a subsequent operation without retraining.

Fusion residuals, full fusion, full maps, body LoRA, low-rank replacement maps and both dense draft-mixture router variants use this same hard-token CE family. Which parameters receive gradients differs; there is no additional router balance/reward loss. The request-level target classifier instead uses source-domain CE.

The MSE pilot instead uses paired 25% base/adapted features: one random position per four-token stratum separately in prompt and response, including partial strata. Identity-initialized square maps minimize

$$D(a, b) = \frac{\|a - b\|^2}{\|b\|^2 + 10^{-6}}, \quad \ell = \frac{1}{5} \sum_j D(M_j h_j, y_j) + D(\hat{z}, z_0), \quad (17)$$

where $\hat{z} = N_0(F_0[M_j h_j]_j)$ and $z_0 = N_0(F_0[y_j]_j)$. With n_i sampled positions/example, P total positions and N examples, batch loss is $[(P/N)/2048] \sum_{(i,t) \in B} \ell_{it}/n_i$, retaining denominator 2048 on partial batches. Train one GPU, 348 steps, peak 10^{-3} ; its 32.775s includes export. It is not a controlled loss-only comparison with CE training.

Full maps and direct full fusion each train 32,768,000 parameters; body-r112 trains 32,112,640. Low-rank replacement maps have no residual identity bypass. Their initialization uses uncentered PCA of first/midpoint/last response positions in each training sequence: $B_i = Q_i$, $A_i = Q_i^\top$. There are 12,288 calibration positions. Five rank- r replacement maps bound total fusion rank by $5r$; additive residual fusion retains the original matrix. If all frozen fusion blocks are invertible, unrestricted square maps can represent any effective fusion, but optimization still differs.

L.5 COMPRESSION, INTERVENTIONS AND DIAGNOSTICS

SVD acts on the effective raw FP32 BA , not on either factor alone. Keep $\Delta F_k = U_k \Sigma_k V_k^\top$ or remove it with $F_0 + \Delta F - \Delta F_k$. Gram eigendecomposition with TF32 disabled, or thin QR and

a small-core SVD, implement the calculation with reconstruction checks. Dense/full-map compression instead subtracts exported BF16 matrices and includes rounding. Rank-two factors store 30,720 numbers but inherit rank-56 training. Body-r3 compression truncates each of 35 projections independently to rank one. No compression retrains weights.

Math/KiCad interventions estimate constants from first examples in 32 uniformly spaced training shards: 6,476/131,597 calibration positions. The earlier math-only mechanism study averages all 930,941 training positions, a different protocol. Heldout mean-energy fractions describe response corrections; evaluation means never construct the tested constant. Probes use 32 length-stratified prompts, at most 4,096 saved response tokens, and eight evenly spaced anchors. CE/top1 normalize by valid slots; all-15 metrics require complete blocks. Different live draft policies visit different anchors, so unpaired live first-token rates do not establish same-prefix prediction effects.

Gradient tests differentiate native CE with respect to FP32 fusion output, checking finite differences at $\pm.05$ and $\pm.1$; BF16 makes derivatives approximate. Signed coordinate permutations preserve correction norm/singular values, not its functional direction. Target-LoRA output-span tests use matching o/down-projection residual coordinates, not arbitrary factor angles. If Q_i spans these output directions, report $\|\Delta F_i Q_i\|_F^2 / \|\Delta F_i\|_F^2$ and divide by $\text{rank}(Q_i)/2560$ for descriptive isotropic enrichment.

NanoCoder diagnosis uses all 128 saved prompts, 1,024 anchors and 13,370 valid slots. Of 827 anchors where native fails before labels end, 180 extend past that failure; across all anchors, 102 fail earlier after adaptation. Net corrected slots are 362. Probability changes at wrong-in-both slots account for 99.0% of net summed CE reduction, not 99% of predictions. Geometry on 28,332 response positions has 90.97% mean energy and 99.640/99.992% top-one/two weight energy. This is descriptive, not a constant-vector deployment experiment.

L.6 SERVING LAYOUTS AND TIMING

Standalone timings use native vLLM verification and fixed selected targets. Math/subtitle/NanoCoder output caps are 2,048, KiCad 8,192. Context limits vary by workload from 5,120 to 16,384. FLASH_ATTN, batch invariance, synchronous scheduling, compilation zero and resolved FULL_DECODE_ONLY graphs are used where reported. Prefix caching is off. Setup, warmup, instrumentation and cold-JIT attempts are separate. Prompt-bootstrap intervals use 10,000 samples except cross-size’s 2,000; neither measures hardware-trial uncertainty.

The independent Math/KiCad dense-mixture comparison uses $N_0(F_0 h + \sum_{e=1}^2 p_e(h, d) B_e A_e h)$ with $p = \text{softmax}(G \text{RMS}(h) + b_d)$; both experts execute. Known target ID selects the actual target and router bias. Initialize $G = 0$, diagonal bias +2, off-diagonal -2. Router-only freezes fitted experts; scratch fits fresh random- A /zero- B experts and router jointly. Uniform block CE, no balancing loss, 4,096 examples/domain, three epochs and 3,072 updates apply. Router-only trains 25,604 parameters in 801.6 seconds; scratch trains 1,745,924 in 802.0 seconds, both on two GPUs. Router-only inherits math/KiCad expert fitting of 179.8/636.9 seconds. Evaluation alternates target IDs with one outstanding request per engine, 128/domain, and includes routing/weight-selection RPC time. It is not concurrent heterogeneous serving. Scratch changes optimization history, so its small advantage does not identify a routing benefit. The current concurrent deployment instead selects exactly one expert per request (Appendix N).

Fit seconds exclude collection/capture/setup except the explicitly identified MSE timer. Family allocation includes additional stages. Reused-control and inherited-fit rows must not be summed as independent costs. Preparation break-even divides preparation GPU-seconds by measured one-GPU seconds saved/request and is specific to that workload. No aggregate project cost is inferred from overlapping report totals.

L.7 IDENTITIES USED IN INTERPRETATION

The algebra used here is standard, not a claim of new theory. Projection onto $-t$ equals projection onto t because the two signs cancel. Expanding squares and using $\sum_n (c_n - \bar{c}) = 0$ yields $\sum_n \|c_n\|^2 = N \|\bar{c}\|^2 + \sum_n \|c_n - \bar{c}\|^2$. An integer $L \in \{0, \dots, K\}$ satisfies $L = \sum_{k=1}^K \mathbf{1}\{L \geq k\}$,

whose expectation gives Equation 8. None of these identities proves low-rank sufficiency, semantic interpretation or a correspondence between CE and runtime speed.

M ADDITIONAL STUDY CONTRACTS

These are earlier continuation and steering studies; their results and fitting costs are retained separately from the main matched sweep.

M.1 NANOCODER CONTINUATIONS AND DENSE OBJECTIVES

All conditions use the NanoCoder target and source pins in Appendix L. The 18,016-prompt set is a deterministic nested expansion of the original 4,096, rounded to full 32-example shards. A 16,384 response cap and 20,480 context limit apply to collection. Reuse naturally stopped original outputs and compatible feature shards; regenerate capped outputs with exact prefix checks. Capture all five BF16 hidden states on the saved adapted-target tokens. The expanded collection has 5,260,572 response tokens and 161.62 GB logical feature storage; hard links mean logical size is not incremental disk use. Evaluation remains 128 CodeAlpaca source-test prompts at a 2,048 output cap. Target training overlap is unknown.

Each continuation independently starts from the initial final fusion checkpoint, resets AdamW, and runs three epochs. Small-data continuations have 1,536 updates; 18K continuations have 6,756. The common two-GPU/global-batch-eight, seed-42, LR- 10^{-4} recipe is unchanged. Eight-anchor sampling does not scale its label count with rollout length. The final checkpoint is always used. The expanded fit has more optimizer updates as well as more examples.

For a response starting at p with length T , dense tiling uses

$$\mathcal{A} = \{p + 15j : p + 15j < p + T - 1, j \geq 0\}. \quad (18)$$

Each clean anchor predicts the next 15 response tokens, omitting unavailable labels. This gives exactly $T - 1$ labels per epoch, not every overlapping window. Total eligible labels are 5,242,556 per epoch. Up to 64 blocks per example are processed in one chunk, with masked future context omitted. The microbatch gradient is the sum of chunk numerator gradients divided by the complete microbatch denominator. For AUF, support weights are detached, so this denominator is computed from the full dynamic support rather than averaging independently normalized chunks. Microbatch gradients are then averaged across two accumulation steps and two GPU ranks, preserving the existing mean-of-microbatch contract.

Uniform, decay and AUF fits share initialization, positions, three epochs and optimizer settings. Uniform fitting is inherited from the dense experiment; all inference conditions receive fresh timing. Decay uses pinned SpecForge; dense tiling, bounded accumulation and the AUF formula implementation are custom. The 15,727,668 eligible slots must not be described as AUF’s nonzero-loss count. Evaluation uses fixed target references, disjoint prompt workers and variant-order rotation; summed single-request time is distinct from concurrent serving makespan. Paired intervals resample complete prompts 10,000 times with seed 20260909. Capped outputs and repetitive continuations are preserved as target behavior, not labeled correct code.

M.2 DIRECT STEERING AND READER CALIBRATION

The Math/KiCad comparison uses the existing 4,096 adapted-target rollouts per domain and frozen five-layer features. A bias adds 2,560 trainables, fusion adds 860,160, and their combination adds 862,720. Bias initializes at zero with LR 10^{-2} ; fusion factors use the original random- A /zero- B initialization and LR 10^{-4} . Three epochs, 1,536 updates, seed 42, uniform CE and final checkpoint apply to all. Before frozen RMSNorm, inference adds FP32 bias to BF16 fusion output and converts to BF16. Static fusion exports fold the matrix update; dynamic serving retains separate factors.

The training mean and reader calibration use the first example from each of 32 uniformly spaced training shards, including all prompt and response positions. They never use evaluation features to construct a correction. Reader calibration follows Equation 25. The random control uses a Gaussian 12800×56 basis, seed 20260909, orthonormalization and the identical fitting rule; only the input

subspace changes. Calibration takes 0.65 seconds on Math and 9.34 seconds on KiCad on CPU and inherits the fitted fusion and its mean output direction.

Twenty-one interventions per domain use 32 length-stratified held-out prefixes and up to eight fixed anchors each, with at most 4,096 cached response tokens. The live KiCad cap is 8,192, so these probes do not cover its full generation horizon. In addition to the readers, compare native, full, mean, centered, bias, half/double/negative bias, three signed permutations, norm-matched foreign bias, joint fit, each joint component alone, and radial/tangent bias. Report full slots, margins, first-failure changes and paired-prefix intervals. Geometry uses FP32 F_0h ; scoring applies FP32 corrections to BF16 fusion output before rounding. The RMSNorm Jacobian is checked using centered FP64 finite differences at step 10^{-4} and relative tolerance 10^{-5} . Record finite radial sign crossings, which are zero here. The local linearization does not justify arbitrary large radial perturbations.

The combined loss interaction $I = L(c_c + b_c) - L(c_c) - L(b_c) + L(0)$ is positive, 0.04618/0.03742 in mean per-prefix Math/KiCad loss. It indicates overlapping benefit for this decomposition, not identifiable attribution. Reader and intervention experiments do not measure live runtime. All calibration and causal findings retain their conditioning on the chosen prefixes and fitted checkpoints.

N AUF EXPANSION AND FINAL DEPLOYMENT CONTRACTS

Matched sampled-AUF bank. All three experts use the model/data pins in Appendix L, 4,096 unique cached adapted-target trajectories and 4,096 maximum response tokens, with EOS stopping. Five-layer features are stored densely in BF16; eight distinct eligible response anchors are sampled at each presentation. Thus dense storage and sampled loss are deliberately different. Rank 56, alpha 56, no bias/dropout, random A and zero B from native fusion, seed 42 and final-checkpoint export apply. Only 860,160 fusion parameters train.

The exact loss is Equation 7, with AUF correctness determined by current draft argmax predictions at earlier eligible slots in the same block. Correctness and its prefix-product support are detached. The first incorrect slot is included; later slots, clean anchors, prompts and padding are excluded. No ground-truth dataset answer, target gradient, teacher-logit KL or router balance term is used. Two GPU ranks each process two examples per microbatch and two microbatches per update. The four token-normalized microbatch losses are averaged equally. AdamW uses $(\beta_1, \beta_2) = (0.9, 0.999)$, epsilon 10^{-8} , weight decay zero, gradient clipping one, LR 10^{-4} , 100-update linear warmup and cosine decay ending at step 2,000. Cycling cached data gives 16,000 cumulative presentations, not 16,000 unique examples. Save steps 500/1,000/1,500/2,000; Math/KiCad evaluate only steps 1,000/2,000, NanoCoder only the final step. No dev or best-loss selection occurs. Math/KiCad final comparisons inherit complete native/uniform measurements from the preceding AUF study; NanoCoder times native/mapped afresh. All use L40S, with no requirement for matching physical GPU UUIDs.

The separate unique-data KiCad study uses all 15,366 eligible deduplicated prompts, including 11,270 new greedy rollouts at the same cap. One pass has 1,921 updates with a six-example final batch split equally across ranks. Last-microbatch weighting follows example fractions; full batches retain the previous local normalization. Five-percent warmup/cosine now spans this run. Evaluate cumulative 8,000 and 15,366 checkpoints. This changes unique coverage and schedule relative to cycling 4,096 examples. The corresponding tables must not be labeled a pure loss or fixed-compute comparison.

Dense causal-objective negative grid. The 18,019 NanoCoder prompts are the full CodeAlpaca source-train split; 18,016 cached trajectories are reused and three additional ones generated at the 16,384 response cap. Actual responses total 5,260,941 tokens (mean 291.97; median 195; p99 853; 72 caps). Teacher capture additionally saves the adapted target’s final normalized pre-head state $g_t \in \mathbb{R}^{2560}$. All five-layer states and final states are dense. For prompt length p , supervise $t = p - 1, \dots, n - 2$, including the first response token. Draft query t sees target context $k < t$ and clean draft-stream tokens $k \leq t$, with common absolute positions. This custom causal stream is distinct from DFlash’s deployment masks. With frozen vocabulary head U , draft final normalized

state z_t and T response labels,

$$L_{\text{NWP}} = -\frac{1}{T} \sum_{t=p-1}^{n-2} \log \text{softmax}(U z_t)_{x_{t+1}}, \quad L_{\text{state}} = \frac{1}{2560T} \sum_{t=p-1}^{n-2} \|z_t - g_t\|^2. \quad (19)$$

The grid uses either loss or their sum with coefficient one. Unlike the block recipe, updates normalize by the actual global response-token count. CE is full vocabulary in checkpointed 128-token projection chunks. All architectures start from the BF16 reconstruction F_* of the initial fitted fusion: $F_* h + B A h$, a fully trainable $W h$ initialized at F_* , or $F_* h + B \text{SiLU}(A h)$. Residual factors have rank 56 with random A /zero B ; full fusion has 32,768,000 parameters. Target, body and norms remain frozen. Three epochs, global batch eight over two GPUs, retained three-example final batch, 6,759 updates, 5% warmup/cosine and the common AdamW settings apply. There are 15,782,823 supervised token presentations. Final checkpoints alone are evaluated on the same 128 prompts with cap 2,048. The nonlinear update remains explicit in vLLM and its cost is included. The previous AUF reference differs in recipe, initialization history and three source rows; it is not a loss-only comparator to this grid.

Concurrent engine and request contract. Use vLLM 0.28.0 AsyncLLM and upstream LoRARequest/Punica metadata with the audited custom request-indexed fusion hook. One BF16 target and one draft body share frozen embedding/head tensors without target-LoRA wrappers propagating into the draft. Each request’s single target ID selects the same expert for every associated draft-context row. Grouped shrink/expand uses rank-56 factors padded with zeros to rank 64 at scale one, no bias, stable buffers and FP32 shrink workspace. Original target verification is unchanged. Bounded checks cover heterogeneous rows, factor export, canonical prompt tokens, target output identity, invalid requests and JIT-free timed passes; these checks do not constitute a mathematical proof for arbitrary future workloads.

The pool has 128 unique mapper-held-out requests/domain and no exact router-train/evaluation text overlap. Preserve each adapter’s canonical system/user template; for NanoCoder preserve the source prompt’s trailing newline. Thinking is disabled, temperature zero, top-p one, top-k disabled, seed zero; output caps are 2,048/8,192/2,048 for Math/KiCad/NanoCoder. Context limit 16,384, maximum twelve sequences and 8,192 batched tokens, prefix caching off and batch-invariant target computation apply. Twelve closed-loop HTTP clients admit requests in the seeded shuffled order. Two repeats use seeds 20260911 + repeat, keeping each native/mapped pair identical and reversing policy order across repetitions. Each job uses one L40S engine; at most four GPUs are active. Setup, short warmup and bounded whole-workload JIT warmup remain outside steady timing. Every retained pass has no new JIT event. Latency starts after the client semaphore and ends at final stream completion; makespan includes drain. TTFT ends at first nonempty content, and TPOT is subsequent duration divided by output length minus one, with denominator one for a single token. Each policy has 1,536 exact-reference comparisons across repetitions/modes; together there are 3,072. Router misclassification is evaluated against the selected target’s AR sequence, not the source-domain target’s sequence.

Prompt-only target classifier. Fit scikit-learn 1.5.2 TF-IDF plus multinomial logistic regression on 4,096 user-only training messages per domain. Strip the Math wrapper for classification, exclude system text, answers, target IDs and response caps as features. Use default Unicode word tokens of length at least two, lowercase unigrams/bigrams, min document frequency two, maximum 32,768 features, sublinear term frequency, smoothed IDF and L2 normalization. With count c_j , document frequency df_j and $N = 12288$,

$$z_j = \mathbf{1}[c_j > 0](1 + \log \max(c_j, 1)) \left(1 + \log \frac{1 + N}{1 + df_j}\right), \quad (20)$$

$$\phi(x) = z/\|z\|_2, \quad p(e \mid x) = \text{softmax}(W \phi(x) + b)_e, \quad (21)$$

$$L_{\text{router}} = -\frac{1}{N} \sum_i \log p(d_i \mid x_i) + \frac{\|W\|_F^2}{2CN}, \quad C = 1. \quad (22)$$

Use zero features when the norm is zero. The intercept is unpenalized. Float64 L-BFGS uses tolerance 10^{-6} , at most 300 iterations and seed 42; actual fit is 48 iterations, 0.944 seconds, 98,307

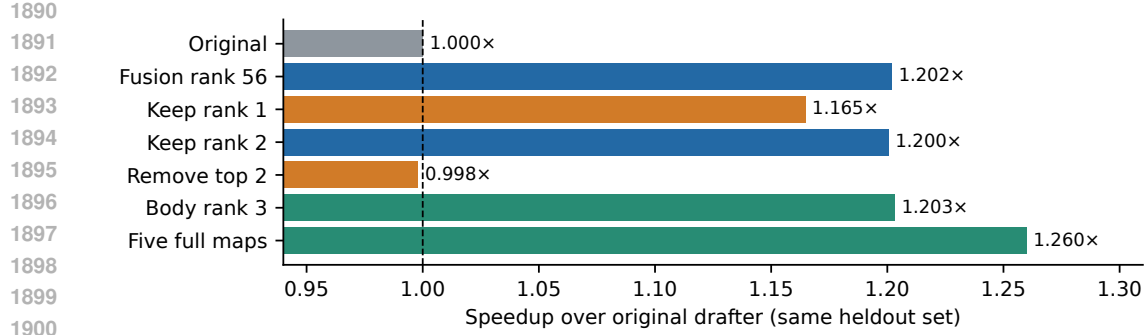


Figure 3: Same-checkpoint interventions on the same heldout adapted-target GSM8K cohort. The two leading update components retain the rank-56 benefit; the remainder alone does not. Full maps and body LoRA are independently fitted comparators.

parameters. Training-only vocabulary/IDF and frozen NumPy export agree with canonical probabilities to 7.77×10^{-16} over 12,672 training/evaluation inputs. Top-1 is selected once before template rendering; no client domain ID is accepted in learned mode. This is supervised source-domain classification, not a sparse target-weight MoE, quality oracle or confidence-calibrated abstaining router. The explicit-ID policy does not use this classifier.

O ADDITIONAL ARCHITECTURE AND UNIFORM-CE MECHANISM EVIDENCE

The following completed studies motivate the final mapper recipe and retain the earlier architecture comparisons. Unless specified otherwise, their fusion fits use uniform block CE; the final AUF intervention results are reported separately in Appendix P.

Objective and adaptation location. On the second heldout cohort, fusion-r56 gives 1.2019x and body-r3 1.2032x; their difference is small. KiCad differs: fusion-r56 gives 1.2660x, while the matched-budget body-r3 control gives 1.4136x. Fusion is a useful adaptation location, not universally the strongest one.

Five low-rank replacement maps perform poorly despite retaining the original fusion. At the matched 768,000-parameter budget, math replacement-r30 gives 0.6112x native speed, versus 1.2116x for residual fusion-r50. Full square maps reach 1.2679x. An additive low-rank change preserves the pretrained path; a low-rank replacement can discard useful directions.

Compression is real, but not sufficient. The top two singular components of the raw math rank-56 update retain 99.926% of its squared weight energy and 99.43% of its heldout latency reduction. Removing those two components returns speed to 0.9980x native (Figure 3). Across seeds 42/43/44, fusion-r56 gives 1.196–1.202x, and compressed rank two gives 1.195–1.200x. Direct rank-two training is weaker under the tested recipe: approximately 1.136x. Compression inherits source fitting cost and does not establish equally easy rank-two optimization. Likewise, truncating a folded full-map update to rank 56 retains 88.706% of its weight energy but only 1.0653x screening speed, versus 1.2534x for rank-56 compression of directly trained full fusion, despite the latter retaining only 63.234% of its weight energy.

Reuse beyond a target LoRA. Math fusion learned from adapted-target data also improves the base target by 1.1926x. Base-generated/base-feature training performs similarly to adapted-target training on the adapted target: fusion-r56 speed ratio 1.00094, paired-prompt interval [0.9951, 1.0072]. These controls favor workload adaptation over necessary repair of the target LoRA. They do not separate changes in labels from changes in features.

Table 29: Response geometry and fixed-prefix loss interventions. Mean energy is descriptive on heldout features; the tested constant μ is independently estimated from training features. CE retention is not speed retention.

	Math	KiCad
Mean-component energy (%)	92.60	90.34
Energy parallel to t (%)	0.23	0.49
Native CE	3.0737	2.1279
Full CE	1.8724	1.3407
Constant CE	1.9914	1.4449
Undo-shift CE	3.0829	1.7293
Perpendicular CE	1.8692	1.3717
Inherited fusion fit (two-GPU seconds)	179.8	636.9

O.1 GEOMETRY OF THE EARLIER UNIFORM-CE UPDATES

For an identical prefix, let h^L, h^0 denote adapted and base features, $u = F_0 h^L$, $u_0 = F_0 h^0$, $t = u - u_0$, and $c = BA h^L$. Feature restoration would suggest $c \approx -t$. We decompose each correction by position:

$$c_{\parallel} = \frac{c^{\top} t}{\max(\|t\|^2, \epsilon)} t, \quad c_{\perp} = c - c_{\parallel}. \quad (23)$$

We analyze native, full, reversed, parallel, perpendicular, undo-shift, training-mean, centered, the other domain’s expert and three signed-coordinate permutations on Math and KiCad. Interventions occur before frozen RMSNorm. BF16 hook arithmetic differs slightly from merged exports (maximum audited per-prompt CE difference 0.0142); these are offline probes, not timing implementations.

Steering rather than feature restoration. Mean cosine between c and t is $-0.0294 / -0.0513$ on Math/KiCad; only 0.23%/0.49% of correction energy is parallel to t . Removing that component preserves 100.3%/96.1% of CE reduction. Exactly undoing the target shift helps KiCad less than the fitted correction and slightly harms Math (Table 29). This argues against simple feature restoration for these checkpoints, not against all representation-matching objectives.

A large mean component. Estimate $\mu = \mathbb{E}_{\text{train}}[BAh]$ from fixed training-shard subsets. Compare $u + c$, $u + \mu$ and $u + c - \mu$. The independently training-derived constant retains 90.1%/86.8% of CE reduction on Math/KiCad (Figure 4). Separately, held-out response mean-energy fractions $N \| \bar{c} \|^2 / \sum_n \| c_n \|^2$ are 92.60%/90.34%. Low-rank weights alone do not imply near-constant outputs, and an evaluation-derived mean is not used to construct these interventions.

Direction and specificity matter. Reversing c raises CE to 5.964/6.428. Signed-coordinate permutations preserve norm and singular values but lose the benefit: CE ranges 3.117–3.223 on Math and 2.804–3.076 on KiCad. Using the other domain’s expert gives CE 2.9346/2.0168, weaker than the corresponding own expert. Sensitivity enrichment in matching target-LoRA residual-output subspaces is $1.05\text{--}1.73\times$ on Math and $1.88\text{--}2.77\times$ on KiCad, but absolute energy is only 2.62–4.33% and 9.38–13.84%. These are descriptive projections, not a semantic interpretation or a calibrated causal null. Native-loss gradients are descending along the correction on all 32 prefixes in each domain. No meaning is assigned to individual singular vectors.

Loss is not accepted length. With $K = 15$, let L count consecutive accepted draft proposals and C_k count verification events accepting at least k proposals. Then

$$\mathbb{E}[L] = \sum_{k=1}^K \Pr(L \geq k), \quad \hat{\tau} = 1 + \frac{\sum_k C_k}{V}, \quad (24)$$

where V counts verification events and the one denotes the verifier token convention. This identity requires no independence assumption. CE can improve without changing argmax, and later corrected positions cannot extend a prefix terminated by an earlier mismatch. NanoCoder illustrates

1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051

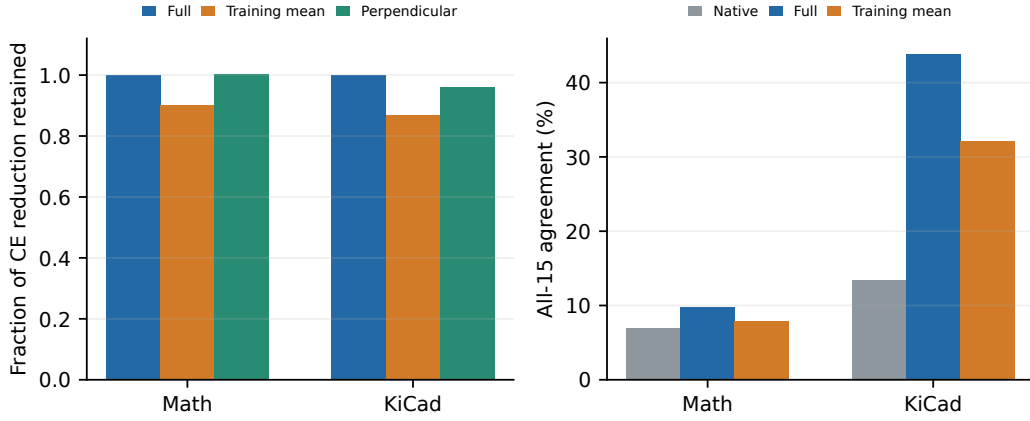


Figure 4: Fixed-prefix interventions on 32 prompts per domain. Left: training means and perpendicular corrections preserve much of the full CE reduction. Right: the mean preserves less all-15 agreement than full fusion. CE retention is not a live speed measurement.

this: fixed-prefix CE improves 2.2607 to 1.9465, but live speed improves only 1.0194 $\times$. Of 13,370 probe slots, 916 change wrong-to-correct and 554 regress; 99.0% of net summed CE reduction is attributed to positions wrong in both conditions. These are boundaries of the current recipe, not universal coding limitations.

P FINAL AUF MAPPER INTERVENTIONS

This study reuses the exact final individual checkpoints of the historical individual-AUF study and the pooled checkpoint of the historical pooled-AUF study; no training or target decoding is performed. Individual parent fitting takes 3.83/10.51/4.54 two-GPU minutes; pooled fitting takes 27.12. Both A and B come from the unmerged FP32 fitted update. Architecture and AUF recipe are unchanged. Source hashes are included in the frozen probe summary and raw per-prefix records.

For each domain, take the first training row of each of 128 fixed feature shards, use response positions only, and calculate the token-weighted mean correction. This differs from the earlier uniform-CE reader calibration’s 32-example prompt-plus-response subset. The means are independently calibrated for each checkpoint/domain, and calibration IDs are disjoint from evaluation. The heldout descriptive mean-energy fraction in $N\|\bar{c}\|^2 / \sum_n \|c_n\|^2$ uses the heldout mean only to summarize geometry, never to construct an intervention.

Math and KiCad retain their existing 32 diagnostic prefixes, with at most 4,096 cached response tokens. The KiCad live cap is 8,192, so these probes do not cover its full generation horizon. NanoCoder selects 32 length-stratified prefixes from its 128 captured evaluation prefixes. Each prefix uses up to eight evenly spaced eligible response anchors; the clean anchor, unavailable future labels and padded positions are excluded. Predictions use the actual adapted target’s saved features and labels throughout. All conditions receive identical anchors, token IDs and masks. Fixed-prefix draft length counts consecutive matching proposal slots without the verifier token; it is not the live vLLM τ statistic.

Thin QR decompositions of B and $A^\top$ reduce SVD to a 56×56 core. FP64 decomposition verifies reconstruction to relative error below 10^{-10} ; truncated factors are then used in FP32 interventions. Keeping the top two means truncating the singular directions of the complete product BA , not either factor. Removing them preserves the rest of that product. Random controls use CUDA generators with seeds 10/11/12, an output-coordinate permutation and independent signs; every position’s correction norm is unchanged. Negative, full, constant and mean-removed conditions complete the principal intervention set.

A hook adds the selected FP32 correction to the native BF16 fusion output before RMSNorm and casts back to BF16. This is an intentional diagnostic implementation; merged inference has dif-

Table 30: Live steering comparisons on 128 prompts per domain. Each entry shows native-relative speedup / acceptance length τ . The mean inherits its parent fusion fit. Fitting times use two GPUs and exclude data collection and feature capture; *inherited parent fitting, with no new optimization.

Correction	Params	Math: speed / τ	KiCad: speed / τ	Fit s: Math/KiCad
None	0	1.0000/4.4771	1.0000/5.9526	—
Training mean	2,560 stored	1.1281/5.0789	1.1880/7.0554	193.0 / 599.8*
Learned bias	2,560	1.0589/4.7534	1.0965/6.5208	191.8 / 600.5
Fusion-r56	860,160	1.2069/5.4482	1.2661/7.5045	193.0 / 599.8
Bias + fusion	862,720	1.2069/5.4588	1.2665/7.5129	194.7 / 601.4

ferent rounding. Zero corrections recover the native token predictions and CE exactly. A separate canonical-forward comparison verifies the fixed-anchor CE readout. Training/model/feature pins inherit the contracts in Appendix L.

First-failure repair is the fraction of native-failing blocks for which the adapted prediction corrects that first failure while preserving every earlier matching slot. Prefix regression is the fraction of all blocks that become wrong at an earlier slot in the native matching prefix. Report both counts and denominators, CE, top1 agreement, target margin and full fifteen-position prefix survival. Paired bootstrap intervals resample the 32 entire prefixes, 10,000 times with seed 42; anchors are not independently resampled.

Additional ± 0.01 correction scales measure centered finite differences of margins against each native competitor. These use BF16 arithmetic, so they are quantized diagnostic slopes, not exact analytic Jacobians. Native first-failure argmax also depends on every competing token and preservation of earlier slots. We retain these controls without treating their fixed-competitor signs as a theorem of acceptance. The detailed theory study remains separate from these measured interventions.

The full condition tables are in Appendix S.22. The frozen JSON includes spectra, training and held-out correction means, energy decompositions, per-condition acceptance metrics, bootstrap intervals, finite-difference statistics and pooled cross-domain mean cosines. Raw per-token arrays remain in the experiment family.

Q FROM FUSION GEOMETRY TO USEFUL STEERING

The preceding interventions show that fusion updates often have a large constant component. They do not show whether a constant is an adequate replacement in live decoding, whether it can be learned directly, or whether its input direction is special. We test these questions separately, keeping the adapted target fixed.

A constant helps, but leaves useful adaptation behind. For Math and KiCad, we independently fit a zero-initialized bias b , a rank-56 fusion update, and their combination. We also extract $\mu = \mathbb{E}_{\text{train}}[BAh]$ from the new fusion fit. The compared inputs to frozen RMSNorm are $F_0h + b$, $F_0h + BAh$, $F_0h + BAh + b$, and $F_0h + \mu$. Each fit uses 4,096 cached target rollouts and three epochs. Bias coordinates use learning rate 10^{-2} , versus 10^{-4} for fusion factors; this is a practical recipe comparison, not an identical-optimizer capacity test.

The mean retains approximately 62%/71% of the *incremental speedup*, computed as $(S_\mu - 1)/(S_{\text{fusion}} - 1)$ for Math/KiCad. This is appreciably less than its approximately 90%/87% retention of fixed-prefix CE improvement. The directly fitted bias is weaker even though the extracted mean is also a constant vector; weaker optimization results therefore do not prove insufficient constant-vector capacity. Adding a bias to fusion provides no resolved speed advantage: paired-prompt ratios are 1.0000 [0.9971, 1.0028] and 1.0004 [0.9996, 1.0011]. Fusion fits take 193.0/599.8 seconds on two GPUs; the much smaller bias fits take 191.8/600.5 seconds, because the frozen drafter still participates in forward/backward computation.

KiCad’s gain is not restricted to outputs stopped at the token cap. On 110 naturally stopped requests, fusion gives $1.2618\times$ native speed; on 18 capped requests it gives $1.2824\times$. Natural stopping does

not certify valid circuitry, but this split rules out an explanation confined to capped continuations in this sample.

An almost constant reader need not encode a new skill. On 32 fixed held-out prefixes per domain, the new fusion fits reproduce the mean-dominance finding. To test its input side, let $Q \in \mathbb{R}^{12800 \times 56}$ be an orthonormal basis of the learned A row space, $z = Q^\top h$, $m = \mathbb{E}_{\text{train}} z$, and $C = \text{Cov}_{\text{train}}(z)$. A calibrated minimum-variance reader is

$$q = \frac{(C + \lambda I)^{-1} m}{m^\top (C + \lambda I)^{-1} m}, \quad c_{\text{reader}}(h) = \mu q^\top Q^\top h, \quad (25)$$

with $\lambda = \max(10^{-8}, 0.01 \text{tr}(C)/56)$. It minimizes $q^\top C q + \lambda \|q\|^2$ subject to $q^\top m = 1$. Calibration uses training features only and no further CE fitting. Orthonormalization makes this test independent of arbitrary scaling between the LoRA factors.

The learned-space reader retains 91.0%/88.3% of full fusion’s CE reduction. Crucially, the same calibration in a random 56-dimensional input subspace, with the *same learned output direction* μ , retains 89.6%/85.8%. Its held-out scalar is less constant (coefficient of variation 0.191/0.204 versus 0.131/0.101), but still useful. This control supports a contribution from generic feature anisotropy; it does not show that a random correction direction works. Full fusion outperforms every tested rank-one reader: learned-reader CE excess is 0.1084 [0.0656, 0.1564] on Math and 0.0917 [0.0527, 0.1372] on KiCad. One random seed and these diagnostic prefixes do not establish a universal mechanism.

Direction changes survive normalization. For $N(u) = g \odot u / r$, $r = \sqrt{\|u\|^2 / d + \epsilon}$, its Jacobian acts as

$$J_N(u)b = \frac{g}{r} \odot \left(b - \frac{u(u^\top b)}{dr^2} \right). \quad (26)$$

Thus a constant vector before RMSNorm can induce an input-dependent normalized shift. Median relative linearization errors are 0.0138/0.0173. Radial-only bias barely changes CE, and tangent-only bias tracks full bias; however, only 0.17%/0.23% of the fitted bias energy is radial initially. This is consistent with directional steering, not a causal identification of RMSNorm as the source of the gain. No no-normalization counterfactual was run. Reversed and signed-permuted biases lose the benefit; the other domain’s norm-matched bias transfers only a small fraction.

In joint bias-plus-fusion fits, the matrix alone retains essentially all CE benefit, while the explicit bias alone retains 3.9%/5.0%. These parts can compensate during fitting, so their contributions are not uniquely identifiable or additive. Together with the live results, the evidence supports *a large steering component plus useful context-dependent correction*, rather than equivalence to a bias or discovery of a semantic skill vector. Complete 21-condition-per-domain controls, intervals and geometry appear in Appendix S.17.

R WHAT LIMITS THE WEAK CODING GAINS?

The initial NanoCoder result is small despite lower prediction loss. We test whether it reflects insufficient examples, rollout length, supervision coverage, or the objective. These are targeted interventions on a fixed fusion architecture, not an upper bound on coding adaptation.

More data helps modestly; a larger cap is not more useful content. Three independent continuations start from the same 4,096-example checkpoint and reset the optimizer: three more epochs on the original data; the same prompts with a 16,384 response cap; or 18,016 nested prompts with that cap. Native-relative speeds are 1.0288 $\times$, 1.0236 $\times$, and 1.0431 $\times$, versus 1.0156 $\times$ for the initial checkpoint in that matched timing trial. The long-cap condition does not beat the extra-optimization control. Only 21/4,096 original responses change when the cap rises; the others had already stopped. The expanded set contains 5.26 million actual response tokens, median length 195, not 18,016 full 16K responses. Capped outputs contribute 22.42% of tokens and include repetition. Expanded data also increases optimizer updates, so the gain is not a compute-matched scaling law.

Table 31: Matched NanoCoder objective experiment. Ordinary CE and prefix length use identical offline anchors; live τ and speed use actual decoding. The reversal between ordinary CE and speed is observed, not inferred from different timing trials.

Objective	Ordinary CE	Prefix length	Live τ	$\times$ native	Fit min
Native	2.2607	4.7256	5.0415	1.0000	—
Uniform	1.8514	5.5020	5.2810	1.0464	44.7*
Decay	1.8555	5.4893	5.2874	1.0469	45.2
AUF	2.1542	5.7578	5.4251	1.0718	46.5

*Reused fit, freshly timed inference. All continuation fits also inherit the initial 3.29-minute fit. Prefix length excludes the verifier token; live τ includes it.

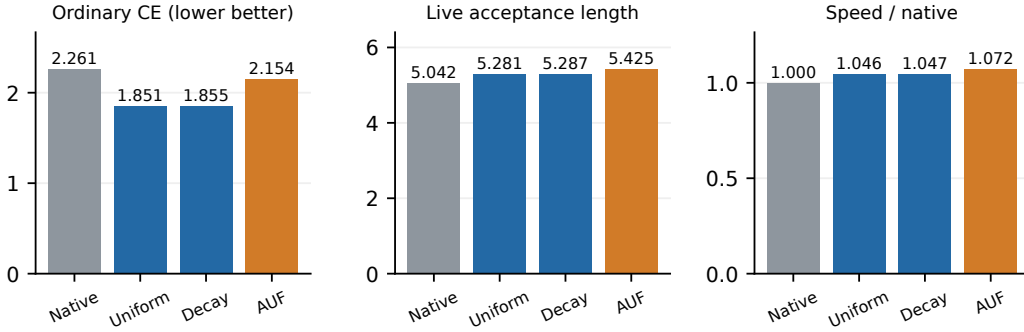


Figure 5: NanoCoder’s matched objective comparison: lower ordinary fixed-prefix CE does not order live acceptance or speed. Earlier expanded-data and dense-coverage comparisons remain separate experiments in the complete tables; their speed ratios should not be pooled into this timing trial.

Dense coverage is not the missing ingredient in this recipe. The sampled fit uses at most eight anchors per example. A separate continuation tiles adjacent 15-label blocks, supervising every response token after its first clean anchor exactly once per epoch: 15,727,668 label occurrences over three epochs. It uses the same initialization, data and 6,756 updates as the sampled 18K fit. Dense supervision takes 44.7 rather than 15.6 fitting minutes, yet yields only $1.0011 \times$ sampled throughput [0.9952, 1.0074]. Live τ changes from 5.2693 to 5.2810. Fixed-prefix CE and agreement improve modestly, but no material live gain is resolved. Tiling changes anchor placement as well as coverage; this does not rule out other anchor distributions.

The objective exposes a loss–acceptance tradeoff. On the same dense data, we compare uniform CE, DFlash’s exponential position decay, and the published accept-until-fail (AUF) rule (Yang & Li, 2026). For slot loss ℓ_{bj} and eligibility m_{bj} ,

$$L = \frac{\sum_{b,j} m_{bj} w_{bj} \ell_{bj}}{\sum_{b,j} m_{bj} w_{bj}}, \quad w_{bj}^{\text{decay}} = e^{-(j-1)/7}, \quad (27)$$

$$w_{bj}^{\text{AUF}} = \text{stopgrad} \prod_{i < j} (1 - m_{bi} + m_{bi} \mathbf{1}[\hat{y}_{bi} = y_{bi}]). \quad (28)$$

AUF includes the first failing prediction and excludes later slots. These are established objectives applied to frozen-interface fitting; AUF is our implementation of the published formula, not author code or a replication of its full training setup.

AUF improves speed by 2.42% over uniform [1.0156, 1.0333], while decay is unresolved at $1.0005 \times$ [0.9958, 1.0050]. AUF’s ordinary CE is *worse*, but it extends more accepted prefixes and improves live acceptance (Table 31). This directly tests the insufficiency of ordinary CE as a model-selection proxy under this recipe. It is not a new acceptance-aware loss.

Even the dense AUF fit remains below $1.10 \times$ native standalone speed. The final matched-bank deployment deliberately includes the weaker NanoCoder domain to measure a practical shared-serving

2214 workload (the historical shared-serving study); it does not turn that domain into a greater-than-10%
2215 standalone result. The negative result is informative: more examples, longer caps and dense super-
2216 vision do not by themselves produce Math/KiCad-sized gains at this interface. Scaling also reduces
2217 top-two weight-energy concentration from 99.99% to 94.38% (90.69% after dense coverage), so
2218 universal rank-two sufficiency is unsupported. None of these findings implies that coding targets are
2219 intrinsically unsuitable for draft adaptation.

2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267

S COMPLETE HISTORICAL LORA RESULTS

These tables preserve earlier completed studies, including negative results. They are not additional cells of the main matched architecture sweep. Each native denominator belongs to its own target, cohort and timing trial. Repeated controls are not independent runs. Screening and heldout denote different test cohorts; neither is training data. Historical standalone vLLM throughput is total output tokens divided by summed warm request time, whereas the main Transformers tables use mean per-request throughput unless explicitly marked pooled. Concurrent throughput divides total output tokens by workload makespan. Fixed-prefix draft length excludes the verifier token; live acceptance includes it where indicated. Matrix energy, CE retention and latency retention are distinct quantities. A missing metric is not inferred from another metric. Parent fitting costs remain inherited for interventions without retraining.

S.1 GSM8K LORA: TOKEN-TRAINED INTERFACE VERSUS DRAFTER LORA

Historical study date: 2026-09-07. The token-trained mapper improves mean acceptance length from **4.3605 to 5.6448** and gives **1.2733 $\times$ speedup over the unchanged drafter**. Direct drafter LoRA reaches 5.6808 and 1.2847 $\times$. All four speculative pipelines match the saved adapted-target autoregressive token IDs and finish reasons on **128/128 prompts**, with **17,128 tokens each**.

Table 32: GSM8K LoRA: token-trained interface versus drafter LoRA: Data and training.

Variant	Trainable parameters	Initialization	Export
Five context maps, each 2,560 $\rightarrow$ 2,560	32,768,000	Identity; original fusion and backbone frozen	Fold each map into its original fusion weight block
Drafter LoRA on q/k/v/o/gate/up/down in all five layers	32,112,640	Rank 112, alpha 112, zero initial update	Merge into the original BF16 draft checkpoint schema

Table 33: GSM8K LoRA: token-trained interface versus drafter LoRA: Complete results and fitting records.

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
GSM8K/test128	AR	17,128	543.265	31.53	—	0.2658 $\times$	1.0000 $\times$
GSM8K/test128	Original	17,128	144.389	118.62	4.3605	1.0000 $\times$	3.7625 $\times$
GSM8K/test128	MSE mapper	17,128	144.949	118.17	4.3442	0.9961 $\times$	3.7480 $\times$
GSM8K/test128	CE mapper	17,128	113.401	151.04	5.6448	1.2733 $\times$	4.7907 $\times$
GSM8K/test128	Body LoRA r112	17,128	112.395	152.39	5.6808	1.2847 $\times$	4.8335 $\times$

Table 34: GSM8K LoRA: token-trained interface versus drafter LoRA: Complete results and fitting records.

Fit	Parameters	Steps	GPUs	Fit s	Fit GPU-h	Epoch-average loss
full/lora	32,112,640	1536	2	224.801	0.1249	1.857326 $\rightarrow$ 1.617403 $\rightarrow$ 1.554068
full/mapper	32,768,000	1536	2	200.402	0.1113	1.894004 $\rightarrow$ 1.641520 $\rightarrow$ 1.455785

S.2 GSM8K LORA: FEATURE-RECONSTRUCTION MAPPER

Historical study date: 2026-09-07. A feature-MSE context mapper gives **0.9961 $\times$ native-drafter speed**, with acceptance length **4.3442 versus 4.3605**. It provides no measured benefit for this target adapter. Equality is an inference-equivalence result, not a math-accuracy score.

Table 35: GSM8K LoRA: feature-reconstruction mapper: Evaluation and final results.

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
GSM8K/test128	AR	17,128	543.265	31.53	—	0.2658 $\times$	1.0000 $\times$
GSM8K/test128	Original	17,128	144.389	118.62	4.3605	1.0000 $\times$	3.7625 $\times$
GSM8K/test128	MSE mapper	17,128	144.949	118.17	4.3442	0.9961 $\times$	3.7480 $\times$

Table 36: GSM8K LoRA: feature-reconstruction mapper: Evaluation and final results.

Fit	Parameters	Steps	GPUs	Fit s	Fit GPU-h	Epoch-average loss
full	32,768,000	348	1	32.775	0.0091	0.008805 $\rightarrow$ 0.003463 $\rightarrow$ 0.000735

S.3 LORA DRAFTER RANK AND COMPRESSION STUDY — 2026-09-07

Historical study date: 2026-09-07. A two-direction fusion update preserves essentially all of the trained rank 56 fusion update’s latency reduction on a fresh 128-prompt GSM8K set. Removing those directions removes the measured gain. The same trained fusion also improves the base target without LoRA, so these results support workload adaptation and do not isolate a LoRA-specific repair mechanism.

Table 37: LoRA drafter rank and compression study — 2026-09-07: Models and data provenance.

Component	Repository	Pinned revision
Base target	Qwen/Qwen3-4B	1cfa9a7208912126459214e8b04321603b3df60c
Target LoRA	witcher/qwen3-4b-gsm8k-grpo	f52e1ed1b3accd404d37a9621db0536dd99bdfc5
Initial draft	z-lab/Qwen3-4B-DF1ash-b16	b74e3a329c4d963783143b1e970d95b002be72bd
Training implementation	SpecForge	953d43a0c1c0f5e32989dc43f91ce5fc2d9ddfef
Dataset	openai/gsm8k	740312add88f781978c0658806c59bc2815b9866

Table 38: LoRA drafter rank and compression study — 2026-09-07: Optimization, precision and checkpoint selection.

Setting	Actual value
Examples / epochs	4,096 / 3
GPUs per fit	2, DDP; at most 4 assistant GPUs in concurrent jobs
Per-GPU microbatch / accumulation	2 examples / 2 microbatches
Effective batch	8 examples
Updates per epoch / total	512 / 1,536
Optimizer	Fused AdamW; betas (0.9,0.999), epsilon 1e-8
Weight decay / gradient norm cap	0 / 1
Peak learning rate	1e-4
Frozen / trainable parameters	BF16 / FP32
Forward	BF16 CUDA autocast, upstream PyTorch SDPA
Checkpoint	Final after three epochs, no dev selection

Table 39: LoRA drafter rank and compression study — 2026-09-07: Complete rank, compression and seed results.

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
screening/adapted	fusion_full	17,128	114.254	149.91	5.6094	1.2638×	4.7549×
screening/adapted	draft_r12	17,128	114.539	149.54	5.5788	1.2606×	4.7430×
heldout/adapted	full_mapper	19,070	123.430	154.50	5.7172	1.2599×	4.9045×
screening/adapted	draft_r6	17,128	114.919	149.04	5.5421	1.2564×	4.7274×
screening/adapted	fusion_svd56	17,128	115.198	148.68	5.5653	1.2534×	4.7159×
screening/adapted	fusion_r224	17,128	116.965	146.44	5.4553	1.2345×	4.6447×
screening/adapted	draft_r3	17,128	117.230	146.11	5.4622	1.2317×	4.6342×
screening/adapted	fusion_r112	17,128	118.052	145.09	5.4143	1.2231×	4.6019×
screening/adapted	fusion_r56	17,128	118.511	144.53	5.3909	1.2184×	4.5841×
screening/adapted	fusion_svd2_from_r56	17,128	119.034	143.89	5.3760	1.2130×	4.5640×
screening/adapted	fusion_svd4_from_r56	17,128	119.099	143.81	5.3751	1.2123×	4.5615×
heldout/adapted / seed 43	draft_r3	19,070	128.402	148.52	5.4954	1.2111×	4.7146×
heldout/adapted	draft_svd1_from_r3	19,070	128.861	147.99	5.4561	1.2068×	4.6978×
heldout/adapted / seed 44	draft_r3	19,070	129.191	147.61	5.4730	1.2037×	4.6858×
heldout/adapted	draft_r3	19,070	129.247	147.55	5.4827	1.2032×	4.6838×
heldout/adapted / seed 44	fusion_r56	19,070	129.372	147.40	5.4455	1.2020×	4.6792×
heldout/adapted	fusion_r56	19,070	129.393	147.38	5.4650	1.2019×	4.6785×
heldout/adapted	fusion_svd2_from_r56	19,070	129.542	147.21	5.4601	1.2005×	4.6731×
heldout/adapted / seed 44	fusion_svd2_from_r56	19,070	129.870	146.84	5.4551	1.1974×	4.6613×
heldout/adapted / seed 43	fusion_r56	19,070	130.045	146.64	5.4384	1.1958×	4.6550×

2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
heldout/adapted / seed 43	fusion_svd2_ from_r56	19,070	130.097	146.58	5.4317	1.1953×	4.6532×
heldout/adapted	fusion_svd1_ from_r56	19,070	133.520	142.83	5.2813	1.1647×	4.5339×
screening/adapted	fusion_svd56_ from_mapper	17,128	135.541	126.37	4.6253	1.0653×	4.0081×
screening/adapted	Original	17,128	144.389	118.62	4.3605	1.0000×	3.7625×
heldout/adapted	original	19,070	155.512	122.63	4.4771	1.0000×	3.8927×
heldout/adapted	fusion_tail_after2	19,070	155.828	122.38	4.4918	0.9980×	3.8848×
screening/adapted	AR	17,128	543.265	31.53	—	0.2658×	1.0000×
heldout/adapted	AR	19,070	605.364	31.50	—	0.2569×	1.0000×
heldout/base	draft_r3	19,918	117.218	169.92	5.5985	1.2024×	4.5747×
heldout/base	fusion_r56	19,918	118.176	168.55	5.5537	1.1926×	4.5376×
heldout/base	original	19,918	140.938	141.32	4.6225	1.0000×	3.8048×
heldout/base	AR	19,918	536.241	37.14	—	0.2628×	1.0000×

Table 40: LoRA drafter rank and compression study — 2026-09-07: Complete rank, compression and seed results.

Fit	Parameters	Steps	GPUs	Fit s	Fit GPU-h	Epoch-average loss
full/draft_r12	3,440,640	1536	2	218.514	0.1214	2.000676 → 1.689013 → 1.646198
full/draft_r3	860,160	1536	2	223.984	0.1244	2.139692 → 1.758259 → 1.718130
full/draft_r6	1,720,320	1536	2	219.958	0.1222	2.062833 → 1.719659 → 1.680087
full/fusion_full	32,768,000	1536	2	188.124	0.1045	1.868230 → 1.647486 → 1.594870
full/fusion_r112	1,720,320	1536	2	175.121	0.0973	2.000636 → 1.767849 → 1.742048
full/fusion_r224	3,440,640	1536	2	172.974	0.0961	1.959222 → 1.754421 → 1.725869
full/fusion_r56	860,160	1536	2	179.799	0.0999	2.049910 → 1.785690 → 1.757877
seed 43/draft_r3	860,160	1536	2	225.234	0.1251	2.149043 → 1.751851 → 1.730815
seed 43/fusion_r56	860,160	1536	2	181.822	0.1010	2.062332 → 1.776181 → 1.767528
seed 44/draft_r3	860,160	1536	2	222.955	0.1239	2.138006 → 1.744472 → 1.722201
seed 44/fusion_r56	860,160	1536	2	180.033	0.1000	2.064461 → 1.777524 → 1.765326

Table 41: LoRA drafter rank and compression study — 2026-09-07: Compression and functional interpretation.

Compression source	Retained rank	Squared update energy
Direct full fusion, BF16 weight difference	56	63.234%
Fusion-r56, raw FP32 factors	2	99.926%
Fusion-r56, raw FP32 factors	4	99.995%
Five full maps, folded BF16 weight difference	56	88.706%

S.4 CONSTANT AND CONTEXT-DEPENDENT COMPONENTS OF MATH FUSION ADAPTATION

Historical study date: 2026-09-08. All four conditions match the fixed math-target AR reference on **128/128 prompts**, token IDs and finish reasons (512 comparisons). This is a post-training intervention on one learned fusion adapter, not a new adapter-training comparison.

Table 42: Constant and context-dependent components of math fusion adaptation: Results.

Correction	Seconds	Tok/s	Tau	Accepted/proposed	vs original (95% CI)
original	155.899	122.32	4.4771	14854/64080	$1.0000 \times [1.0000, 1.0000]$
constant	138.189	138.00	5.0816	15355/56430	$1.1282 \times [1.1054, 1.1516]$
centered	139.164	137.03	5.0438	15326/56850	$1.1203 \times [1.1031, 1.1387]$
full	128.352	148.58	5.4650	15583/52350	$1.2146 \times [1.1880, 1.2428]$

Table 43: Constant and context-dependent components of math fusion adaptation: Results.

Correction	$P(L \geq 1)$	$P(L \geq 2)$	$P(L \geq 4)$	$P(L \geq 8)$	$P(L \geq 15)$
original	0.7633	0.5934	0.3418	0.1533	0.0363
constant	0.8328	0.6776	0.4208	0.1823	0.0385
centered	0.8211	0.6612	0.4124	0.1855	0.0430
full	0.8309	0.6788	0.4619	0.2246	0.0461

S.5 FUSION UPDATES VERSUS FIVE FACTORIZED REPLACEMENT MAPS

Historical study date: 2026-09-08. Every target call uses the selected math or KiCad LoRA on the same pinned Qwen3-4B. The drafter alone changes.

Table 44: Fusion updates versus five factorized replacement maps: math.

Configuration	Tok/s	Tau	Acceptance rate	vs original	95% CI	vs AR	Fit seconds
native (shared)	122.07	4.4771	0.2318	1.0000×	[1.0000,1.0000]	3.9111×	inherited
fusion_r56 (shared)	147.42	5.4650	0.2977	1.2077×	[1.1814,1.2355]	4.7233×	inherited
fusion_svd2 (shared)	147.52	5.4601	0.2973	1.2085×	[1.1826,1.2359]	4.7266×	inherited
draft_r3 (shared)	148.99	5.4827	0.2988	1.2206×	[1.1949,1.2490]	4.7737×	inherited
maps_r2	39.18	1.3793	0.0253	0.3210×	[0.3088,0.3343]	1.2553×	184.3
maps_r30	74.61	2.6765	0.1118	0.6112×	[0.5801,0.6446]	2.3905×	176.2
fusion_r50	147.90	5.4603	0.2974	1.2116×	[1.1850,1.2401]	4.7386×	172.9
maps_r56	93.90	3.3791	0.1586	0.7692×	[0.7316,0.8099]	3.0084×	172.8
maps_r112	116.30	4.2294	0.2153	0.9527×	[0.9105,0.9980]	3.7262×	174.1
maps_full	154.77	5.7281	0.3152	1.2679×	[1.2317,1.3072]	4.9588×	190.0

Table 45: Fusion updates versus five factorized replacement maps: kicad.

Configuration	Tok/s	Tau	Acceptance rate	vs original	95% CI	vs AR	Fit seconds
native (shared)	159.57	5.9526	0.3302	1.0000×	[1.0000,1.0000]	5.5663×	inherited
fusion_r56 (shared)	202.01	7.5048	0.4337	1.2660×	[1.2545,1.2769]	7.0470×	inherited
fusion_svd2 (shared)	197.00	7.3211	0.4214	1.2346×	[1.2251,1.2436]	6.8720×	inherited
draft_r3	225.57	8.4115	0.4941	1.4136×	[1.4003,1.4265]	7.8687×	612.8

Table 46: Fusion updates versus five factorized replacement maps: Training trajectories.

Fit	Parameters	Updates	Seconds	Epoch mean CE
kicad_s42/draft_r3	860,160	1536	612.8	1.095217 → 0.706113 → 0.650230
math_s42/fusion_r50	768,000	1536	172.9	2.054942 → 1.785131 → 1.758492
math_s42/maps_full	32,768,000	1536	190.0	1.893610 → 1.641244 → 1.455419
math_s42/maps_r112	2,867,200	1536	174.1	2.600475 → 2.131617 → 1.987675
math_s42/maps_r2	51,200	1536	184.3	5.098604 → 4.701770 → 4.631856
math_s42/maps_r30	768,000	1536	176.2	3.757738 → 3.026264 → 2.845455
math_s42/maps_r56	1,433,600	1536	172.8	3.174238 → 2.525214 → 2.365354

S.6 KICAD-LoRA TARGET: ORIGINAL VERSUS ADAPTED DFLASH DRAFTER

Historical study date: 2026-09-08. The deployment target is **Qwen3-4B with the KiCad LoRA active in every pipeline**. Only the drafter changes. No unadapted-target comparison is part of this experiment. All three speculative pipelines match the same LoRA-target AR reference on 128/128 prompts, token IDs and finish reason (384 speculative comparisons).

Table 47: KiCad-LoRA target: original versus adapted DFlash drafter: Results.

Drafter	Seconds	Tok/s	Acceptance length	vs original	vs AR	Training loop
ar	22344.800	28.67	—	$0.1797\times$	$1.0000\times$	—
native	4014.312	159.57	5.9526	$1.0000\times$	$5.5663\times$	—
fusion_r56	3170.844	202.01	7.5048	$1.2660\times$	$7.0470\times$	636.9s, 2 GPUs
fusion_svd2	3251.552	197.00	7.3211	$1.2346\times$	$6.8720\times$	inherits rank56 fit

Table 48: KiCad-LoRA target: original versus adapted DFlash drafter: Results.

Drafter	95% CI vs original	Accepted / proposed	Iterations	Cap terminations	Max observed allocated GiB
ar	[0.1747, 0.1846]	—/—	—	18	34.80
native	[1.0000, 1.0000]	533063/1614480	107632	18	34.57
fusion_r56	[1.2545, 1.2769]	555343/1280610	85374	18	34.57
fusion_svd2	[1.2251, 1.2436]	553227/1312800	87520	18	34.57

Table 49: KiCad-LoRA target: original versus adapted DFlash drafter: Results.

Drafter	$P(L \geq 1)$	$P(L \geq 2)$	$P(L \geq 4)$	$P(L \geq 8)$	$P(L \geq 15)$
native	0.7277	0.6490	0.5017	0.2834	0.0626
fusion_r56	0.6919	0.6167	0.5468	0.4078	0.2363
fusion_svd2	0.6976	0.6223	0.5481	0.3957	0.2051

Table 50: KiCad-LoRA target: original versus adapted DFlash drafter: Results.

Phase	Allocated GPU-hours	Phase wall span (min)	Longest job seconds	Slurm jobs
validation	0.4108	174.95	319	29426, 29427, 29424, 29434, 29436, 29439, 29440, 29504, 29505, 29506
generation	1.9925	30.37	1822	29443, 29444, 29445, 29446
feature capture	0.6333	9.73	584	29468, 29469, 29470, 29471
fit and cache validation	0.5017	15.05	903	29473
compression	0.0064	0.38	23	29477
eval ar	6.4336	102.70	6156	29479, 29482, 29480, 29481
eval native	1.2044	19.35	1161	29509, 29510, 29511, 29512
eval fusion_r56	0.9517	18.30	904	29514, 29515, 29516, 29517
eval fusion_svd2	0.9800	21.45	940	29521, 29522, 29523, 29524

S.7 MATCHED TRAINING-TARGET AND DIRECT-RANK CONTROLS

Historical study date: 2026-09-08. Training the math drafter update from the base target works about as well as training from its LoRA target. Direct rank 2 improves acceptance, but falls short of training rank 56 and compressing to rank 2 under this recipe. Eight fitted-model/target combinations match their target-specific AR token IDs and finish reasons on 128/128 prompts each (1,024 comparisons).

Table 51: Matched training-target and direct-rank controls: Evaluation and complete results.

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
heldout/base	AR	19,918	536.241	37.14	—	0.2628×	1.0000×
heldout/base	Original	19,918	140.938	141.32	4.6225	1.0000×	3.8048×
heldout/base	adapted_fusion_r56	19,918	118.176	168.55	5.5537	1.1926×	4.5376×
heldout/base	adapted_body_r3	19,918	117.218	169.92	5.5985	1.2024×	4.5747×
heldout/base	base_fusion_r56	19,918	117.771	169.13	5.5715	1.1967×	4.5533×
heldout/base	base_draft_r3	19,918	116.891	170.40	5.5854	1.2057×	4.5875×
heldout/base	base_fusion_r2	19,918	125.142	159.16	5.2421	1.1262×	4.2851×
heldout/base	adapted_fusion_r2	19,918	124.693	159.74	5.2421	1.1303×	4.3005×
heldout/adapted	AR	19,070	605.364	31.50	—	0.2569×	1.0000×
heldout/adapted	Original	19,070	155.512	122.63	4.4771	1.0000×	3.8927×
heldout/adapted	adapted_fusion_r56	19,070	129.393	147.38	5.4650	1.2019×	4.6785×
heldout/adapted	adapted_body_r3	19,070	129.247	147.55	5.4827	1.2032×	4.6838×
heldout/adapted	base_fusion_r56	19,070	129.271	147.52	5.4569	1.2030×	4.6829×
heldout/adapted	base_draft_r3	19,070	128.400	148.52	5.4724	1.2112×	4.7147×
heldout/adapted	base_fusion_r2	19,070	136.833	139.37	5.1567	1.1365×	4.4241×
heldout/adapted	adapted_fusion_r2	19,070	136.842	139.36	5.1453	1.1364×	4.4238×

Table 52: Matched training-target and direct-rank controls: Evaluation and complete results.

Fit	Parameters	Steps	GPUs	Fit s	Fit GPU-h	Epoch-average loss
adapted/fusion_r2	30,720	1536	2	173.820	0.0966	2.454466 → 1.948402 → 1.916502
base/draft_r3	860,160	1536	2	225.810	0.1254	2.117895 → 1.755461 → 1.725585
base/fusion_r2	30,720	1536	2	177.681	0.0987	2.425743 → 1.943397 → 1.922145
base/fusion_r56	860,160	1536	2	178.787	0.0993	2.033630 → 1.781838 → 1.764915

S.8 DFLASH INTERFACE ADAPTATION ACROSS MATH AND SUBTITLE WORKLOADS

Historical study date: 2026-09-08. All 20 speculative configurations match AR on 128/128 prompts (2,560 exact token-ID and finish-reason comparisons). Math adaptation improves decoding; subtitle adaptation yields no practically substantial gain under this recipe.

Table 53: DFlash interface adaptation across math and subtitle workloads: Experimental design.

Evaluation workload	Target	Draft variants
GSM8K	Math target LoRA	Original; math/subtitle fusion-r56, drafter-wide-r3, compressed fusion-r2
English-Spanish batches	Subtitle target LoRA	Same seven variants
GSM8K	Base target without LoRA	Original; math fusion-r56; subtitle fusion-r56
English-Spanish batches	Base target without LoRA	Same three variants

Table 54: DFlash interface adaptation across math and subtitle workloads: Complete results and training records.

Cohort / target	Draft	Tokens	Seconds	Tok/s	Tau	vs native	vs AR
math/math	AR	19,070	610.995	31.21	—	0.2557×	1.0000×
math/math	original	19,070	156.222	122.07	4.4771	1.0000×	3.9111×
math/math	math_fusion	19,070	129.357	147.42	5.4650	1.2077×	4.7233×
math/math	math_body	19,070	127.991	148.99	5.4827	1.2206×	4.7737×
math/math	math_rank2	19,070	129.267	147.52	5.4601	1.2085×	4.7266×
math/math	subtitle_fusion	19,070	151.635	125.76	4.6076	1.0302×	4.0294×
math/math	subtitle_body	19,070	154.769	123.22	4.5002	1.0094×	3.9478×
math/math	subtitle_rank2	19,070	151.074	126.23	4.6043	1.0341×	4.0443×
subtitle/subtitle	AR	12,310	397.316	30.98	—	0.5443×	1.0000×
subtitle/subtitle	original	12,310	216.273	56.92	2.0573	1.0000×	1.8371×
subtitle/subtitle	math_fusion	12,310	218.622	56.31	2.0281	0.9893×	1.8174×
subtitle/subtitle	math_body	12,310	216.268	56.92	2.0541	1.0000×	1.8371×
subtitle/subtitle	math_rank2	12,310	217.931	56.49	2.0291	0.9924×	1.8231×
subtitle/subtitle	subtitle_fusion	12,310	214.472	57.40	2.0652	1.0084×	1.8525×
subtitle/subtitle	subtitle_body	12,310	215.045	57.24	2.0511	1.0057×	1.8476×
subtitle/subtitle	subtitle_rank2	12,310	214.892	57.28	2.0616	1.0064×	1.8489×
math/base	AR	19,918	542.645	36.71	—	0.2593×	1.0000×
math/base	original	19,918	140.708	141.56	4.6225	1.0000×	3.8565×
math/base	math_fusion	19,918	118.148	168.59	5.5537	1.1909×	4.5929×
math/base	subtitle_fusion	19,918	137.255	145.12	4.7367	1.0252×	3.9536×
subtitle/base	AR	11,517	314.039	36.67	—	0.6237×	1.0000×
subtitle/base	original	11,517	195.870	58.80	1.9063	1.0000×	1.6033×
subtitle/base	math_fusion	11,517	194.613	59.18	1.9199	1.0065×	1.6137×
subtitle/base	subtitle_fusion	11,517	195.991	58.76	1.9123	0.9994×	1.6023×

Table 55: DFlash interface adaptation across math and subtitle workloads: Complete results and training records.

Fit	Parameters	Steps	GPUs	Fit s	Fit GPU-h	Epoch-average loss
full/draft_r3	860,160	576	2	85.266	0.0474	5.188511 → 4.920046 → 4.854699
full/fusion_r56	860,160	576	2	69.681	0.0387	5.097506 → 4.939624 → 4.899751

Table 56: DFlash interface adaptation across math and subtitle workloads: Acceptance positions and task diagnostic.

Workload/target	Draft	P(L≥1)	P(L≥2)	P(L≥4)	P(L≥8)
Math/math LoRA	Original	0.7633	0.5934	0.3418	0.1533
Math/math LoRA	Math fusion	0.8309	0.6788	0.4619	0.2246
Math/math LoRA	Math rank2	0.8302	0.6779	0.4632	0.2247
Subtitles/subtitle LoRA	Original	0.4614	0.2108	0.0492	0.0249
Subtitles/subtitle LoRA	Subtitle fusion	0.4667	0.2100	0.0503	0.0246

2808	Workload/target	Draft	$P(L \geq 1)$	$P(L \geq 2)$	$P(L \geq 4)$	$P(L \geq 8)$
2809						
2810	Subtitles/subtitle LoRA	Subtitle rank2	0.4659	0.2088	0.0504	0.0247
2811						
2812						
2813						
2814						
2815						
2816						
2817						
2818						
2819						
2820						
2821						
2822						
2823						
2824						
2825						
2826						
2827						
2828						
2829						
2830						
2831						
2832						
2833						
2834						
2835						
2836						
2837						
2838						
2839						
2840						
2841						
2842						
2843						
2844						
2845						
2846						
2847						
2848						
2849						
2850						
2851						
2852						
2853						
2854						
2855						
2856						
2857						
2858						
2859						
2860						
2861						

S.9 FUSION STEERING ON MATH AND KICAD

Historical study date: 2026-09-09. Paired fixed-prefix interventions use independently fitted Math/KiCad fusion-r56 experts. Reused uniform-CE training takes 179.8/636.9 seconds on two GPUs, respectively. These are offline prediction and geometry results on 32 prompts per domain, not live speed measurements. Methods define training-only mean calibration, target-shift projection and the two-domain cross-expert control.

Table 57: Fusion steering on Math and KiCad: Direction and low-rank geometry.

Domain	Top1 / top2 weight energy	Response correction energy in mean component	Mean $\cos(c, t)$	Correction energy parallel to t	CE gain retained by $c_{\perp}$
Math	83.28% /99.93%	92.60%	-0.0294	0.23%	100.3%
KiCad	95.56% /98.46%	90.34%	-0.0513	0.49%	96.1%

Table 58: Fusion steering on Math and KiCad: Causal fixed-prefix results.

Domain	Native	Own correction	Training mean μ	Centered $c - \mu$	Parallel only	Perpendicular only	Undo target shift	Reversed correction
Math	3.0737	1.8724	1.9914	2.7516	3.0712	1.8692	3.0829	5.9635
KiCad	2.1279	1.3407	1.4449	1.6503	1.9945	1.3717	1.7293	6.4284

Table 59: Fusion steering on Math and KiCad: Causal fixed-prefix results.

Domain	Top1 native / own / mean	All15 fixed-prefix agreement native / own / mean
Math	47.16% /53.85% /51.89%	6.98% /9.77% /7.91%
KiCad	61.47% /71.76% /69.31%	13.39% /43.75% /32.14%

Table 60: Fusion steering on Math and KiCad: Causal fixed-prefix results.

Target	Math expert	KiCad expert
Math	1.8724	2.9346
KiCad	2.0168	1.3407

S.10 TWO-TARGET FUSION-EXPERT ROUTING

Historical study date: 2026-09-09. The independently trained Math/KiCad pair compares direct target-ID selection, a frozen-expert router, and jointly fitted experts plus router. It uses uniform block CE, 4,096 examples per domain, three epochs and 3,072 updates. This is serial alternating-target evaluation, not concurrent serving. Router-only fitting takes 801.6 seconds and inherits 816.7 seconds of two-GPU expert fitting; scratch takes 802.0 seconds without those expert fits.

Table 61: Two-target fusion-expert routing: Exact loss and fitting recipe.

Set	Fit	Trainable params	Steps	Fit seconds	Fit GPU-h	Epoch mean CE
pair	router	25,604	3072	801.6	0.4453	1.32490 $\rightarrow$ 1.33357 $\rightarrow$ 1.32453
pair	scratch	1,745,924	3072	802.0	0.4455	1.53988 $\rightarrow$ 1.33379 $\rightarrow$ 1.29622

Table 62: Two-target fusion-expert routing: Measurement contract and results.

Set	Policy	Domain/traffic	Tok/s	Tau	vs Native	vs Select	vs inherited AR
pair	native	traffic	157.14	5.8963	1.0000 $\times$	0.7919 $\times$	5.4687 $\times$
pair	native	math	119.85	4.4771	1.0000 $\times$	0.8310 $\times$	3.8400 $\times$
pair	native	kicad	158.61	5.9526	1.0000 $\times$	0.7904 $\times$	5.5328 $\times$
pair	select	traffic	198.42	7.4247	1.2627 $\times$	1.0000 $\times$	6.9053 $\times$
pair	select	math	144.23	5.4650	1.2034 $\times$	1.0000 $\times$	4.6211 $\times$
pair	select	kicad	200.66	7.5048	1.2652 $\times$	1.0000 $\times$	7.0000 $\times$
pair	router	traffic	194.56	7.4190	1.2382 $\times$	0.9806 $\times$	6.7712 $\times$
pair	router	math	141.55	5.4517	1.1811 $\times$	0.9814 $\times$	4.5353 $\times$
pair	router	kicad	196.76	7.4996	1.2405 $\times$	0.9805 $\times$	6.8637 $\times$
pair	scratch	traffic	200.16	7.6272	1.2738 $\times$	1.0088 $\times$	6.9660 $\times$
pair	scratch	math	142.35	5.4849	1.1878 $\times$	0.9870 $\times$	4.5610 $\times$
pair	scratch	kicad	202.61	7.7170	1.2774 $\times$	1.0097 $\times$	7.0679 $\times$

Table 63: Two-target fusion-expert routing: Measurement contract and results.

Set/policy	Traffic95% CI vs Native	vs Select	Routing mean/p95 ms	Routing time fraction
pair/native	[1.0000, 1.0000]	[0.7854, 0.7989]	0.750/1.372	0.0046%
pair/select	[1.2518, 1.2733]	[1.0000, 1.0000]	2.090/2.662	0.0161%
pair/router	[1.2275, 1.2485]	[0.9796, 0.9815]	1.023/1.615	0.0077%
pair/scratch	[1.2622, 1.2849]	[1.0068, 1.0108]	1.004/1.625	0.0078%

Table 64: Two-target fusion-expert routing: Routing and execution cost.

Set/fit	Context domain	Mean expert probabilities (math, KiCad)	Mean entropy
pair/router	math	0.9978, 0.0022	0.0139
pair/router	kicad	0.0300, 0.9700	0.0654
pair/scratch	math	0.9990, 0.0010	0.0064
pair/scratch	kicad	0.0569, 0.9431	0.0415

2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023

S.11 NANOCODER: FUSION-ONLY ADAPTATION

Historical study date: 2026-09-09. The deployed target in every condition is **Qwen3-4B + user-namebetter/nanocoder-v1**. This study fits one fusion-LoRA checkpoint and compares it with the unchanged native DFlash under identical target, prompts and decoding. No drafter-body update, architecture search, learned router or MoE fit is included.

Table 65: NanoCoder: fusion-only adaptation: Benchmark and results.

Pipeline	Sum request seconds	Tokens/s	vs AR	vs native	Mean length τ	Proposal acceptance
Adapted target AR	909.63	31.15	1.0000×	—	—	—
Adapted target + native DFlash	203.05	139.53	4.4798×	1.0000×	5.0415	26.94%
Adapted target + fusion-r56 DFlash	199.18	142.25	4.5669×	1.0194×	5.1320	27.55%

S.12 NANOCODER: DENSE TOKEN SUPERVISION FOR FUSION LoRA

Historical study date: 2026-09-09. Dense coverage gives **1.0011** $\times$ **the sampled fit** (paired-prompt 95% interval **0.9952–1.0074** $\times$): no convincing live speed improvement in this trial, despite modestly better fixed-prefix predictions.

Table 66: NanoCoder: dense token supervision for fusion LoRA: Matched vLLM benchmark.

Condition	Tok/s	vs native	vs sampled	vs AR (inherited)	Mean τ	Acceptance fraction	Fit minutes
Native	139.90	1.0000 $\times$	0.9604 $\times$	4.4917 $\times$	5.0415	26.94%	—
Sampled 18K	145.67	1.0413 $\times$	1.0000 $\times$	4.6770 $\times$	5.2693	28.46%	15.6
Dense 18K	145.84	1.0424 $\times$	1.0011 $\times$	4.6822 $\times$	5.2810	28.54%	44.7

Table 67: NanoCoder: dense token supervision for fusion LoRA: Fixed-prefix diagnostic.

Model	CE	Top1	All15 agreement	Fixed-prefix accepted draft tokens	Mean correction energy fraction
Native	2.2607	50.39%	8.89%	4.7256	—
Sampled18K	1.8739	55.30%	11.93%	5.3965	74.40%
Dense18K	1.8514	55.75%	12.51%	5.5020	73.58%

S.13 NANOCODER: WHERE FUSION ADAPTATION HELPS AND FAILS

Historical study date: 2026-09-09. The completed fusion-r56 model improves prediction probabilities more than uninterrupted exact drafting. On all 128 held-out prompts, fixed-prefix CE falls 2.2607 $\rightarrow$ 1.9465, while top1 agreement rises 50.39% $\rightarrow$ 53.10%. Approximately 99.0% of the **net summed CE reduction** occurs on positions wrong in both conditions. This is an accounting decomposition, not a claim that 99% of predictions are wrong.

Table 68: NanoCoder: where fusion adaptation helps and fails: Results.

Metric	Native	Fusion-r56
Valid-token-weighted CE	2.2607	1.9465
Top1 agreement	50.39%	53.10%
All 15 exact agreement	8.89%	10.29%
Mean fixed-prefix accepted draft tokens	4.7256	4.9844
Live acceptance length including verifier token (inherited)	5.0415	5.1320
Live speedup vs native (inherited)	1.0000 $\times$	1.0194 $\times$

Table 69: NanoCoder: where fusion adaptation helps and fails: Results.

Prediction transition	Token positions	Summed CE reduction (native minus adapted)
Wrong $\rightarrow$ wrong	5,717	+4,159.66
Wrong $\rightarrow$ correct	916	+1,062.35
Correct $\rightarrow$ wrong	554	-586.39
Correct $\rightarrow$ correct	6,183	-435.63

Table 70: NanoCoder: where fusion adaptation helps and fails: Results.

Proposal position	Native $\rightarrow$ adapted top1	Native $\rightarrow$ adapted prefix survival
1	82.91% $\rightarrow$ 85.94%	82.91% $\rightarrow$ 85.94%
5	59.53% $\rightarrow$ 61.88%	43.83% $\rightarrow$ 46.75%
10	40.27% $\rightarrow$ 42.76%	22.85% $\rightarrow$ 23.64%
15	23.98% $\rightarrow$ 27.60%	8.89% $\rightarrow$ 10.29%

S.14 NANOCODER ACCEPTANCE-AWARE OBJECTIVE COMPARISON

Historical study date: 2026-09-09. The target is the actual BF16 Qwen3-4B with NanoCoder active in every evaluation. This study changes the fusion-adaptation objective while retaining the completed dense experiment’s data, initialization, supervision positions and optimizer budget.

Table 71: NanoCoder acceptance-aware objective comparison: Results.

Objective	tok/s	× native	× uniform	Mean accepted length τ	Accepted/ proposed	Fit seconds (2 GPUs)
native	139.64	1.0000	0.9557	5.0415	26.94%	—
uniform	146.12	1.0464	1.0000	5.2810	28.54%	2683.4 (reused)
decay7	146.19	1.0469	1.0005	5.2874	28.58%	2714.0
auf	149.66	1.0718	1.0242	5.4251	29.50%	2792.1

Table 72: NanoCoder acceptance-aware objective comparison: Results.

Objective	Timed seconds	× AR (inherited timing)	Native ratio 95% interval	Capped / 128	Fit GPU-hours
native	202.9	4.483	[1.0000, 1.0000]	0	—
uniform	193.9	4.691	[1.0345, 1.0601]	0	1.491 (reused)
decay7	193.8	4.694	[1.0357, 1.0594]	0	1.508
auf	189.3	4.805	[1.0586, 1.0868]	0	1.551

Table 73: NanoCoder acceptance-aware objective comparison: Results.

Fit	CE	Top1 %	Fixed-prefix draft length	Extended / shortened vs uniform
native	2.2607	50.39	4.7256	—
uniform	1.8514	55.75	5.5020	—
decay7	1.8555	55.50	5.4893	34 / 50
auf	2.1542	56.16	5.7578	130 / 81

S.15 NANOCODER: CONTINUED FUSION TRAINING WITH MORE EXAMPLES AND LONGER ROLLOUTS

Historical study date: 2026-09-09. Test whether NanoCoder’s modest initial gain improves through additional fusion-only training. Every target call uses the same BF16 Qwen3-4B plus NanoCoder LoRA. Preserve the original completed experiment and its checkpoint; run three predeclared continuations with identical final-checkpoint selection. No target fine-tuning, draft-body adaptation, MoE training or mixture deployment is included.

Table 74: NanoCoder: continued fusion training with more examples and longer rollouts: Three continued fits.

Fit	Training prompts	Response cap	Additional epochs	Additional updates
Control	4,096 original	4,096	3	1,536
Longer rollouts	4,096 original	16,384	3	1,536
Expanded data	18,016 nested	16,384	3	6,756

Table 75: NanoCoder: continued fusion training with more examples and longer rollouts: Evaluation.

Condition	Tok/s	vs native	vs initial fit	vs AR (inherited timing)	Mean τ	Acceptance fraction	Incremental fit seconds
Native	139.59	1.0000×	0.9847×	4.4815×	5.0415	26.94%	—
Initial fit	141.76	1.0156×	1.0000×	4.5514×	5.1320	27.55%	—
More optimization	143.60	1.0288×	1.0130×	4.6106×	5.1863	27.91%	199.8
Longer rollouts	142.88	1.0236×	1.0079×	4.5872×	5.1684	27.79%	220.8
Expanded data	145.61	1.0431×	1.0271×	4.6749×	5.2693	28.46%	935.6

Table 76: NanoCoder: continued fusion training with more examples and longer rollouts: Fixed-prefix comparison.

Checkpoint	CE	Top1	All15 agreement	Fixed-prefix draft length	Correction mean energy
original	1.9465	53.10%	10.29%	4.9844	90.97%
control	1.9252	53.74%	10.41%	5.0811	87.71%
long	1.9242	53.77%	10.53%	5.0723	87.35%
scale	1.8739	55.30%	11.93%	5.3965	74.40%

S.16 LEARNED CONTEXT STEERING VERSUS FUSION ADAPTATION

Historical study date: 2026-09-09. This study tests whether the benefit of fusion adaptation can be obtained with a single learned context vector, and whether adding that vector to fusion LoRA helps further. Math and KiCad always use their own active target LoRAs.

Table 77: Learned context steering versus fusion adaptation: Results.

Domain / variant	tok/s	$\times$ native	τ	Acceptance %	Fit seconds	Train GPU-hours
math/native	122.49	1.0000	4.4771	23.18	—	—
math/training_mean	138.17	1.1281	5.0789	27.19	193.0 (parent fit)	0.107 (reused)
math/bias	129.70	1.0589	4.7534	25.02	191.8	0.107
math/fusion_r56	147.84	1.2069	5.4482	29.65	193.0	0.107
math/bias_fusion	147.83	1.2069	5.4588	29.73	194.7	0.108
kicad/native	159.61	1.0000	5.9526	33.02	—	—
kicad/training_mean	189.62	1.1880	7.0554	40.37	599.8 (parent fit)	0.333 (reused)
kicad/bias	175.01	1.0965	6.5208	36.81	600.5	0.334
kicad/fusion_r56	202.08	1.2661	7.5045	43.36	599.8	0.333
kicad/bias_fusion	202.15	1.2665	7.5129	43.42	601.4	0.334

Table 78: Learned context steering versus fusion adaptation: Results.

Domain / variant	Timed seconds	$\times$ AR (inherited timing)	Native ratio 95% interval	Capped /128
math/native	155.7	3.924	[1.0000, 1.0000]	0
math/training_mean	138.0	4.427	[1.1050, 1.1516]	0
math/bias	147.0	4.155	[1.0482, 1.0703]	0
math/fusion_r56	129.0	4.737	[1.1802, 1.2361]	0
math/bias_fusion	129.0	4.736	[1.1811, 1.2351]	0
kicad/native	4013.2	5.568	[1.0000, 1.0000]	18
kicad/training_mean	3378.0	6.615	[1.1800, 1.1960]	18
kicad/bias	3660.2	6.105	[1.0911, 1.1015]	18
kicad/fusion_r56	3169.8	7.049	[1.2548, 1.2772]	18
kicad/bias_fusion	3168.7	7.052	[1.2554, 1.2777]	18

Table 79: Learned context steering versus fusion adaptation: Results.

Domain / alternative	$\times$ fusion	Paired 95% interval
math/training_mean	0.9346	[0.9261, 0.9426]
math/bias	0.8773	[0.8612, 0.8926]
math/bias_fusion	1.0000	[0.9971, 1.0028]
kicad/training_mean	0.9384	[0.9349, 0.9419]
kicad/bias	0.8660	[0.8611, 0.8711]
kicad/bias_fusion	1.0004	[0.9996, 1.0011]

Table 80: Learned context steering versus fusion adaptation: Results.

Domain / variant	Natural-stop prompts	$\times$ native, natural stops	Capped prompts	$\times$ native, capped
math/training_mean	128	1.1281	0	—
math/bias	128	1.0589	0	—
math/fusion_r56	128	1.2069	0	—
math/bias_fusion	128	1.2069	0	—
kicad/training_mean	110	1.1885	18	1.1862
kicad/bias	110	1.0978	18	1.0914
kicad/fusion_r56	110	1.2618	18	1.2824
kicad/bias_fusion	110	1.2618	18	1.2848

S.17 CAUSAL ANALYSIS OF LEARNED CONTEXT STEERING

Historical study date: 2026-09-09. This separate analysis uses the completed Math/KiCad steering checkpoints. It introduces no further gradient training or target generation; two rank-one reader controls are calibrated analytically on cached training features. All labels and cached hidden states come from the actual adapted targets.

Table 81: Causal analysis of learned context steering: Results.

Domain / intervention	CE	Top1 %	Fixed-prefix draft length	All15 %
math/random_rowspace_reader	1.9975	51.77	5.1484	7.91
math/constant_reader	1.9814	51.65	5.1719	7.91
math/reader_residual	2.7740	49.24	4.7930	7.44
math/svd1	1.9860	52.15	5.1445	8.37
math/native	3.0737	47.16	4.6875	6.98
math/fusion	1.8730	53.79	5.3125	9.77
math/fusion_mean	1.9904	51.89	5.1641	7.91
math/fusion_centered	2.7506	49.42	4.8359	7.44
math/bias	2.4139	50.04	4.8711	7.91
math/bias_half	2.6951	48.92	4.7500	7.91
math/bias_double	2.1260	50.70	4.9062	6.98
math/negative_bias	4.0568	44.40	4.4375	6.51
math/cross_bias_norm_matched	2.8256	48.14	4.7188	8.37
math/combo	1.8720	53.79	5.3398	10.23
math/combo_bias_only	3.0268	47.37	4.6875	6.98
math/combo_residual_only	1.8728	53.70	5.2773	8.84
math/radial_bias	3.0736	47.22	4.6953	6.98
math/tangent_bias	2.4137	49.99	4.8750	7.91
math/permutated_bias_10	3.1098	47.16	4.7266	7.91
math/permutated_bias_11	3.0548	47.61	4.7344	6.98
math/permutated_bias_12	3.0768	47.13	4.7070	8.37
kiCad/random_rowspace_reader	1.4527	68.87	7.8438	29.46
kiCad/constant_reader	1.4326	69.49	7.8398	31.25
kiCad/reader_residual	1.6665	67.75	7.4336	26.79
kiCad/svd1	1.4409	69.10	7.8867	33.93
kiCad/native	2.1279	61.47	6.3633	13.39
kiCad/fusion	1.3409	71.85	8.3359	43.75
kiCad/fusion_mean	1.4441	69.28	7.8906	31.70
kiCad/fusion_centered	1.6509	67.87	7.4805	27.23
kiCad/bias	1.6889	67.28	7.3477	23.66
kiCad/bias_half	1.8324	64.98	6.9688	18.75
kiCad/bias_double	1.6219	66.51	7.3203	21.43
kiCad/negative_bias	3.1794	51.59	5.1094	4.91
kiCad/cross_bias_norm_matched	1.9988	62.21	6.5156	15.62
kiCad/combo	1.3399	71.79	8.3320	43.75
kiCad/combo_bias_only	2.0882	62.18	6.5195	14.29
kiCad/combo_residual_only	1.3421	71.88	8.3594	44.20
kiCad/radial_bias	2.1272	61.65	6.4375	13.84
kiCad/tangent_bias	1.6883	67.07	7.3047	23.21
kiCad/permutated_bias_10	2.1647	61.26	6.4336	12.50
kiCad/permutated_bias_11	2.1686	61.11	6.4414	13.39
kiCad/permutated_bias_12	2.1339	61.32	6.3242	12.50

Table 82: Causal analysis of learned context steering: Results.

Domain	Cos(b, μ)	Fusion mean energy %	Combined mean energy %	Radial bias energy %	RMS linearization relative error, median
math	0.5781	92.61	92.60	0.17	0.0138
kiCad	0.4271	90.33	90.13	0.23	0.0173

3348
3349
3350
3351
3352
3353
3354
3355
3356
3357
3358
3359
3360
3361
3362
3363
3364
3365
3366
3367
3368
3369
3370
3371
3372
3373
3374
3375
3376
3377
3378
3379
3380
3381
3382
3383
3384
3385
3386
3387
3388
3389
3390
3391
3392
3393
3394
3395
3396
3397
3398
3399
3400
3401

Table 83: Causal analysis of learned context steering: Results.

Domain	Reader held-out mean	Reader held-out CV	SVD1 input scalar CV	CPU calibration seconds
math	0.9730	0.1313	0.2637	0.65
kicad	1.0142	0.1008	0.3157	9.34

Table 84: Causal analysis of learned context steering: Results.

Domain	Learned-rowspace reader CV	Random-rowspace reader CV
math	0.1313	0.1912
kicad	0.1008	0.2042

Table 85: Causal analysis of learned context steering: Results.

Domain / intervention	Retained fusion CE gain %	CE reduction vs native, paired 95% interval
math/constant_reader	91.0	1.0923 [0.8995, 1.2978]
math/random_rowspace_reader	89.6	1.0762 [0.8829, 1.2794]
math/reader_residual	25.0	0.2997 [0.1913, 0.4216]
math/svd1	90.6	1.0877 [0.8960, 1.2909]
math/fusion_mean	90.2	1.0833 [0.8896, 1.2877]
math/fusion_centered	26.9	0.3231 [0.1855, 0.4717]
math/bias	55.0	0.6598 [0.5514, 0.7756]
math/combo	100.1	1.2017 [0.9807, 1.4335]
math/radial_bias	0.0	0.0001 [-0.0024, 0.0027]
math/tangent_bias	55.0	0.6600 [0.5514, 0.7755]
math/cross_bias_norm_matched	20.7	0.2481 [0.1777, 0.3280]
kicad/constant_reader	88.3	0.6952 [0.5834, 0.8130]
kicad/random_rowspace_reader	85.8	0.6752 [0.5638, 0.7919]
kicad/reader_residual	58.6	0.4614 [0.3672, 0.5639]
kicad/svd1	87.3	0.6869 [0.5675, 0.8135]
kicad/fusion_mean	86.9	0.6838 [0.5736, 0.7999]
kicad/fusion_centered	60.6	0.4770 [0.3819, 0.5806]
kicad/bias	55.8	0.4390 [0.3523, 0.5315]
kicad/combo	100.1	0.7880 [0.6523, 0.9311]
kicad/radial_bias	0.1	0.0007 [-0.0012, 0.0028]
kicad/tangent_bias	55.9	0.4395 [0.3530, 0.5322]
kicad/cross_bias_norm_matched	16.4	0.1291 [0.0971, 0.1644]

Table 86: Causal analysis of learned context steering: Results.

Domain / rank-one control	CE excess over full fusion	Paired 95% interval
math/constant_reader	0.1084	[0.0656, 0.1564]
math/random_rowspace_reader	0.1245	[0.0807, 0.1727]
math/svd1	0.1131	[0.0721, 0.1530]
kicad/constant_reader	0.0917	[0.0527, 0.1372]
kicad/random_rowspace_reader	0.1118	[0.0705, 0.1587]
kicad/svd1	0.1000	[0.0615, 0.1517]

Table 87: Causal analysis of learned context steering: Results.

Domain	Minimum finite radial scale	Radial sign crossings %
math	0.99871	0.00000
kicad	0.99769	0.00000

3402
3403
3404
3405
3406
3407
3408
3409
3410
3411
3412
3413
3414
3415
3416
3417
3418
3419
3420
3421
3422
3423
3424
3425
3426
3427
3428
3429
3430
3431
3432
3433
3434
3435
3436
3437
3438
3439
3440
3441
3442
3443
3444
3445
3446
3447
3448
3449
3450
3451
3452
3453
3454
3455

Table 88: Causal analysis of learned context steering: Results.

Domain	Top-two singular-direction weight energy, fusion %	Joint-fit matrix %
math	99.924	99.926
kicad	98.460	98.399

S.18 GSM8K AND KiCAD: ACCEPT-UNTIL-FAIL FUSION ADAPTATION

Historical study date: 2026-09-10. Standalone inference always uses Qwen3-4B with the corresponding target LoRA active.

Table 89: GSM8K and KiCad: accept-until-fail fusion adaptation: Results.

Domain / variant	tokens/s	$\times$ native [paired 95% CI]	$\times$ uniform [paired 95% CI]	τ	draft-only	accepted/ proposed
math/native	121.85	1.0000 [1.0000, 1.0000]	0.8282 [0.8085, 0.8468]	4.4771	3.4771	23.18%
math/uniform	147.12	1.2074 [1.1809, 1.2368]	1.0000 [1.0000, 1.0000]	5.4482	4.4482	29.65%
math/auf	152.18	1.2490 [1.2216, 1.2795]	1.0344 [1.0229, 1.0460]	5.6604	4.6604	31.07%
kiCAD/native	159.37	1.0000 [1.0000, 1.0000]	0.7896 [0.7826, 0.7970]	5.9526	4.9526	33.02%
kiCAD/uniform	201.83	1.2665 [1.2547, 1.2778]	1.0000 [1.0000, 1.0000]	7.5045	6.5045	43.36%
kiCAD/auf	210.80	1.3227 [1.3067, 1.3382]	1.0444 [1.0400, 1.0488]	7.8209	6.8209	45.47%

S.19 GSM8K / KiCad AUF: TRAINING-DURATION CHECKPOINTS ON 4,096 UNIQUE EXAMPLES

Historical study date: 2026-09-10.

Reuse the same 4,096 cached active-target examples/domain and 4096 response cap. Counts denote cumulative example presentations, not unique prompts. No new decoding/capture. Initialize fusion-r56 afresh from native DFlash, PEFT random A/zero B, seed42. AUF training keeps 8 sampled masked-block anchors/example, block16 with clean anchor excluded, $\alpha/r=1$ and 860,160 trainables; drafter body, norms, embedding/head and target frozen.

Table 90: GSM8K / KiCad AUF: training-duration checkpoints on 4,096 unique examples: Results.

Domain / checkpoint	tokens/s	$\times$ native	$\times$ uniform	$\times$ prior AUF	τ	draft-only	accepted/proposed	Training min / GPU-hours
math/native	121.85	1.0000	0.8282	0.8007	4.4771	3.4771	23.18%	reused
math/uniform	147.12	1.2074	1.0000	0.9667	5.4482	4.4482	29.65%	reused
math/auf_12288	152.18	1.2490	1.0344	1.0000	5.6604	4.6604	31.07%	reused
math/auf_8000	152.77	1.2538	1.0384	1.0038	5.6587	4.6587	31.06%	1.94 / 0.065
math/auf_16000	153.44	1.2593	1.0430	1.0083	5.6914	4.6914	31.28%	3.83 / 0.128
kiCad/native	159.37	1.0000	0.7896	0.7560	5.9526	4.9526	33.02%	reused
kiCad/uniform	201.83	1.2665	1.0000	0.9575	7.5045	6.5045	43.36%	reused
kiCad/auf_12288	210.80	1.3227	1.0444	1.0000	7.8209	6.8209	45.47%	reused
kiCad/auf_8000	210.11	1.3184	1.0410	0.9967	7.7954	6.7954	45.30%	5.37 / 0.179
kiCad/auf_16000	213.37	1.3389	1.0572	1.0122	7.9123	6.9123	46.08%	10.51 / 0.350

3564
3565
3566
3567
3568
3569
3570
3571
3572
3573
3574
3575
3576
3577
3578
3579
3580
3581
3582
3583
3584
3585
3586
3587
3588
3589
3590
3591
3592
3593
3594
3595
3596
3597
3598
3599
3600
3601
3602
3603
3604
3605
3606
3607
3608
3609
3610
3611
3612
3613
3614
3615
3616
3617

S.20 KiCad AUF: EXPANDED UNIQUE TRAINING DATA

Historical study date: 2026-09-10. Target: Qwen3-4B with the pinned KiCad LoRA active. Reuse the canonical KiCad prompt/split contract and 4096-response-token cap. The eligible deduplicated training pool contains 15,366 prompts: 4,096 cached plus 11,270 newly generated. Evaluation uses the same 128 held-out prompts and 8192 response-token cap as the preceding study. Thinking disabled; greedy vLLM on L40S.

Table 91: KiCad AUF: expanded unique training data: Results.

Checkpoint	Tokens/s	x native	x uniform	Mean acceptance length	Fit seconds
8000	210.31	1.3196	1.0420	7.7954	469.2
15366	212.88	1.3358	1.0547	7.8950	944.9

S.21 NANOCODER: DENSE CAUSAL NEXT-TOKEN AND HIDDEN-STATE OBJECTIVES

Historical study date: 2026-09-10. **Result.** The best new variant is **full_ce**, at **0.6690**× **native DFlash** (paired-prompt 95% interval 0.6536–0.6845). None of the new variants exceeds the requested 10% threshold. This is exploratory selection among nine variants on one fixed evaluation set, not independent confirmation of the winning configuration.

Table 92: NanoCoder: dense causal next-token and hidden-state objectives: Results.

Variant	Output tokens/s	Speedup vs native [95% CI]	Vs prior AUF	Acceptance length τ	Accepted draft-only length	Accepted/ proposed	Training wall / GPU-hours
native	139.72	1.0000 × [1.0000, 1.0000]	0.9340 ×	5.0415	4.0415	26.94%	reused checkpoint
auf	149.60	1.0707 × [1.0570, 1.0860]	1.0000 ×	5.4251	4.4251	29.50%	reused checkpoint
linear56_ce	82.65	0.5915 × [0.5763, 0.6070]	0.5525 ×	2.9287	1.9287	12.86%	41.64 min / 1.388
linear56_mse	37.15	0.2659 × [0.2513, 0.2812]	0.2483 ×	1.2900	0.2900	1.93%	37.40 min / 1.247
linear56_ce_mse	69.09	0.4945 × [0.4772, 0.5126]	0.4619 ×	2.4348	1.4348	9.57%	41.06 min / 1.369
full_ce	93.47	0.6690 × [0.6536, 0.6845]	0.6248 ×	3.3212	2.3212	15.47%	41.87 min / 1.396
full_mse	41.61	0.2978 × [0.2823, 0.3145]	0.2781 ×	1.4532	0.4532	3.02%	37.28 min / 1.243
full_ce_mse	64.45	0.4613 × [0.4429, 0.4809]	0.4308 ×	2.2712	1.2712	8.47%	41.63 min / 1.388
mlp56_ce	84.74	0.6065 × [0.5919, 0.6215]	0.5664 ×	3.0192	2.0192	13.46%	40.91 min / 1.364
mlp56_mse	37.33	0.2672 × [0.2531, 0.2821]	0.2495 ×	1.3034	0.3034	2.02%	36.81 min / 1.227
mlp56_ce_mse	69.10	0.4946 × [0.4777, 0.5122]	0.4619 ×	2.4474	1.4474	9.65%	41.07 min / 1.369

S.22 FINAL AUF FUSION MAPPER MECHANISM PROBES

Historical study date: 2026-09-11. This separate study investigates the final individual fusion-r56 AUF mappers and the one pooled fusion-r56 AUF mapper. It does not train new mappers or measure live throughput. The body-LoRA comparison is a separate experiment.

Table 93: Final AUF fusion mapper mechanism probes: Results.

Domain	Checkpoint	Intervention	Mean CE	Fixed-prefix draft length	First-failure repair fraction	Prefix regression fraction	New training min
math	native	none	3.0737	4.6875	0.0000	0.0000	0
math	individual	full	2.1050	5.4297	0.3206	0.0938	0
math	individual	mean	2.2545	5.3203	0.3254	0.0781	0
math	individual	centered	2.8278	4.9219	0.1005	0.0469	0
math	individual	svd2	2.1086	5.4648	0.3301	0.0898	0
math	individual	without_svd2	3.0611	4.6836	0.0096	0.0078	0
math	individual	negative	4.9029	2.5117	0.0766	0.5469	0
math	individual	random_10	3.1415	4.5820	0.1531	0.1445	0
math	individual	random_11	3.1729	4.3789	0.1340	0.1719	0
math	individual	random_12	3.3019	4.4922	0.1100	0.1445	0
math	pooled	full	2.1220	5.5156	0.3206	0.0898	0
math	pooled	mean	2.2907	5.2500	0.3014	0.0898	0
math	pooled	centered	2.8257	5.0352	0.1340	0.0664	0
math	pooled	svd2	2.4344	5.2383	0.2727	0.0508	0
math	pooled	without_svd2	2.5335	5.1172	0.2632	0.0820	0
math	pooled	negative	4.8673	2.5625	0.0526	0.5312	0
math	pooled	random_10	3.1609	4.5391	0.1675	0.1875	0
math	pooled	random_11	3.1680	4.2891	0.1388	0.1797	0
math	pooled	random_12	3.3186	4.4648	0.1292	0.1641	0
kiCAD	native	none	2.1279	6.3633	0.0000	0.0000	0
kiCAD	individual	full	1.6347	8.4688	0.5306	0.0195	0
kiCAD	individual	mean	1.7552	8.0156	0.4745	0.0430	0
kiCAD	individual	centered	1.6873	7.7422	0.3980	0.0234	0
kiCAD	individual	svd2	1.7035	8.1875	0.5102	0.0312	0
kiCAD	individual	without_svd2	1.8947	7.0430	0.2449	0.0352	0
kiCAD	individual	negative	6.1063	1.0664	0.0051	0.8203	0
kiCAD	individual	random_10	2.9539	4.6328	0.0765	0.4180	0
kiCAD	individual	random_11	2.7715	4.5117	0.0459	0.4023	0
kiCAD	individual	random_12	2.7874	4.7930	0.1020	0.4258	0
kiCAD	pooled	full	1.6746	8.4141	0.5306	0.0195	0
kiCAD	pooled	mean	1.7743	7.9727	0.4745	0.0430	0
kiCAD	pooled	centered	1.7543	7.6797	0.3673	0.0234	0
kiCAD	pooled	svd2	1.7457	8.0508	0.4694	0.0312	0
kiCAD	pooled	without_svd2	1.8533	7.2344	0.2602	0.0273	0
kiCAD	pooled	negative	5.7360	1.2969	0.0000	0.7930	0
kiCAD	pooled	random_10	2.7080	5.1680	0.0918	0.3320	0
kiCAD	pooled	random_11	2.6625	4.6836	0.0561	0.4102	0
kiCAD	pooled	random_12	2.6760	4.9336	0.1224	0.3984	0
nanocoder	native	none	2.3808	4.4297	0.0000	0.0000	0
nanocoder	individual	full	2.2544	5.1836	0.3119	0.0742	0
nanocoder	individual	mean	2.3029	4.8711	0.2294	0.0703	0
nanocoder	individual	centered	2.3054	4.9453	0.2064	0.0391	0
nanocoder	individual	svd2	2.2595	5.1992	0.3073	0.0664	0
nanocoder	individual	without_svd2	2.3725	4.5000	0.0367	0.0039	0
nanocoder	individual	negative	2.8634	3.7344	0.0642	0.3086	0
nanocoder	individual	random_10	2.4292	4.4023	0.1284	0.1133	0
nanocoder	individual	random_11	2.4378	4.4219	0.0917	0.1172	0
nanocoder	individual	random_12	2.4002	4.3906	0.0872	0.1250	0
nanocoder	pooled	full	2.2846	5.2383	0.2936	0.0625	0
nanocoder	pooled	mean	2.3235	4.8320	0.1972	0.0703	0
nanocoder	pooled	centered	2.3102	4.7227	0.1651	0.0469	0
nanocoder	pooled	svd2	2.2915	4.7109	0.1606	0.0352	0
nanocoder	pooled	without_svd2	2.3806	4.9336	0.2248	0.0742	0
nanocoder	pooled	negative	2.7937	3.7344	0.0550	0.2969	0
nanocoder	pooled	random_10	2.4250	4.2812	0.0963	0.1250	0
nanocoder	pooled	random_11	2.4666	4.4375	0.1193	0.1055	0
nanocoder	pooled	random_12	2.3977	4.3789	0.0872	0.1250	0

S.23 MULTI-LoRA MIXTURE WITH MATCHED SAMPLED-AUF FUSION MAPPERS

Historical study date: 2026-09-11. One BF16 Qwen3-4B target and one frozen DFlash drafter; active GSM8K,KiCad,NanoCoder LoRAs. All mapper checkpoints are reused: fusion-r56,alpha56,4096 unique examples/domain,4096 response-token cap,8 sampled clean-anchor+15-mask blocks,2000 optimizer updates/16000 presentations. AUF loss $L=\text{sum}(m*w*CE)/\text{sum}(m*w)$, detached strict-prefix correctness weights,first failure included. Native random-A/zero-B initialization,AdamW $1e-4$,betas(.9,.999),eps $1e-8$,wd0,clip1,100-step warmup/cosine to2000,seed42,globalbatch8 across2GPUs. Frozen draft body/norms/embed/head. Target feature layers[1,9,17,25,33],dense cache; sampled loss. Checkpoint paths/hashes and measured fitting costs in setup/selection.json. No mapper retraining in this deployment study.

Table 94: Multi-LoRA mixture with matched sampled-AUF fusion mappers: Results.

Domain	Mapper training wall min (2 GPUs)
math	3.83
kicad	10.51
nanocoder	4.54

Table 95: Multi-LoRA mixture with matched sampled-AUF fusion mappers: Results.

Repeat	Speedup vs native	Native τ	Mapped τ
0	1.3176x	5.855	7.678
1	1.3288x	5.855	7.678

S.24 LEARNED TARGET-LORA ROUTER WITH MATCHED SAMPLED-AUF FUSION MAPPERS

Historical study date: 2026-09-11. One BF16 Qwen3-4B target and one frozen DFlash drafter; active GSM8K,KiCad,NanoCoder LoRAs. All mapper checkpoints are reused: fusion-r56,alpha56,4096 unique examples/domain,4096 response-token cap,8 sampled clean-anchor+15-mask blocks,2000 optimizer updates/16000 presentations. AUF loss $L=\text{sum}(m*w*CE)/\text{sum}(m*w)$, detached strict-prefix correctness weights,first failure included. Native random-A/zero-B initialization,AdamW $1e-4$,betas(.9,.999),eps $1e-8$,wd0,clip1,100-step warmup/cosine to2000,seed42,globalbatch8 across2GPUs. Frozen draft body/norms/embed/head. Target feature layers[1,9,17,25,33],dense cache; sampled loss. Checkpoint paths/hashes and measured fitting costs in setup/selection.json. No mapper retraining in this deployment study.

Table 96: Learned target-LoRA router with matched sampled-AUF fusion mappers: Results.

Domain	Mapper training wall min (2 GPUs)
math	3.83
kicad	10.51
nanocoder	4.54

Table 97: Learned target-LoRA router with matched sampled-AUF fusion mappers: Results.

Repeat	Speedup vs native	Native τ	Mapped τ
0	1.3212x	5.856	7.680
1	1.3243x	5.856	7.680

3834
3835
3836
3837
3838
3839
3840
3841
3842
3843
3844
3845
3846
3847
3848
3849
3850
3851
3852
3853
3854
3855
3856
3857
3858
3859
3860
3861
3862
3863
3864
3865
3866
3867
3868
3869
3870
3871
3872
3873
3874
3875
3876
3877
3878
3879
3880
3881
3882
3883
3884
3885
3886
3887

S.25 BODY LORA: DFLASH DECAY CE VERSUS AUF

Historical study date: 2026-09-11. Body rank16/alpha32 on all attention and MLP projections, frozen fusion and norms; 4,587,520 parameters. 4,096 cached trajectories per domain, 4,096 response cap, eight anchors, 2,000 updates (16,000 presentations), global batch eight on two L40S GPUs, seed42, AdamW LR1e-4, 100-step warmup/cosine. Decay is $\exp(-(j-1)/7)$ over all valid proposal labels, normalized by weight sum. Final checkpoint; 128 heldout prompts, vLLM, caps2,048/8,192/2,048.

Table 98: Body LoRA: DFlash decay CE versus AUF: Results.

Target	Decay/native	Decay/AUF body	Decay/AUF mapper	Acceptance length	Fit minutes (2 GPUs)
math	1.2628x	0.9623x	1.0103x	5.7161	5.18
kicad	1.7973x	1.0341x	1.3447x	10.7627	13.47
nanocoder	1.0583x	0.9759x	1.0048x	5.3200	5.49

S.26 JOINT FUSION MAPPER AND DRAFTER-BODY AUF

Historical study date: 2026-09-11. Joint fusion-r56/alpha56 and body-r16/alpha32, 5,447,680 parameters; 4,096 unique trajectories per domain, 4,096 response cap, eight anchors, AUF, 2,000 updates (16,000 presentations), seed42, AdamW LR1e-4, batch eight on two L40S GPUs. Final checkpoint, frozen target, embedding/head and norms. Same128 heldout vLLM prompts per domain; caps2,048/8,192/2,048. Joint initialization has an extra adapter, so this is not a loss-only comparison.

Table 99: Joint fusion mapper and drafter-body AUF: Results.

Target	Joint/native	Joint/body	Joint/mapper	Acceptance length	Fit minutes (2 GPUs)
math	1.3081x	0.9968x	1.0466x	5.9439	5.37
kicad	1.7661x	1.0162x	1.3213x	10.5273	16.67
nanocoder	1.0919x	1.0069x	1.0368x	5.5085	5.53

S.27 NANOCODER SAMPLED-ANCHOR AUF MATCHED TO GSM8K/KiCAD

Historical study date: 2026-09-11. Fusion mapper only: r56,alpha56,860160 trainable parameters. Native DFlash body,norms,embedding/head frozen. Reuse4096 unique CodeAlpaca trajectories generated by Qwen3-4B with NanoCoder adapter active,greedy nonthinking,4096 response cap, EOS enabled. Exact prompts,IDs,token references and source locations/hashes are retained in setup and validation/cache.json. Target adapter revision f078bfc2722bdb7bf644906cc7d81662ab3de9d2; source contract inherits dflash_nanocoder_20260909. No new generation or feature extraction.

Table 100: NanoCoder sampled-anchor AUF matched to GSM8K/KiCad: Results.

Target / checkpoint	Speedup vs native	Acceptance length	Training wall min (2 GPUs)
GSM8K sampled AUF seen16000 (existing)	1.2593x	5.691	3.83
KiCad sampled AUF seen16000 (existing)	1.3389x	7.912	10.51
NanoCoder sampled AUF seen16000	1.0590x	5.356	4.54

S.28 POOLED AUF MAPPER: EXPLICIT-ID MIXTURE

Historical study date: 2026-09-11. One fusion-r56 alpha56 shared across GSM8K,KiCad, NanoCoder; target LoRAs active. Native and three-separate controls inherited from dflash_auf_serving_20260911, unchanged exact request manifests. New pooled and AR measurements: 12clients, 128requests/domain, 2repeats, greedy/nonthinking, caps2048/8192/2048. Router reused without training.

Table 101: Pooled AUF mapper: Explicit-ID mixture: Results.

Repeat	Mode	Tokens/s	Acceptance length	Speed vs fresh AR	Fit wall min (2 GPU's)
0	native (inherited)	1273.98	5.8553	6.5200	—
0	three separate (inherited)	1678.61	7.6782	8.5909	3.83 /10.51 /4.54
0	one pooled	1671.89	7.6170	8.5565	27.12
0	AR	195.39	—	1.0000	—
1	native (inherited)	1273.32	5.8553	6.5500	—
1	three separate (inherited)	1691.96	7.6782	8.7036	3.83 /10.51 /4.54
1	one pooled	1673.98	7.6170	8.6111	27.12
1	AR	194.40	—	1.0000	—

S.29 POOLED AUF MAPPER: LEARNED TARGET ROUTER

Historical study date: 2026-09-11. One fusion-r56 alpha56 shared across GSM8K,KiCad, NanoCoder; target LoRAs active. Native and three-separate controls inherited from dflash_auf_serving_20260911, unchanged exact request manifests. New pooled and AR measurements:12clients,128requests/domain,2repeats,greedy/nonthinking,caps2048/8192/2048. Router reused without training.

Table 102: Pooled AUF mapper: Learned target router: Results.

Repeat	Mode	Tokens/s	Acceptance length	Speed vs fresh AR	Fit wall min (2 GPUs)
0	native (inherited)	1280.52	5.8560	6.5518	—
0	three separate (inherited)	1691.87	7.6798	8.6565	3.83 /10.51 /4.54
0	one pooled	1682.03	7.6188	8.6061	27.12
0	AR	195.45	—	1.0000	—
1	native (inherited)	1285.61	5.8560	6.5729	—
1	three separate (inherited)	1702.59	7.6798	8.7048	3.83 /10.51 /4.54
1	one pooled	1695.52	7.6188	8.6687	27.12
1	AR	195.59	—	1.0000	—

S.30 AUF GEOMETRY, MARGINS AND PREFIX SURVIVAL

The following are fixed-prefix diagnostics, not live timings. Mean-energy fractions measure the squared constant component of corrections, separately on training and heldout response positions. Top-two weight energy concerns the matrix, not its feature outputs. Mean length-gain retention is $(L_{\text{mean}} - L_0)/(L_{\text{full}} - L_0)$. Confidence intervals resample whole heldout prefixes. Fixed-competitor correlations hold the native competing token fixed; maximum-competitor correlations allow the competitor to change. A positive predicted margin change does not guarantee correction of the first failure or preservation of earlier slots.

Table 103: AUF response geometry (percentages) and full-minus-native draft-length intervals.

Domain	Source	Train mean %	Heldout mean %	Top-two weight %	Mean length gain %	Full length gain 95% CI
math	individual	86.775	86.987	99.954	85.263	0.4258 / 1.0664
math	pooled	80.776	80.621	83.031	67.925	0.5352 / 1.1289
kicad	individual	91.058	91.290	96.266	78.479	1.7188 / 2.5117
kicad	pooled	85.868	86.326	83.031	78.476	1.6719 / 2.4414
nanocoder	individual	84.689	86.260	99.823	58.549	0.4883 / 1.0234
nanocoder	pooled	77.525	80.328	83.031	49.758	0.5273 / 1.0898

Table 104: Quantized finite differences at correction scales plus/minus 0.01.

Domain	Source	Native failures	Fixed rival corr.	Max rival corr.	Sign accuracy	Positive but unrepaired
math	individual	209	0.6714	0.5746	0.6699	39
math	pooled	209	0.6418	0.5013	0.6364	45
kicad	individual	196	0.7325	0.6876	0.7959	19
kicad	pooled	196	0.7579	0.6742	0.7755	16
nanocoder	individual	218	0.5158	0.4847	0.6697	30
nanocoder	pooled	218	0.4899	0.4758	0.6743	29

Table 105: Pooled correction-mean cosine across domains.

Domain pair	Cosine
math_kicad	0.4516
math_nanocoder	0.6834
kicad_nanocoder	0.3809

Table 106: Fixed-prefix agreement, target margin and survival probabilities; no verifier bonus.

Domain	Condition	Top1	Margin	P(L>=1)	P(L>=4)	P(L>=8)	P(L>=15)
math	native	0.4716	0.6482	0.8828	0.4648	0.2422	0.0586
math	individual_full	0.5441	1.5156	0.8945	0.5117	0.3203	0.0898
math	individual_mean	0.5168	1.1833	0.9062	0.5117	0.2969	0.0781
math	individual_centered	0.4969	1.0142	0.8828	0.4844	0.2695	0.0703
math	individual_svd2	0.5426	1.5075	0.8945	0.5117	0.3203	0.0898
math	individual_without_svd2	0.4734	0.6643	0.8828	0.4648	0.2383	0.0586
math	individual_negative	0.2963	-2.6891	0.7500	0.2578	0.0664	0.0117
math	individual_epsilon_plus	0.4725	0.6656	0.8789	0.4609	0.2422	0.0586
math	individual_epsilon_minus	0.4710	0.6240	0.8828	0.4648	0.2422	0.0586
math	individual_random_10	0.4633	0.3073	0.8594	0.4609	0.2305	0.0625
math	individual_random_11	0.4490	0.3033	0.8672	0.4258	0.2148	0.0547
math	individual_random_12	0.4520	0.1976	0.8750	0.4570	0.2227	0.0625
math	pooled_full	0.5382	1.4983	0.8984	0.5195	0.3203	0.0938
math	pooled_mean	0.5162	1.1259	0.8984	0.5117	0.2852	0.0742
math	pooled_centered	0.5031	1.0616	0.8789	0.4766	0.2852	0.0781

	Domain	Condition	Top1	Margin	P(L>=1)	P(L>=4)	P(L>=8)	P(L>=15)
4158								
4159	math	pooled_svd2	0.5215	1.4150	0.8945	0.5117	0.2812	0.0938
4160	math	pooled_	0.5025	1.0233	0.8945	0.4922	0.2734	0.0820
4161		without_svd2						
4162	math	pooled_negative	0.2957	-2.6611	0.7422	0.2617	0.0742	0.0117
4163	math	pooled_epsilon_	0.4725	0.6653	0.8828	0.4688	0.2461	0.0586
4164		plus						
4165	math	pooled_epsilon_	0.4704	0.6282	0.8828	0.4648	0.2383	0.0586
4166		minus						
4167	math	pooled_	0.4600	0.2263	0.8711	0.4609	0.2188	0.0586
4168		random_10						
4169	math	pooled_	0.4490	0.2950	0.8633	0.4219	0.1914	0.0586
4170		random_11						
4171	math	pooled_	0.4526	0.1525	0.8711	0.4492	0.2148	0.0703
4172		random_12						
4173	math	native	0.6147	2.1665	0.9492	0.5781	0.4102	0.1172
4174	math	individual_full	0.7208	4.3323	0.9453	0.6172	0.5430	0.4180
4175	math	individual_	0.6937	3.5820	0.9453	0.6055	0.5312	0.3164
4176		mean						
4177	math	individual_	0.6854	3.5241	0.9492	0.6055	0.5078	0.2656
4178		centered						
4179	math	individual_svd2	0.7070	3.9424	0.9453	0.6094	0.5273	0.3672
4180	math	individual_	0.6530	2.6867	0.9492	0.5938	0.4688	0.1758
4181		without_svd2						
4182	math	individual_	0.1751	-4.2051	0.4961	0.0781	0.0234	0.0000
4183		negative						
4184	math	individual_	0.6197	2.2197	0.9531	0.5859	0.4297	0.1250
4185		epsilon_plus						
4186	math	individual_	0.6111	2.1124	0.9453	0.5781	0.4102	0.1172
4187		epsilon_minus						
4188	math	individual_	0.5139	-0.0155	0.9141	0.4805	0.2461	0.0391
4189		random_10						
4190	math	individual_	0.5032	0.3192	0.9141	0.4648	0.2461	0.0312
4191		random_11						
4192	math	individual_	0.5133	0.3615	0.8672	0.4883	0.2812	0.0469
4193		random_12						
4194	math	pooled_full	0.7193	4.2551	0.9453	0.6133	0.5352	0.4219
4195	math	pooled_mean	0.6940	3.5689	0.9453	0.6055	0.5234	0.3203
4196	math	pooled_centered	0.6807	3.4700	0.9492	0.5977	0.5039	0.2734
4197	math	pooled_svd2	0.6952	3.7068	0.9453	0.6055	0.5312	0.3320
4198	math	pooled_	0.6548	2.9266	0.9492	0.6055	0.4727	0.2031
4199		without_svd2						
4200	math	pooled_negative	0.2011	-3.8661	0.5586	0.0898	0.0312	0.0000
4201	math	pooled_epsilon_	0.6203	2.2189	0.9492	0.5859	0.4297	0.1250
4202		plus						
4203	math	pooled_epsilon_	0.6111	2.1139	0.9453	0.5742	0.4102	0.1172
4204		minus						
4205	math	pooled_	0.5433	0.4680	0.9336	0.5195	0.3164	0.0625
4206		random_10						
4207	math	pooled_	0.5215	0.4983	0.8945	0.4922	0.2695	0.0352
4208		random_11						
4209	math	pooled_	0.5203	0.6065	0.8906	0.5000	0.2969	0.0391
4210		random_12						
4211	nanocoder	native	0.4845	0.8752	0.8125	0.4102	0.2344	0.0391
4212	nanocoder	individual_full	0.5218	1.2656	0.8945	0.4883	0.2812	0.0703
4213	nanocoder	individual_	0.5113	1.1396	0.8438	0.4414	0.2773	0.0703
4214		mean						
4215	nanocoder	individual_	0.5056	1.0563	0.8555	0.4570	0.2656	0.0664
4216		centered						
4217	nanocoder	individual_svd2	0.5236	1.2560	0.8984	0.4961	0.2773	0.0703
4218	nanocoder	individual_	0.4866	0.8921	0.8125	0.4219	0.2344	0.0430
4219		without_svd2						
4220	nanocoder	individual_	0.4043	-0.0755	0.7422	0.3633	0.2109	0.0234
4221		negative						
4222	nanocoder	individual_	0.4830	0.8826	0.8086	0.4102	0.2344	0.0430
4223		epsilon_plus						
4224	nanocoder	individual_	0.4827	0.8704	0.8086	0.4062	0.2344	0.0391
4225		epsilon_minus						
4226	nanocoder	individual_	0.4710	0.7014	0.8086	0.4141	0.2383	0.0469
4227		random_10						
4228	nanocoder	individual_	0.4770	0.7593	0.7812	0.4102	0.2461	0.0547
4229		random_11						
4230	nanocoder	individual_	0.4695	0.7314	0.7930	0.4062	0.2188	0.0586
4231		random_12						
4232	nanocoder	pooled_full	0.5218	1.1930	0.8867	0.4805	0.2891	0.0742
4233	nanocoder	pooled_mean	0.5056	1.1213	0.8359	0.4453	0.2617	0.0664
4234	nanocoder	pooled_centered	0.5011	1.0183	0.8438	0.4375	0.2578	0.0469
4235	nanocoder	pooled_svd2	0.5008	0.9771	0.8359	0.4297	0.2617	0.0586
4236	nanocoder	pooled_	0.5092	1.1101	0.8516	0.4570	0.2656	0.0664
4237		without_svd2						

4212	Domain	Condition	Top1	Margin	P(L>=1)	P(L>=4)	P(L>=8)	P(L>=15)
4213	nanocoder	pooled_negative	0.4139	0.0423	0.7422	0.3711	0.2031	0.0273
4214	nanocoder	pooled_epsilon_	0.4839	0.8812	0.8086	0.4102	0.2383	0.0391
4215		plus						
4216	nanocoder	pooled_epsilon_	0.4827	0.8701	0.8086	0.4102	0.2344	0.0391
4217		minus						
4218	nanocoder	pooled_	0.4746	0.7036	0.8086	0.4102	0.2227	0.0430
4219		random_10						
4220	nanocoder	pooled_	0.4803	0.7112	0.7930	0.4258	0.2422	0.0469
4221		random_11						
4222	nanocoder	pooled_	0.4713	0.7394	0.7891	0.4062	0.2305	0.0508
4223		random_12						
4224								
4225								
4226								
4227								
4228								
4229								
4230								
4231								
4232								
4233								
4234								
4235								
4236								
4237								
4238								
4239								
4240								
4241								
4242								
4243								
4244								
4245								
4246								
4247								
4248								
4249								
4250								
4251								
4252								
4253								
4254								
4255								
4256								
4257								
4258								
4259								
4260								
4261								
4262								
4263								
4264								
4265								